Универзитет у Београду
Електротехнички факултет



ДИПЛОМСКИ РАД

Реализација интерактивног алата са графичким корисничким интерфејсом за евалуацију мера перформанси система

ментор:
проф. др Јелица Протић

кандидат:
Данијел Асков

Београд, септембар 2026. године

Сажетак

У оквиру овог рада реализован је вишеплатформски педагошки софтверски алат са графичким корисничким интерфејсом за лако и интерактивно креирање модела система са редовима за чекање и евалуацију мера перформанси таквих система коришћењем (1) симулација базираних на дискретним догађајима и (2) добро познатих аналитичких метода. Поред тога, имплементирана је „шта-ако" анализа која омогућава кориснику да стекне увид у то како промене одређених параметара у моделу утичу на мере перформанси система.

Кључне речи: системи са редовима за чекање, евалуација мера перформанси, симулација базирана на дискретним догађајима, Џексонов алгоритам, Гордон-Њуелов алгоритам.

САДРЖАЈ

1 Увод **1**
1.1 Мотивација и циљ рада . . . 1
1.2 Структура рада . . . 3

2 Теоријска разматрања **4**
2.1 Основе вероватноће . . . 4
2.1.1 Важне расподеле . . . 8
2.2 Основе статистике . . . 14
2.3 Основе стохастичке анализе . . . 17
2.3.1 Марковљеви ланци у дискретном времену . . . 18
2.3.2 Марковљеви ланци у непрекидном времену . . . 23
2.3.3 Важни Марковљеви ланци . . . 25

3 Системи са опслужним центрима **29**
3.1 Системи са једним опслужним центром . . . 29
3.1.1 Мере перформанси . . . 30
3.1.2 M/M/1 систем . . . 31
3.1.3 M/M/m систем . . . 33
3.2 Мреже са опслужним центрима . . . 35
3.2.1 Мере перформанси . . . 38
3.2.2 Мреже са решењем у облику производа . . . 39

4 Симулација заснована на дискретним догађајима **45**
4.1 Мере перформанси . . . 46

5 Анализа захтева **49**
5.1 Графички кориснички интерфејс . . . 49
5.2 Функционални захтеви . . . 60
5.2.1 Основне операције . . . 60
5.2.2 Измена мреже . . . 63
5.2.3 Симулација . . . 67
5.2.4 Аналитичко решавање . . . 69
5.2.5 Мере перформанси . . . 70
5.2.6 „Шта-ако” анализа . . . 70
5.3 Нефункционални захтеви . . . 70

6 Пројектовање и имплементација апликације **72**
6.1 Пројектовање и коришћени пројектни обрасци 72
6.2 Програмски код и његова организација 75
6.3 CI/CD 80

7 Верификација и валидација апликације **83**
7.1 Креирање нове мреже 84
7.2 Додавање новог елемента мреже 84
7.3 Измена постојећег елемента мреже 85
7.4 Покретање аналитичког решавања 85
7.5 Преглед резултата аналитичког решавања 86
7.6 Покретање „Шта-ако" анализе базиране на аналитичком решавању мреже 86
7.7 Покретање интерактивне симулације 86

8 Закључак **99**

СПИСАК СЛИКА

2.1 *Случајна променљива, њена расподела и хистограм њене расподеле.* . . . 5
2.2 *Дијаграм преласка за једноставан МЛДВ.* . . . 19
2.3 *Дијаграм преласка за процес рађања и умирања.* . . . 26
2.4 *Суперпозиција Поасонових процеса.* . . . 28
2.5 *Декомпозиција Поасоновог процеса.* . . . 28

3.1 *Опслужни центар са m сервера.* . . . 29

6.1 *Пројектни образац Стање у контексту апликације.* . . . 74

7.1 *Главни прозор апликације.* . . . 84
7.2 *Избор ставке* Нова мрежа *из менија* Датотека. . . . 84
7.3 *Изглед апликације након избора ставке* Нова мрежа *из менија* Датотека. . 85
7.4 *Додавање новог елемента мреже.* . . . 88
7.5 *Промена оријентације елемента мреже.* . . . 89
7.6 *Измена параметра расподеле времена опслуживања сервисног центра.* . 90
7.7 *Покретање аналитичког решавања модела.* . . . 91
7.8 *Модални прозор након завршетка аналитичког решавања модела, са изабраним опслужним центром* Опслужни центар 1. . . . 92
7.9 *Модални прозор након завршетка аналитичког решавања модела, са изабраним опслужним центром* Опслужни центар 1 *и картицом* Вероватноће стања. . . . 92
7.10 *Модални прозор након завршетка аналитичког решавања модела, са изабраним опслужним центром* Улазно-излазни уређај. . . . 93
7.11 *Модални прозор након завршетка аналитичког решавања модела, са изабраним опслужним центром* Улазно-излазни уређај *и картицом* Вероватноће стања. . . . 93
7.12 *Модални прозор након завршетка аналитичког решавања модела, са изабраним опслужним центром* Диск. . . . 94
7.13 *Модални прозор након завршетка аналитичког решавања модела, са изабраним опслужним центром* Диск *и картицом* Вероватноће стања. . . . 94
7.14 *Модални прозор за преглед мера перформанси, при чему је изабрана картица* Графикон. . . . 95
7.15 *Модални прозор за преглед мера перформанси, при чему је изабрана картица* Табела. . . . 95
7.16 *Модални прозор за „шта-ако” анализу.* . . . 96
7.17 *Модални прозор за „шта-ако” анализу након покретања аналитичког решавања мреже.* . . . 96

7.18 *Модални прозор* Параметри симулације. 96
7.19 *Главни прозор апликације током извршавања интерактивне симулације модела.* 97
7.20 *Главни прозор апликације након притиска на дугме* Следећи догађај. . . . 97
7.21 *Главни прозор апликације након завршетка анимације преласка посла.* . 98

СПИСАК ТАБЕЛА

4.1 *Генерисање узорка и тежина за оцену мера перформанси чворова у мрежи.* 47
4.2 *Генерисање узорка и тежина за оцену мера перформанси мреже.* 47

5.1 *Значење термина у спецификацији захтева.* 49

Поглавље 1

УВОД

1.1 Мотивација и циљ рада

У савременим рачунарским, комуникационим и производним системима, процена *мера перформанси*[1] игра кључну улогу у пројектовању, оптимизацији и планирању ресурса. Мере перформанси које се често узимају у обзир јесу искоришћеност ресурса, време обраде, време чекања итд. Имајући у виду да сам систем није доступан у фази пројектовања, те се не могу изводити непосредна мерења над истим, до релевантних мера перформанси система најчешће се долази коришћењем *модела*.

Први корак у одређивању мера перформанси базираном на моделу јесте процес формализације током којег се креира математички прецизан опис реалног система на основу неформалног описа система (најчешће на природном језику). Неформални опис система укључује структурне и функционалне информације, као и мере перформанси система које су од интереса у конкретном случају. Формализација подразумева концептуализацију система, тј. прављење апстрактног и поједностављеног погледа на систем за задату намену. Два често коришћена типа концептуализације који се користе приликом одређивања мера перформанси система су:

- *Марковљев модел* — систем се посматра као *Марковљев процес* који се на врло интуитиван начин представља као граф чији чворови представљају стања у којима се систем може наћи, док су гране прелази између тих стања.

- *Модел мреже са опслужним центрима* — систем се представља као скуп *опслужних центара* повезаних гранама кроз које ентитети (корисници, послови, захтеви) пролазе. Уколико нема слободног сервера који може да га обради, ентитет чека у реду за чекање опслужног центра. Након обраде наставља свој пут кроз мрежу преласком на неки од опслужних центара са којима је тренутни опслужни центар повезан или, ако је мрежа отворена, напушта систем. Треба напоменути да се понашање (динамика) појединачних опслужних центара у оквиру мреже описује раније поменутим Марковљевим моделом за који се, баш из тог разлога, каже да је *семантички модел*, док се за модел мреже са опслужним центрима каже да је *синтаксички модел* или *модел вишег нивоа*, јер описује структуру и топологију система.

[1] Често се са истим значењем користе и термини *индикатори перформанси* и *показатељи перформанси*.

Након успостављања модела система, следећи корак јесте одређивање мера перформанси система применом одговарајућих *метода решавања модела*. У зависности од претпоставки уведених током формализације система, могу се применити методе које припадају следећим категоријама:

- *Аналитичке методе* — идеја је да се формални опис система преведе у једну или више једначина, одакле се могу добити тачне или приближне мере перформанси. Све аналитичке методе могу се, угрубо, поделити у две велике групе:
  - Методе које дају решења у затвореном облику — уколико је систем једначина једноставан или садржи неку врсту правилности која се да искористити, могу се извести формуле са коначним бројем операција које дају тачна решења. Битно ограничење приликом налажења решења у затвореном облику јесте што су претпоставке о систему прилично рестриктивне, те често није могуће моделовање иоле сложенијег реалног система на адекватан начин. Значајна предност метода које дају решења у затвореном облику јесте њихова *умерена рачунска сложеност*, која омогућава брзо израчунавање мера перформанси чак и за веће системе.
  - Методе које не дају решења у затвореном облику — уколико је систем једначина такав да не допушта решење у затвореном облику, примењују се различити итеративни алгоритми и алгоритми из нумеричке анализе који дају углавном апроксимативна решења (треба напоменути да постоје изузеци, тј. постоје методе у овој групи метода које дају тачна решења). Типично, ове методе имају *већу рачунску сложеност*.
- *Симулационе методе* — поједини модели су такви да још увек није пронађено аналитичко решење или пак постоји алгоритам али је његова рачунска комплексност превелика за практичну употребу. У таквим ситуацијама примењују се симулационе технике. Каже се да се приликом симулације модел „извршава", при чему се у току симулације прикупљају узорци чијом се статистичком анализом након завршетка симулације оцењују тражене мере перформанси. Симулационе методе су, по правилу, *флексибилније* у односу на аналитичке методе, јер не захтевају увођење превише рестриктивних претпоставки.
- *Хибридне методе* — развијени су и поступци за налажење решења модела у којима се на комплементаран начин комбинују предности различитих аналитичких и симулационих метода.

Перформансе рачунарских система (ПРС) је обавезан предмет у осмом семестру основних академских студија на Електротехничком факултету Универзитета у Београду на студијским програмима Софтверско инжењерство и Електротехника и рачунарство (модул Рачунарска техника и информатика). ПРС оспособљава студенте за горе описани процес моделовања рачунарских система и евалуације њихових перформанси на бази модела коришћењем различитих аналитичких метода од којих су неке обрађене и у овом раду. Додатно, у оквиру необавезног пројектног задатка од студената се очекује, између осталог, имплементација и неке симулационе методе што, имајући у виду да се симулационе методе експлицитно не обрађују на аудиторним предавањима и вежбама,

подразумева доста самосталног рада. Самим тим, један број студената избегавањем пројектног задатка никада не долази у контакт са практичном имплементацијом алгоритама који се презентују на предмету и нема прилику да на интерактиван начин, кроз експериментисање са различитим моделима и параметрима модела, стекне осећај и интуицију о томе како измене у моделу система утичу на мере перформанси система. Коришћење специјализованог образовног софтвера јесте у складу са IEEE препорукама изнетим у [1] и значајно доприноси унапређењу квалитета наставе. Како је ПРС један од најважнијих стручних предмета на факултету, циљ овог рада јесте реализација управо једног таквог педагошког софтверског алата који ће кроз адекватан графички кориснички интерфејс омогућити студентима да на лак и брз начин креирају моделе, подешавају релевантне параметре модела, покрећу аналитичке и симулационе методе решавања модела и анализирају добијене индикаторе перформанси.

1.2 Структура рада

У другом поглављу дат је кратак преглед свих важнијих појмова из математичке вероватноће и статистике, а затим су сажето објашњене основе стохастичке анализе која је незаобилазна када је у питању разумевање система са редовима за чекање. Треће поглавље посвећено је аналитичким методама за рачунање мера перформанси система са редовима за чекање. У четвртом поглављу обрађено је одређивање мера перформанси система са редовима за чекање коришћењем симулације засноване на дискретним догађајима. Пето поглавље садржи анализу захтева које је реализовани алат требало да испуни. У шестом поглављу описане су кључне одлуке које се односе на пројектовање и имплементацију алата којима је обезбеђено испуњење захтева из претходног поглавља. Седмо поглавље описује процес верификације и валидације алата. Осмо поглавље садржи закључак и неколико предлога за даљи развој алата.

Поглавље 2

ТЕОРИЈСКА РАЗМАТРАЊА

У наставку је дат кратак осврт на неке основне дефиниције и теореме из области вероватноће и статистике које ће се користити у остатку овог поглавља.

2.1 Основе вероватноће

При анализи случајних појава често је потребно да се исходи случајних експеримента изразе бројевима. Уместо да се сваки појединачни елементарни исход разматра појединачно, уводи се правило које сваком исходу придружује реалан број. На тај начин се добија математички погодна репрезентација случајних феномена, која олакшава њихово даље проучавање:

Дефиниција 2.1.1. Случајна променљива или **случајна величина** је пресликавање скупа свих исхода неког случајног експеримента у скуп реалних бројева.

Каже се да вредност случајне променљиве *одражава* резултат случајног експеримента, при чему пресликавање поменуто у дефиницији 2.1.1 не мора бити бијективно, као што је илустровано следећим примером:

Пример 2.1.2. Нека се новчић баца два пута. Тада је простор исхода дат са $\Omega = \{\text{ПП}, \text{ПГ}, \text{ГП}, \text{ГГ}\}$. Случајна променљива X која представља број палих писама дата је са: $X(\text{ПП}) = 0$, $X(\text{ПГ}) = 1$, $X(\text{ГП}) = 1$, $X(\text{ГГ}) = 2$. Дакле, елементарни исходи ПГ и ГП пресликавају се у исти реалан број, 1.

Случајној променљивој X може се придружити вероватноћа P_X, дефинисана на подскуповима скупа реалних бројева $\mathbb{R}$ на следећи начин:

$$P_X(B) = P(\{\omega \in \Omega \,|\, X(\omega) \in B\}), \quad B \subset \mathbb{R}, \tag{2.1}$$

или, краће $P_X(B) = P(X \in B)$. Функција дефинисана са (2.1) назива се **расподела вероватноће случајне променљиве** X или, краће, **расподела случајне променљиве** X. Ако је скуп свих могућих вредности случајне променљиве X дискретан (коначан или пребројив), онда се за случајну променљиву, односно њену расподелу, каже да је **дискретна**.

Дефиниција 2.1.3. Нека су $x_1, x_2, ...$ све могуће вредности *дискретне* случајне променљиве X са вероватноћама $p_i = P_X(X = x_i), i = 1, 2, ...$ Скуп уређених парова

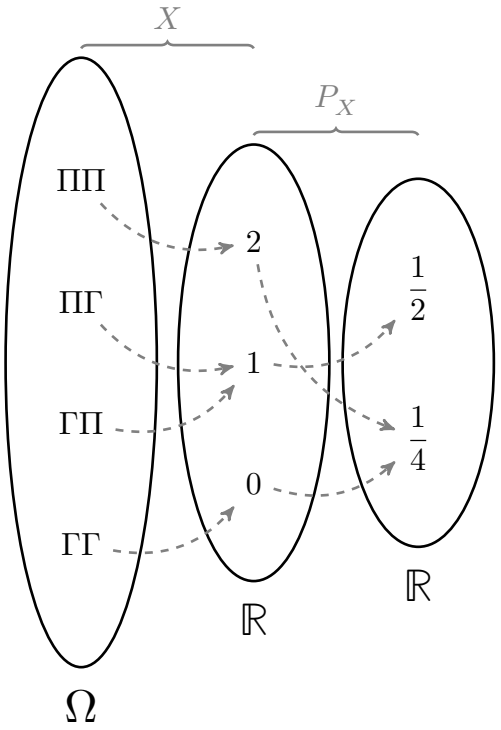
$\{(x_i, p_i) \mid i = 1, 2, ...\}$ или, нешто прегледније,

$$\begin{pmatrix} x_1 & x_2 & ... \\ p_1 & p_2 & ... \end{pmatrix},$$

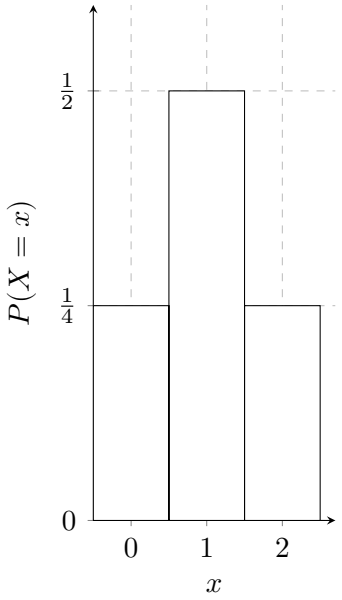
назива се **закон расподеле** дискретне случајне променљиве X.

Закон расподеле у потпуности одређује дискретну расподелу, а може се графички приказати *хистограмом*.

Пример 2.1.4. На слици 2.1а приказана је случајна променљива X из примера 2.1.2 и њена расподела, док је на слици 2.1б приказан хистограм расподеле те случајне променљиве.



(а) *Случајна променљива и њена расподела.*



(б) *Хистограм расподеле случајне променљиве.*

Слика 2.1: *Случајна променљива, њена расподела и хистограм њене расподеле.*

Приметимо да је укупна површина свих правоугаоника на хистограму једнака 1, што је и очекивано, јер је укупна вероватноћа свих могућих вредности случајне променљиве X једнака 1.

Треба напоменути да случајне променљиве које се користе у применама најчешће немају експлицитно дефинисан скуп елементарних исхода као у примеру 2.1.2.

У применама је пожељно имати *један број* којим се карактерише случајна променљива. Овакви бројеви који „резимирају" све могуће вредности случајне променљиве називају се *нумеричке карактеристике* случајне променљиве, односно одговарајуће расподеле. У наставку су дефинисане битне нумеричке карактеристике дискретних случајних променљивих које ће се користити у остатку овог поглавља.

Дефиниција 2.1.5. Нека је X дискретна случајна променљива са вредностима из скупа $\{x_1, x_2, ...\}$. Реалан број $\mathrm{E}(X)$ дефинисан као

$$\mathrm{E}(X) = \sum_i x_i P(X = x_i), \tag{2.2}$$

при чему је ред са десне стране једнакости апсолутно конвергентан, назива се **математичким очекивањем** дискретне случајне променљиве X. Поред наведене нотације, користи се и $\mathrm{E}\,X$.

У наставку је дефинисано неколико нумеричких карактеристика које се називају *мерама растӯрања* [1] случајне променљиве око математичког очекивања.

Дефиниција 2.1.6. Нека је X дискретна случајна променљива са математичким очекивањем $\mathrm{E}(X)$. **Варијанса** или **дисперзија** случајне променљиве X је математичко очекивање квадрата одступања случајне променљиве X од њеног математичког очекивања, тј. $\mathrm{Var}(X) = \mathrm{E}\left((X - \mathrm{E}(X))^2\right) = \sum_i (x_i - \mathrm{E}(X))^2 P(X = x_i)$.

Није једноставно дати интуитивно тумачење варијансе, јер варијанса има мерну јединицу која је квадрат мерне јединице случајне променљиве. Зато се користе још две нумеричке карактеристике: стандардна девијација, која има исту мерну јединицу као и случајна променљива, и коефицијент варијације, који је бездимензионална величина те омогућава поређење варијабилности случајних променљивих које имају различите мерне јединице.

Дефиниција 2.1.7. Квадратни корен варијансе случајне променљиве X назива се **стандардна девијација** случајне променљиве X, тј. $\sigma(X) = \sqrt{\mathrm{Var}(X)}$.

Дефиниција 2.1.8. Стандардна девијација случајне променљиве X подељена са њеним математичким очекивањем назива се **коефицијент варијације** случајне променљиве X, тј. $\mathrm{CV}(X) = \sigma(X)/\mathrm{E}(X)$.

Приметимо да су, за разлику од математичког очекивања, и варијанса и стандардна девијација увек ненегативни реални бројеви. Уколико је варијанса, односно стандардна девијација, једнака нули, онда ће случајна променљива имати само једну, потпуно предвидљиву вредност.

За случајне променљиве које нису дискретне закон расподеле није могуће користити, јер је вероватноћа да ће случајна променљива имати било коју конкретну вредност једнака нули. Показује се да је расподела сваке случајне променљиве (била она дискретна или не) у потпуности одређена својим вредностима на интервалима облика $(-\infty, x]$. Тако се добија функција једне реалне променљиве x, која се зове *функција расподеле* или *кумулативна функција расподеле*.

Дефиниција 2.1.9. Нека је X случајна променљива. Функција $x \mapsto F(x)$ дефинисана са

$$F(x) = P(X \leq x) = P(\{\omega \in \Omega \mid X(\omega) \leq x\}), -\infty < x < +\infty$$

назива се **функцијом расподеле** случајне променљиве X.

[1] Користе се и речи *расӯршеност*, *расејање*, *варијабилност*.

Посебно важне недискретне случајне промењљиве јесу непрекидне случајне промењљиве које дефинишемо коришћењем раније дефинисане функције расподеле:

Дефиниција 2.1.10. Нека је F функција расподеле случајне промењљиве X. Ако постоји ненегативна функција f дефинисана на $\mathbb{R}$ и таква да за свако $x \in \mathbb{R}$ важи да је

$$F(x) = \int_{-\infty}^{x} f(t)\,\mathrm{d}t,$$

онда се каже да је X **непрекидна случајна промењљива**. Функција f назива се **функција густине вероватноће** случајне промењљиве X, односно **функцијом густине** (или, краће, **густином**) одговарајуће расподеле.

Ако је случајна промењљива непрекидна, онда је њена функција расподеле такође непрекидна. Обрнуто не мора да важи, тј. непрекидност функције расподеле не гарантује да је случајна промењљива непрекидна. Случајна промењљива не мора да буде ни непрекидна ни дискретна.

Функција густине вероватноће непрекидне случајне промењљиве је аналогна закону расподеле дискретне случајне промењљиве. Тако, ако се упореди дефиниција математичког очекивања непрекидне случајне промењљиве која је дата у наставку са дефиницијом 2.1.5, лако се да приметити да је сума замењена интегралом, а закон расподеле — функцијом густине вероватноће:

Дефиниција 2.1.11. Реалан број $\mathrm{E}\,(X)$ дефинисан као

$$\mathrm{E}\,(X) = \int_{-\infty}^{+\infty} x f(x)\,\mathrm{d}x,$$

при чему је несвојствени интеграл са десне стране једнакости апсолутно конвергентан, назива се **математичким очекивањем** непрекидне случајне промењљиве X.

Као што ће постати јасно касније, веома важна јесте и функција која врши *инверзно* пресликавање у односу на функцију расподеле. Ако је функција расподеле F непрекидна и строго растућа, онда постоји јединствен реалан број x_* за који је $F(x_*) = p$, односно $x_* = F^{-1}(p)$. Тако дефинисан број x_* назива се *квантил реда* p посматране расподеле. Уколико F није свуда непрекидна или је константна на неким интервалима (дакле, није строго растућа), онда се може десити да једначина $F(x_*) = p$ нема ни једно решење по x_*, или да их има бесконачно много. Зато је општа дефиниција квантила нешто сложенија [2]:

Дефиниција 2.1.12. Нека је F функција расподеле неке случајне промењљиве. Сваки број x за који важе неједнакости

$$F(x_-) \leq p \ \wedge \ F(x) \geq p$$

где је $0 < p < 1$ назива се **квантил реда** p.

За примену теорије вероватноће у разумевању појава у реалном свету веома је битан појам *конвергенције* случајних промењљивих. Две врсте конвергенције на које ће се позивати текст у наставку су *конвергенција у вероватноћи* и *конвергенција у расподели*.

Дефиниција 2.1.13. Нека су $X_1, X_2, ...$ и X случајне променљиве. Низ $\{X_n\}$ **конвергира ка** X **у вероватноћи** ако је

$$\lim_{n\to\infty} P\left(|X_n - X| > \epsilon\right) = 0, \text{ за свако } \epsilon > 0.$$

Низ $\{X_n\}$ **конвергира ка** X **у расподели** или **слабо конвергира** ако је

$$\lim_{n\to\infty} P\left(X_n \leq x\right) = P\left(X \leq x\right)$$

у свакој тачки $x \in \mathbb{R}$ у којој је функција расподеле $P_X = F(x) = P(X \leq x)$ непрекидна.

У наставку су размотрене расподеле које су значајне у стохастичкој анализи.

2.1.1 Важне расподеле

Дискретне расподеле

Дефиниција 2.1.14. Бернулијев експеримент је случајни експеримент који има два исхода: успех и неуспех. Нека случајна променљива X има вредност 0 ако се догодио неуспех, а вредност 1 ако се догодио успех. Закон расподеле дат са:

$$\begin{pmatrix} 0 & 1 \\ 1-p & p \end{pmatrix}$$

назива се **Бернулијева расподела** са параметром p и пише се $X \sim \text{Bern}\left(p\right)$.

Нумеричке карактеристике Бернулијеве случајне променљиве су:

$$\text{E}\left(X\right) = p,\ \text{Var}\left(X\right) = p(1-p),\ \sigma\left(X\right) = \sqrt{p(1-p)},\ \text{CV}\left(X\right) = \sqrt{\frac{1-p}{p}}.$$

Дефиниција 2.1.15. Нека се Бернулијеви експерименти са вероватноћом успеха p понављају *до првог успеха*. Нека је X случајна променљива која представља број изведених експеримената до првог успеха (не рачунајући експеримент који је завршен успехом). Случајна променљива X има следећи закон расподеле:

$$\begin{pmatrix} 0 & 1 & 2 & ... \\ p & (1-p)p & (1-p)^2p & ... \end{pmatrix}$$

или, краће,

$$P\left(X = k\right) = (1-p)^k p, \quad k = 0, 1, 2, ... \tag{2.3}$$

Ова расподела се назива **геометријска расподела** са параметром p и пише се $X \sim \text{Geom}\left(p\right)$.

Често се користи и алтернативна дефиниција геометријске расподеле. Случајна променљива $Y = X + 1$ представља број изведених експеримената до првог успеха *укључујући* и експеримент који је завршен успехом, а има закон расподеле:

$$\begin{pmatrix} 1 & 2 & 3 & ... \\ p & (1-p)p & (1-p)^2p & ... \end{pmatrix}$$

или, краће,

$$P\left(Y=k\right)=(1-p)^{k-1}p,\quad k=1,2,3,... \tag{2.4}$$

За ову расподелу се такође користи назив **геометријска расподела** са параметром p и пише се $Y\sim\mathrm{Geom}\left(p\right)$.

У наставку ће се користити дефиниција (2.3), али се све нумеричке карактеристике могу лако израчунати и када се користи дефиниција (2.4).

Математичко очекивање случајне променљиве са геометријском расподелом налази се на основу дефиниције (2.2):

$$\mathrm{E}\left(X\right)=\sum_{k=1}^{\infty}k(1-p)^{k}p=p\sum_{k=1}^{\infty}k(1-p)^{k}.$$

Увођењем смене $q=1-p$ добија се:

$$\mathrm{E}\left(X\right)=p\sum_{k=1}^{\infty}kq^{k}=pq\sum_{k=1}^{\infty}kq^{k-1}=pq\cdot\frac{1}{(1-q)^{2}}.$$

Како је $1-q=p$, добија се:

$$\mathrm{E}\left(X\right)=p\cdot\frac{1-p}{p^{2}}=\frac{1-p}{p}. \tag{2.5}$$

Остале нумеричке карактеристике геометријске случајне променљиве су:

$$\mathrm{Var}\left(X\right)=\frac{1-p}{p^{2}},\ \sigma\left(X\right)=\frac{\sqrt{1-p}}{p},\ \mathrm{CV}\left(X\right)=\frac{1}{\sqrt{1-p}}.$$

Дефиниција 2.1.16. Случајна променљива X за коју важи:

$$P\left(X=k\right)=\frac{\lambda^{k}}{k!}\mathrm{e}^{-\lambda},\quad k=0,1,2,...,\lambda>0$$

назива се **Поасоновом случајном променљивом** са параметром λ, а њена расподела се назива **Поасонова расподела** са параметром λ и записује се $X\sim\mathrm{Pois}\left(\lambda\right)$.

Нумеричке карактеристике Поасонове случајне променљиве су:

$$\mathrm{E}\left(X\right)=\lambda,\ \mathrm{Var}\left(X\right)=\lambda,\ \sigma\left(X\right)=\sqrt{\lambda},\ \mathrm{CV}\left(X\right)=\frac{1}{\sqrt{\lambda}}.$$

Поасонова расподела се користи за моделовање броја догађаја који се дешавају у фиксном временском или просторном интервалу при чему се догађаји дешавају независно један од другог и зна се просечан број догађаја у том временском или просторном интервалу (λ). Број телефонских позива у јединици времена, број особа у неком реду, број аутобуса који долазе на стајалиште у јединици времена, број стабала по јединици површине у некој шуми, број радиоактивних распада неког материјала у јединици времена итд. могу се моделовати Поасоновом расподелом.

Непрекидне расподеле

Случајне променљиве са непрекидним расподелама наведеним у наставку користе се за моделовање различитих појава у физичким, биолошким, друштвеним и другим системима. У контексту система са опслужним центрима (видети главу 3) њима ће бити моделовано време између два узастопна приспећа захтева у систем, као и време опслуживања захтева на серверу опслужног центра.

Најједноставнија непрекидна случајна променљива јесте она која има увек исту, унапред познату вредност, тј. детерминистичка случајна променљива.

Дефиниција 2.1.17. Расподела са параметром k дефинисана помоћу густине

$$f(x) = \delta(x - k),$$

где је δ Диракова делта функција, назива се **детерминистичка расподела** са параметром $k \in \mathbb{R}$. Ако случајна променљива X има детерминистичку расподелу са параметром k, пише се $X \sim \mathrm{Det}\,(k)$ и каже да је X **детерминистичка случајна променљива**.

Интеграцијом густине детерминистичке расподеле добија се одговарајућа функција расподеле:

$$F(x) = \int_{-\infty}^{x} f(t)\,\mathrm{d}t = \begin{cases} 0, & x < k, \\ 1, & x \geq k. \end{cases}$$

У наставку су дате нумеричке карактеристике детерминистичке случајне променљиве:

$$\mathrm{E}\,(X) = k,\ \mathrm{Var}\,(X) = 0,\ \sigma\,(X) = 0,\ \mathrm{CV}\,(X) = 0.$$

За нијансу сложенија непрекидна случајна променљива јесте она која има једнаку вероватноћу да узме било коју вредност из неког интервала. Таква случајна променљива назива се униформна случајна променљива.

Дефиниција 2.1.18. Расподела са параметрима a и b, где је $a < b$, дефинисана помоћу густине

$$f(x) = \begin{cases} \dfrac{1}{b-a}, & a \leq x \leq b, \\ 0, & \text{иначе}, \end{cases}$$

назива се **униформна расподела** на интервалу (a, b). Ако случајна променљива X има униформну расподелу на интервалу (a, b), пише се $X \sim \mathrm{Unif}\,(a, b)$ и каже да је X **униформна случајна променљива**.

Функција расподеле униформне случајне променљиве добија се интеграцијом њене густине вероватноће:

$$F(x) = \int_{-\infty}^{x} f(t)\,\mathrm{d}t = \begin{cases} 0, & x < a, \\ \dfrac{x-a}{b-a}, & a \leq x \leq b, \\ 1, & x > b. \end{cases}$$

У наставку су дате нумеричке карактеристике униформне случајне променљиве:

$$\mathrm{E}\,(X) = \frac{a+b}{2},\ \mathrm{Var}\,(X) = \frac{(b-a)^2}{12},\ \sigma\,(X) = \frac{b-a}{\sqrt{12}},\ \mathrm{CV}\,(X) = \frac{b-a}{\sqrt{3}(a+b)}.$$

Најважнија непрекидна расподела у стохастичкој анализи и, уједно, најпогоднија расподела за аналитичко решавање многих модела, јесте експоненцијална расподела.

Дефиниција 2.1.19. Расподела са параметром $\lambda > 0$ дефинисана помоћу густине

$$f(x) = \begin{cases} \lambda\,\mathrm{e}^{-\lambda x}, & x \geq 0, \\ 0, & x < 0, \end{cases}$$

назива се **експоненцијална расподела**. Ако случајна променљива X има експоненцијалну расподелу са параметром λ, пише се $X \sim \mathrm{Exp}\,(\lambda)$ и каже да је X **експоненцијална случајна променљива**.

Функција расподеле експоненцијалне случајне променљиве добија се интеграцијом њене густине вероватноће:

$$F(x) = \int_{-\infty}^{x} f(t)\,\mathrm{d}t = \begin{cases} 1 - \mathrm{e}^{-\lambda x}, & x \geq 0, \\ 0, & x < 0. \end{cases}$$

На основу функције густине вероватноће добијају се нумеричке карактеристике експоненцијалне случајне променљиве:

$$\mathrm{E}\,(X) = \frac{1}{\lambda},\ \mathrm{Var}\,(X) = \frac{1}{\lambda^2},\ \sigma\,(X) = \frac{1}{\lambda},\ \mathrm{CV}\,(X) = 1.$$

Експоненцијалном расподелом моделује се велики број појава. Између осталог, дужина живота уређаја, односно време између два квара уређаја јесте случајна променљива са експоненцијалном расподелом; такође, време између доласка клијената на шалтер такође јесте експоненцијална случајна променљива; у контексту система са опслужним центрима, време обраде захтева на серверу опслужног центра, као и време између два узастопна приспећа захтева у опслужни центар јесу експоненцијалне случајне променљиве.

Параметар λ одређује се као реципрочна вредност просечног времена између два узастопна догађаја (шта год у конкретном моделу ти догађаји били). Најважнија карактеристика експоненцијалне расподеле јесте *особина одсуства меморије* (енг. *memoryless property*) коју нема ни једна друга непрекидна расподела (једина друга расподела која има ову особину јесте геометријска расподела која је дискретна). Нека случајна променљива X има експоненцијалну расподелу са параметром λ. Тада за сваки $s, t > 0$ важи:

$$P\,(X > s + t \mid X > s) = P\,(X > t)\,.$$

Горња једнакост се лако доказује:

$$
\begin{aligned}
P\left(X > s+t \mid X > s\right) &= \frac{P\left(X > s+t \wedge X > s\right)}{P\left(X > s\right)} \\
&= \frac{P\left(X > s+t\right)}{P\left(X > s\right)} \\
&= \frac{1-F(s+t)}{1-F(s)} \\
&= \frac{\mathrm{e}^{-\lambda(s+t)}}{\mathrm{e}^{-\lambda s}} \\
&= \mathrm{e}^{-\lambda t} \\
&= 1-F(t) \\
&= P\left(X > t\right).
\end{aligned}
$$

Међутим, као што је примећено у [3], емпиријски подаци показују да експоненцијална расподела није увек добра апроксимација за расподеле у реалним системима. На пример, коефицијент варијације времена сервисирања процесора је често већи од 1, док је коефицијент варијације времена обраде на периферним уређајима обично мањи од 1. Зато, у наставку су дате још неке непрекидне расподеле које се користе у стохастичкој анализи.

За апроксимацију емпиријских расподела са коефицијентом варијације већим од 1 често се користи хиперекспоненцијална расподела. Она представља генерализацију експоненцијалне расподеле тако што се уместо експоненцијалне расподеле са једним фиксним параметром λ користи експоненцијална расподела са случајно изабраним параметром из k-торке $(\lambda_1, \lambda_2, \ldots, \lambda_k)$, при чему одговарајуће вероватноће избора износе $(p_1, p_2, \ldots, p_k)$, где је $\sum_{i=1}^{k} p_i = 1$ и $p_i \geq 0$ за сваки $i = 1, 2, \ldots, k$.

Дефиниција 2.1.20. Функција расподеле дефинисана помоћу густине

$$
f(x) = \begin{cases} \displaystyle\sum_{i=1}^{k} p_i \lambda_i \mathrm{e}^{-\lambda_i x}, & x \geq 0, \\ 0, & x < 0, \end{cases}
$$

назива се **хиперекспоненцијална расподела** са параметрима $(p_1, p_2, \ldots, p_k)$ и $(\lambda_1, \lambda_2, \ldots, \lambda_k)$. Случајна променљива која има хиперекспоненцијалну расподелу назива се **хиперекспоненцијалном случајном променљивом**.

Нумеричке карактеристике хиперекспоненцијалне случајне променљиве X су дате у наставку:

$$
\mathrm{E}\left(X\right) = \sum_{i=1}^{k} \frac{p_i}{\lambda_i}, \mathrm{Var}\left(X\right) = 2\sum_{i=1}^{k} \frac{p_i}{\lambda_i^2} - \left(\sum_{i=1}^{k} \frac{p_i}{\lambda_i}\right)^2, \sigma\left(X\right) = \sqrt{\mathrm{Var}\left(X\right)}, \mathrm{CV}\left(X\right) = \frac{\sigma\left(X\right)}{\mathrm{E}\left(X\right)}.
$$

Емпиријске расподеле са коефицијентом варијације мањим од 1 могу се апроксимирати Ерланговом расподелом. Док се код хиперекспоненцијалне расподеле на случајан начин бира једна експоненцијална расподела из скупа могућих експоненцијалних расподела са различитим параметрима, Ерлангова расподела подразумева k секвенцијалних експоненцијалних „фаза” са истим параметром λ.

Дефиниција 2.1.21. Расподела са параметрима k и λ, где је k позитиван цео број, а $\lambda > 0$, дефинисана помоћу густине

$$f(x) = \begin{cases} \dfrac{\lambda^k x^{k-1}}{(k-1)!} \mathrm{e}^{-\lambda x}, & x \geq 0, \\ 0, & x < 0, \end{cases}$$

назива се **Ерлангова расподела**. Случајна променљива која има Ерлангову расподелу назива се **Ерланговом случајном променљивом**.

Јасно је да, ако је $Y_i \sim \mathrm{Exp}\,(\lambda)$, $i = 1, 2, \ldots, k$, онда је $Y = \sum_{i=1}^{k} Y_i$ Ерлангова случајна променљива са параметрима k и λ. Нумеричке карактеристике Ерлангове случајне променљиве X су дате у наставку:

$$\mathrm{E}\,(X) = \frac{k}{\lambda},\ \mathrm{Var}\,(X) = \frac{k}{\lambda^2},\ \sigma\,(X) = \frac{\sqrt{k}}{\lambda},\ \mathrm{CV}\,(X) = \frac{1}{\sqrt{k}}.$$

Једна важна расподела која је добила на значају у анализи саобраћаја у рачунарским мрежама јесте Паретова расподела.

Дефиниција 2.1.22. Расподела са параметрима $x_m > 0$ и $\alpha > 0$, дефинисана помоћу густине

$$f(x) = \begin{cases} \dfrac{\alpha x_m^{\alpha}}{x^{\alpha+1}}, & x \geq x_m, \\ 0, & x < x_m, \end{cases}$$

назива се **Паретова расподела**. Случајна променљива која има Паретову расподелу назива се **Паретовом случајном променљивом**.

Нумеричке карактеристике Паретове случајне променљиве X су дате у наставку:

$$\mathrm{E}\,(X) = \begin{cases} \dfrac{\alpha x_m}{\alpha - 1}, & \alpha > 1, \\ \infty, & \alpha \leq 1; \end{cases}$$

$$\mathrm{Var}\,(X) = \begin{cases} \dfrac{\alpha x_m^2}{(\alpha-1)^2(\alpha-2)}, & \alpha > 2, \\ \infty, & 1 < \alpha \leq 2; \end{cases}$$

$$\sigma\,(X) = \begin{cases} \sqrt{\dfrac{\alpha x_m^2}{(\alpha-1)^2(\alpha-2)}}, & \alpha > 2, \\ \infty, & 1 < \alpha \leq 2; \end{cases}$$

$$\mathrm{CV}\,(X) = \begin{cases} \sqrt{\dfrac{1}{\alpha(\alpha-2)}}, & \alpha > 2, \\ \infty, & 1 < \alpha \leq 2. \end{cases}$$

Паретова расподела је пример расподеле са „тешким” или „дугачким” репом (енгл. *heavy-tailed distribution, long-tailed distribution*) које се користе за моделовање појава у којима су екстремне вредности релативно вероватне.

2.2 Основе статистике

Проучавање случајних појава подразумева могућност *регистровања*[2], тј. *мерења* разних података [4]. На пример, у контексту мрежа са опслужним центрима бележи се број послова који се налазе у оквиру сваког опслужног центра, мери се време које протекне од доласка посла у опслужни центар до његовог одласка итд. Регистровани подаци зову се *статистички подаци* и представљају вредности неког *обележја* (често се ради о бројној вредности, као што је случај у наведеним примерима).

Измерене вредности неког обележја су независне случајне променљиве које имају идентичну, али непознату функцију расподеле F. Мерење се обично понавља n пута, а добијени скуп са n случајних променљивих [2] или n-торка случајних променљивих [4] назива се *независан (прост) узорак*, или, кратко, *узорак*. Број n назива се *обим узорка*. Математичка статистика се, између осталог, бави *оцењивањем* нумеричких карактеристика функције расподеле F. У општем случају, оцењивана нумеричка карактеристика расподеле F биће означена са θ, одговарајућа оцена биће означена са $\hat{\theta}$, а скуп свих могућих вредности оцењиване нумеричке карактеристике биће означен са Θ. Као што ће постати јасно у наставку текста, нумеричка карактеристика која се оцењује приликом одређивања индикатора перформанси статистичком обрадом узорака прикупљених током симулације мреже са опслужним центрима јесте *математичко очекивање расподеле одговарајућег индикатора перформанси*.

Оцењивање неке нумеричке карактеристике непознате расподеле на основу узорка подразумева коришћење *статистика*.

Дефиниција 2.2.1. Свака случајна променљива која је функција само од елемената узорка назива се **статистиком**.

Свака статистика *може*, у принципу, бити оцена непознате нумеричке карактеристике. Међутим, оцена има смисла само ако је у неком смислу *блиска* стварној вредности нумеричке карактеристике [2]. У наставку су формализована својства које оцена треба да има како би се сматрала добром.

Дефиниција 2.2.2. Нека је $\{X_1, X_2, \ldots, X_n\}$ узорак из расподеле F са неком нумеричком карактеристиком θ. Нека се статистика $\hat{\theta}$ користи као оцена за θ. Тада:

- $\hat{\theta}$ се назива **центрираном** или **непристрасном оценом** ако важи

$$\mathrm{E}\,\hat{\theta} = \theta,$$

  тј. ако је математичко очекивање статистике $\hat{\theta}$ једнако нумеричкој карактеристици θ.

- $\hat{\theta}$ се назива **стабилном оценом** ако $\hat{\theta}$ конвергира у вероватноћи ка θ када обим узорка n тежи бесконачности, тј. ако је

$$\lim_{n\to\infty} P(|\hat{\theta} - \theta| > \epsilon) = 0$$

  за свако $\theta \in \Theta$ и $\epsilon > 0$.

[2]Користе се и термини *бележење*, *посматрање*, *евидентирање*.

У наставку су дате дефиниције стандардне оцене математичког очекивања, варијансе и стандардне девијације.

Дефиниција 2.2.3. Нека је $\{X_1, X_2, \dots, X_n\}$ узорак из расподеле F са непознатим математичким очекивањем μ. Тада се статистика

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} X_i$$

назива **узорачком средином** и представља стандардну оцену математичког очекивања μ.

Дефиниција 2.2.4. Нека је $\{X_1, X_2, \dots, X_n\}$ узорак из расподеле F са непознатом варијансом σ^2. Тада се статистика

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \hat{\mu})^2$$

назива **узорачком варијансом** и представља стандардну оцену варијансе σ^2.

Дефиниција 2.2.5. Нека је $\{X_1, X_2, \dots, X_n\}$ узорак из расподеле F са непознатом стандардном девијацијом σ. Тада се статистика

$$s = \sqrt{s^2} = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(X_i - \hat{\mu})^2}$$

назива **узорачком стандардном девијацијом** и представља стандардну оцену стандардне девијације σ.

Лако се може показати да су наведене стандардне оцене центриране и стабилне. Оне се називају *тачкасте оцене*, јер дају само једну (приближну) бројну вредност. Међутим, у пракси је често корисније имати интервал који са великом вероватноћом садржи тачну вредност нумеричке карактеристике. То су тзв. *интервалне оцене* или *интервали поверења*.

Дефиниција 2.2.6. Нека је $\{X_1, X_2, \dots, X_n\}$ узорак из расподеле F са непознатом нумеричком карактеристиком θ. Нека су Y_1 и Y_2 статистике над дати узорком. Интервал $[Y_1, Y_2]$ назива се **интервалом поверења** за нумеричку карактеристику θ са нивоом поверења $1-\alpha$ $(0 < \alpha < 1)$ ако важи $P(\theta \in [Y_1, Y_2]) = 1 - \alpha$.

У применама се углавном употребљавају стандардне вредности нивоа поверења: 90%, 95% и 99% (дакле: $\alpha = 0{,}1$, $\alpha = 0{,}05$ и $\alpha = 0{,}01$, респективно). Најчешће тачно одређивање интервала поверења није могуће или је превише компликовано. Тада се одређују приближни интервали поверења који су базирани на централној граничној теореми. Централна гранична теорема даје асимптотску расподелу узорачке средине. Термин *централна* указује на то да ова теорема има централно место у теорији вероватноће и њеним применама, а обично се наводи у следећем облику[2]:

Теорема 2.2.7 (Централна гранична теорема)**.** *Нека је* $\{X_1, X_2, \ldots, X_n\}$ *независан узорак из расподеле* F *са математичким очекивањем* μ *и варијансом* σ^2*. Тада случајна променљива*

$$Z = \frac{\sum_{i=1}^{n} X_i - n\mu}{\sigma\sqrt{n}} \tag{2.6}$$

конвергира у расподели ка $\mathcal{N}(0,1)$*.*

Ако се узорачка средина означи са

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} X_i$$

и именилац и бројилац у разломку на десној страни (2.6) помноже са $1/n$ добија се:

$$Z = \frac{\hat{\mu} - \mu}{\sigma/\sqrt{n}}$$

Како Z конвергира у расподели ка $\mathcal{N}(0,1)$, то случајна променљива

$$\hat{\mu} = \frac{\sigma}{\sqrt{n}} Z + \mu$$

конвергира у расподели ка $\mathcal{N}(\mu, \sigma/\sqrt{n})$.

На основу централне граничне теореме могу се одредити интервали поверења за математичко очекивање μ када је позната варијанса σ^2:

Теорема 2.2.8. *Нека је* $\{X_1, X_2, \ldots, X_n\}$ *независан узорак из расподеле са математичким очекивањем* μ *и варијансом* σ^2*, при чему је* σ^2 *познато. Нека је* q_u *квантил реда и стандардне нормалне расподеле. Двострани интервал поверења за* μ *са нивоом поверења* $1-\alpha$ $(0 < \alpha < 1)$ *је дат са*

$$P\left(\mu \in \left[\hat{\mu} - q_{1-\alpha/2}\frac{\sigma}{\sqrt{n}}, \hat{\mu} + q_{1-\alpha/2}\frac{\sigma}{\sqrt{n}}\right]\right) = 1-\alpha. \tag{2.7}$$

Ако варијанса σ^2 није позната (што је најчешћи случај у применама) може се користити узорачка варијанса s^2, при чему се показује да $Z = \dfrac{\hat{\mu} - \mu}{s/\sqrt{n}}$ конвергира у расподели ка $t(n-1)$, тј. уместо нормалне имамо Студентову t расподелу, при чему је n обим узорка. Студентова расподела је слична нормалној расподели, али има веће „репове" и користи се када је обим узорка мали (обично мањи од 30). За велико n Студентова расподела може се апроксимирати нормалном расподелом $\mathcal{N}(0,1)$.

Теорема 2.2.9. *Нека је* $\{X_1, X_2, \ldots, X_n\}$ *независан узорак из расподеле са математичким очекивањем* μ *и варијансом* σ^2*, при чему је* σ^2 *непозната. Нека је* $q_{n-1,u}$ *квантил реда и Студентове* t *расподеле са* $n-1$ *степени слободе. Двострани интервал поверења за* μ *са нивоом поверења* $1-\alpha$ $(0 < \alpha < 1)$ *је дат са*

$$P\left(\mu \in \left[\hat{\mu} - q_{n-1,1-\alpha/2}\frac{s}{\sqrt{n}}, \hat{\mu} + q_{n-1,1-\alpha/2}\frac{s}{\sqrt{n}}\right]\right) = 1-\alpha, \tag{2.8}$$

где је s *узорачка стандардна девијација.*

Веома је важна правилна интерпретација интервала поверења. Интервал поверења са нивоом поверења $1-\alpha$ *не значи* да је вероватноћа да је нумеричка карактеристика θ унутар тог интервала једнака $1-\alpha$ [5], већ да, ако би се узорковање *поновило* N пута, онда би у $(1-\alpha)N$ случајева нумеричка карактеристика θ била унутар интервала поверења, а у αN случајева би била ван тог интервала. На пример, ако бисмо израчунали интервал поверења на основу 100 узорака са нивоом поверења 95%, онда би у 95 случајева нумеричка карактеристика била унутар израчунатог интервала, а у 5 случајева ван њега.

Такође, битно је нагласити на који начин ниво поверења утиче на *ширину* интервала поверења. Наиме, што је ниво поверења већи, то је интервал поверења шири; што је ниво поверења мањи, то је интервал поверења ужи.

2.3 Основе стохастичке анализе

У наставку је дата дефиниција стохастичког процеса, а затим и дефиниције повезаних појмова који се могу наћи у [2] и [4]:

Дефиниција 2.3.1. Стохастички[3] **(случајни) процес** је фамилија случајних променљивих $\{X_t \mid t \in T\}$, где је T неки бесконачан подскуп скупа реалних бројева. Обично се индекс t интерпретира као *време* и узима $T = [0, +\infty)$.

Скуп T може бити дискретан (тј. пребројив) и тада имамо низ случајних променљивих (тј. случајни низ) који се назива **процес у дискретном времену**; уколико је скуп T непребројив, онда имамо **случајни процес у непрекидном времену**.

Скуп свих могућих вредности случајних променљивих X_t за $t \in T$ назива се **скуп (простор) стања** или **фазни простор** случајног процеса и означава се са S.

Поред овде наведене, у литератури се могу наћи и алтернативне нотације за случајни процес; нпр. у [2] и [4] се користе следеће нотације, респективно: $\{X_t\}_{t\in T}$ и $\{X_t, t \in T\}$. Ако је $X_{t^*} = s^*$, при чему је $t^* \in T$ и $s^* \in S$, онда се каже да се стохастички систем у временском тренутку t^* налази у стању s^*. Математичка теорија случајних процеса назива се *стохастичка анализа* [4] или, краће, *стохастика*.

Дефиниција 2.3.2. Стохастички процес $\{X_t \,|\, t \in T\}$ при чему за свако t_i $(i \in \mathbb{N})$ где је $0 = t_1 < t_2 < \cdots < t_n < t_{n+1} < \cdots$ и свако $s_i \in S$ $(i \in \mathbb{N})$ важи:

$$P\left(X_{t_{n+1}} \leq s_{n+1} \,\middle|\, X_{t_1} = s_1, X_{t_2} = s_2, \ldots, X_{t_n} = s_n\right)$$
$$=$$
$$P\left(X_{t_{n+1}} \leq s_{n+1} \,\middle|\, X_{t_n} = s_n\right)$$

назива се **Марковљевим (Марковским) процесом**. Горња једнакост назива се **Марковљево (Марковско) својство**.

Као што је примећено у [2], ако тренутак t_n $(n \in \mathbb{N})$ интерпретирамо као садашњост, а тренутак t_{n+1} као будућност онда је смисао наведене дефиниције следећи: будућност зависи од прошлости само преко садашњости. Другим речима, целокупна информација

[3] Грчки глагол στόχος значи *циљати, гађати*. Фигуративно значење је *нагађати*. У научном контексту, придев *стохастички* се користи за описивање нечега чије је понашање *описано расподелом вероватноће*.

из прошлости процеса $\{X_t \mid t \in T\}$ која утиче на будућност овог процеса садржана је само у садашњој вредности тог процеса; ако се зна садашња вредност процеса тада познавање претходне историје процеса није потребно.

Дефиниција 2.3.3. Каже се да је Марковљев процес **временски хомоген** или, краће, **хомоген**, ако вероватноћа да се систем налази у неком стању s_{n+1} под условом да се у претходном временском тренутку налазио у стању s_n не зависи од конкретног временског тренутка посматрања система. Изражено математичком нотацијом, ако за Марковљев процес важи

$$P\left(X_{t_{n+1}} \leq s_{n+1} \,\middle|\, X_{t_n} = s_n\right) = P\left(X_{t_{n+1}-t_n} \leq s_{n+1} \,\middle|\, X_{t_1} = s_n\right),$$

онда се каже да је Марковљев процес **хомоген**.

2.3.1 Марковљеви ланци у дискретном времену

Дефиниција 2.3.4. Марковљев процес $\{X_t \mid t \in T\}$ који има дискретан простор стања S назива се **Марковљев ланац**.

Марковљеви ланци у дискретном времену биће означени акронимом МЛДВ, а Марковљеви ланци у непрекидном времену биће означени акронимом МЛНВ.

У наставку се разматрају Марковљеви ланци у дискретном времену. Без губитка општости, а ради једноставније нотације, узимаће се да је $T \subseteq \mathbb{N}_0$ и $S \subseteq \mathbb{N}_0$, тј. временске тренутке и стања биће означени бројевима $0, 1, 2, 3$ итд.

Дефиниција 2.3.5. Нека је дат МЛДВ. **Вероватноћа преласка** (каже се и: **вероватноћа прелаза**) из стања i у временском тренутку t_1 у стање j у временском тренутку t_2 у тачно $n = t_2 - t_1$ корака означава се са $p_{ij}^{(n)}(t_1, t_2)$ и дефинише се на следећи начин:

$$p_{ij}^{(n)}(t_1, t_2) = P\left(X_{t_2} = j \,\middle|\, X_{t_1} = i\right),\ 0 \leq t_1 \leq t_2. \tag{2.9}$$

Специјално, када је $n = 1$, користи се нотација

$$p_{ij}^{(1)}(t) = P\left(X_{t+1} = j \mid X_t = i\right) = p_{ij}(t), \tag{2.10}$$

што представља вероватноћу преласка из стања i у временском тренутку t у стање j у једном кораку.

У случају *хомогеног* МЛДВ, вероватноћа преласка из стања i у стање j у n корака *не зависи* од конкретних временских тренутака t_1 и t_2, већ само од разлике $n = t_2 - t_1$. Зато, једначина (2.9) постаје:

$$p_{ij}^{(n)}(t_1, t_2) = P\left(X_{t_2} = j \,\middle|\, X_{t_1} = i\right) = P\left(X_n = j \mid X_0 = i\right) = p_{ij}^{(n)}.$$

С друге стране, једначину (2.10) можемо поједноставити на следећи начин:

$$p_{ij}^{(1)}(t) = P\left(X_{t+1} = j \,\middle|\, X_t = i\right) = P\left(X_1 = j \mid X_0 = i\right) = p_{ij}.$$

Дефиниција 2.3.6. Квадратна матрица

$$\mathbf{P}^{(n)} = \left[p_{ij}^{(n)}\right]_{|S|\times|S|} = \begin{bmatrix} p_{00}^{(n)} & p_{01}^{(n)} & \cdots & p_{0j}^{(n)} & \cdots \\ p_{10}^{(n)} & p_{11}^{(n)} & \cdots & p_{1j}^{(n)} & \cdots \\ \vdots & \vdots & \ddots & \vdots & \cdots \\ p_{i0}^{(n)} & p_{i1}^{(n)} & \cdots & p_{ij}^{(n)} & \cdots \\ \vdots & \vdots & \cdots & \vdots & \ddots \end{bmatrix}$$

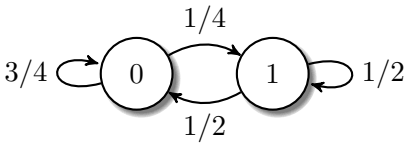
зове се **матрица вероватноћа преласка** [4] (или, краће, **матрица преласка** или **матрица прелаза**) **Марковљевог ланца у n корака**.

Специјално, када је $n = 1$, матрица преласка се означава са $\mathbf{P}$ и назива се **матрица преласка у једном кораку**.

За квадратну матрицу $\mathbf{P}^{(n)}$ каже се да је *стохастичка*, јер важи $\sum_j p_{ij}^{(n)} = 1$, где је $0 \leq p_{ij}^{(n)} \leq 1$ за све $i, j \in S$.

Хомогени МЛДВ се веома интуитивно представљају тзв. *дијаграмом преласка* (или *дијаграмом стања*) — коначним (ако је скуп стања коначан) или бесконачним (ако је скуп стања бесконачан) оријентисаним графом, где се стање i представља чвором графа, а прелазак у једном кораку из стања i у стање j представља се оријентисаном граном од чвора i до чвора j чија тежина представља вероватноћу преласка из стања i у стање j, p_{ij}.

Пример 2.3.7. Једноставан хомоген МЛДВ са два стања представљен је на слици 2.2.



Слика 2.2: *Дијаграм преласка за једноставан МЛДВ.*

Матрица преласка у једном кораку за овај МЛДВ је:

$$\mathbf{P} = \begin{bmatrix} 3/4 & 1/4 \\ 1/2 & 1/2 \end{bmatrix}.$$

Дакле, ако се МЛДВ налази у стању 0, онда је вероватноћа да ће у следећем кораку прећи у стање 1 једнака 1/4, а вероватноћа да ће остати у истом стању једнака је 3/4. С друге стране, ако се МЛДВ налази у стању 1, онда је једнако вероватно да ће у следећем кораку остати у истом стању или прећи у стање 0.

Приметимо да се на основу теореме о потпуној вероватноћи и Марковљевог својства може формулисати рекурзивна процедура за рачунање вероватноће преласка из стања i у стање k у n корака на бази вероватноћа преласка у једном кораку. Прелазак МЛДВ из стања i у тренутку t_1 у стање k у тренутку t_3 у $n = t_3 - t_1$ корака може се посматрати као:

1. Прелазак из стања i у неко међустање j у тренутку t_2, где је $t_1 < t_2 < t_3$;

2. Прелазак из међустања j у стање k у тренутку t_3.

Горња процедура се може формализовати на следећи начин:

Теорема 2.3.8. *Нека је дат МЛДВ. Вероватноћа преласка из стања i у стање k у n корака једнака је:*

$$p_{ik}^{(n)}(t_1, t_3) = \sum_{j \in S} p_{ij}^{(t_2 - t_1)}(t_1, t_2) p_{jk}^{(t_3 - t_2)}(t_2, t_3),\ 0 \leq t_1 < t_2 < t_3. \tag{2.11}$$

Једначине (2.11) *познате су као* ***једначине Чепмен-Колмогорова****.*

Ако је МЛДВ *хомоген*, онда једначине Чепмен-Колмогорова постају:

$$p_{ik}^{(n)} = \sum_{j \in S} p_{ij}^{(m)} p_{jk}^{(n-m)},\ 0 < m < n. \tag{2.12}$$

Како једначина (2.12) важи за свако $m < n$, стављањем $m = 1$ добија се:

$$p_{ik}^{(n)} = \sum_{j \in S} p_{ij}^{(1)} p_{jk}^{(n-1)}. \tag{2.13}$$

Коришћењем матрице преласка у n корака, горњу релацију можемо записати у матричном облику:

$$\mathbf{P}^{(n)} = \mathbf{P}^{(1)} \mathbf{P}^{(n-1)}. \tag{2.14}$$

Како је $\mathbf{P}^{(1)} = \mathbf{P}$, из једначине (2.14) добија се следећи резултат:

Теорема 2.3.9. *Нека је дат хомоген МЛДВ. Тада се матрица преласка у n корака рачуна на следећи начин: $\mathbf{P}^{(n)} = \mathbf{P}^n$. Такође, важи и: $\mathbf{P}^{(m+n)} = \mathbf{P}^m \mathbf{P}^n$.*

Пример 2.3.10. Занимљиво је посматрати понашање матрице преласка у n корака када n постепено расте. Нека је дат хомогени МЛДВ из примера 2.3.7. Матрица преласка у 2 корака изгледа на следећи начин:

$$\mathbf{P}^{(2)} = \mathbf{P}^2 = \begin{bmatrix} 0{,}75 & 0{,}25 \\ 0{,}5 & 0{,}5 \end{bmatrix}^2 = \begin{bmatrix} 0{,}6875 & 0{,}3125 \\ 0{,}625 & 0{,}375 \end{bmatrix}.$$

Матрица преласка у 3 корака је:

$$\mathbf{P}^{(3)} = \mathbf{P}^3 = \mathbf{P}^2 \mathbf{P} = \begin{bmatrix} 0{,}67188 & 0{,}32813 \\ 0{,}65625 & 0{,}34375 \end{bmatrix}.$$

Матрица преласка у 4 корака има следећи облик:

$$\mathbf{P}^{(4)} = \mathbf{P}^4 = \mathbf{P}^3 \mathbf{P} = \begin{bmatrix} 0{,}66797 & 0{,}33203 \\ 0{,}66406 & 0{,}33594 \end{bmatrix}.$$

Наслућује се да, са порастом броја корака n, матрица преласка овог МЛДВ конвергира ка матрици која има *идентичне колоне.*

Дефиниција 2.3.11. Вероватноћа да се МЛДВ налази у стању i $(i = 0, 1, 2, ...)$ у n корака $(n = 0, 1, 2, ...)$ назива се **транзијентном вероватноћом стања** i **у** n **корака** и означава се са $\nu_i(n)$. **Вектор транзијентних вероватноћа стања** МЛДВ у n корака означава се са $\boldsymbol{\nu}(n) = [\nu_0(n)\ \nu_1(n)\ \nu_2(n)\ ...]$.

Ако је познат вектор транзијентних вероватноћа стања у временском кораку $0, \boldsymbol{\nu}(0) = [\nu_0(0)\ \nu_1(0)\ \nu_2(0)\ ...]$, онда се вектор транзијентних вероватноћа стања у n корака добија множењем *вектора почетних вероватноћа стања* и матрице преласка у n корака:

$$\boldsymbol{\nu}(n) = \boldsymbol{\nu}(0)\,\mathbf{P}^{(n)} = \boldsymbol{\nu}(0)\,\mathbf{P}^n.$$

Алтернативно, приликом рачунања вектора транзијентних вероватноћа стања у n корака може се заобићи степеновање матрице преласка под условом да је познат вектор транзијентних вероватноћа стања у $n-1$ корака:

$$\boldsymbol{\nu}(n) = \boldsymbol{\nu}(n-1)\,\mathbf{P}.$$

Посебно значајан јесте случај када множење са матрицом преласка не мења сам вектор вероватноћа стања.

Дефиниција 2.3.12. Вектор вероватноћа стања хомогеног МЛДВ који остаје *непромењен* када се помножи са матрицом преласка у једном кораку назива се **вектором стационарних вероватноћа стања** или, ако се интерпретира као закон расподеле, **стационарном расподелом вероватноће стања** и означава се са $\boldsymbol{\nu} = [\nu_0\ \nu_1\ \nu_2\ ...]$. Вектор стационарних вероватноћа стања задовољава следеће релације:

$$\boldsymbol{\nu} = \boldsymbol{\nu}\mathbf{P} \quad \text{и} \quad \sum_{i \in S} \nu_i = 1. \tag{2.15}$$

Друга једначина се често представља у матричном облику

$$\boldsymbol{\nu}\mathbf{1} = 1,$$

где је $\mathbf{1} = [1\ 1\ 1\ ...]^\top$, а назива се **нормализационим условом**.

Треба приметити да може постојати више од једног вектора стационарних вероватноћа стања. Заправо, може их бити бесконачно много.

Одређивањем граничне вредности транзијентних вероватноћа стања добијају се граничне вероватноће стања:

Дефиниција 2.3.13. Вектор граничних вероватноћа стања хомогеног МЛДВ дефинише се на следећи начин:

$$\tilde{\boldsymbol{\nu}} = \lim_{n\to\infty} \boldsymbol{\nu}(n) = \lim_{n\to\infty} \boldsymbol{\nu}(0)\,\mathbf{P}^{(n)} = \lim_{n\to\infty} \boldsymbol{\nu}(0)\,\mathbf{P}^n = \boldsymbol{\nu}(0) \lim_{n\to\infty} \mathbf{P}^n.$$

Гранична вредност $\lim_{n\to\infty} \mathbf{P}^n$ се назива **граничном матрицом преласка** и означава се са $\tilde{\mathbf{P}}$, тј. $\tilde{\boldsymbol{\nu}} = \boldsymbol{\nu}(0)\,\tilde{\mathbf{P}}$.

Посебно је занимљив случај када вектор граничних вероватноћа стања не зависи од вектора почетних вероватноћа стања.

Дефиниција 2.3.14. Уколико вектор граничних вероватноћа стања хомогеног МЛДВ *не зависи* од вектора почетних вероватноћа стања и важе следеће релације

$$\tilde{\nu}_i > 0\,(\forall i \in S) \quad \text{и} \quad \sum_{i \in S} \tilde{\nu}_i = 1,$$

онда се овај вектор назива **јединствени вектор равнотежних вероватноћа стања** МЛДВ и означава се са $\boldsymbol{\pi}$. За хомогени МЛДВ се тада каже да је **егодичан**.

Пример 2.3.15. Нека је дат МЛДВ из примера 2.3.7. Гранична матрица преласка је:

$$\tilde{\mathbf{P}} = \lim_{n \to \infty} \mathbf{P}^n = \begin{bmatrix} 0{,}6\overline{6} & 0{,}3\overline{3} \\ 0{,}6\overline{6} & 0{,}3\overline{3} \end{bmatrix}$$

Како су колоне граничне матрице идентичне и све вредности у оквиру исте врсте матрице строго веће од нуле и у збиру једнаке 1, јединствени вектор равнотежних вероватноћа стања постоји и добија се издвајањем било ког реда граничне матрице преласка, тј. $\boldsymbol{\pi} = [0{,}6\overline{6}\ 0{,}3\overline{3}]$.

Уколико јединствени вектор равнотежних вероватноћа стања постоји, онда је он истовремено и вектор стационарних вероватноћа стања (обрнуто не мора да важи), те се може добити решавањем једначина (2.15), односно није потребно рачунати граничну матрицу преласка. Уз напомену да формална разлика постоји, у наставку ће се, када се ергодичност подразумева, уместо јединствених равнотежних вероватноћа стања чешће говорити о стационарним вероватноћама стања.

Треба приметити да јединствени вектор равнотежних вероватноћа стања *не постоји* за сваки МЛДВ. Довољни и потребни услови за његово постојање могу се наћи у [3].

Веома је битна веза између хомогених Марковљевих ланаца у дискретном времену и геометријске расподеле. Наиме, нека је R_i случајна променљива која представља *време боравка* случајног процеса у стању i, односно број корака које хомогени МЛДВ проведе у стању i пре него што пређе у неко друго стање. Вероватноћа да случајни процес напусти стање i износи $1 - p_{ii} = \sum_{i \neq j} p_{ij}$, тј. она не зависи нити од тога на који начин је МЛДВ дошао у стање i, нити од тога колико је времена већ провео у том стању. Дакле, време боравка у стању i заправо представља број Бернулијевих експеримената до првог успеха, при чему је успешан исход дефинисан као *напуштање* тренутног стања. Према томе, време боравка у стању i има геометријску расподелу, односно једину дискретну расподелу која има својство *одсуства меморије*. Минимални број корака потребан да би се напустило стање i износи 1, те се користи формула (2.4), док је вероватноћа успеха једнака $1 - p_{ii}$. Дакле, важи:

$$P\,(R_i = k) = (1 - p_{ii}) p_{ii}^{k-1},\ k = 1, 2, 3, ...$$

Према томе, очекивано време боравка у стању i износи:

$$\mathrm{E}\,(R_i) = \frac{1}{1 - p_{ii}}.$$

У реалним системима, међутим, време није дискретна величина, тако да су за примене много значајнији Марковљеви ланци у *непрекидном времену*.

2.3.2 Марковљеви ланци у непрекидном времену

Дефиниција 2.3.16. Нека је дат МЛНВ. **Вероватноћа преласка** (каже се и **вероватноћа прелаза**) из стања i у стање j у оквиру временског интервала $[t_1, t_2)$ означава се са $p_{ij}(t_1, t_2)$ и дефинише се на следећи начин:

$$p_{ij}(t_1, t_2) = P\left(X_{t_2} = j \,\middle|\, X_{t_1} = i\right),\ 0 \leq t_1 \leq t_2. \tag{2.16}$$

Специјално, када је $t_1 = t_2$, важи:

$$p_{ij}(t_1, t_1) = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Као и у случају МЛДВ, ако је МЛНВ *хомоген*, онда вероватноћа преласка из стања i у стање j не зависи од конкретних временских тренутака t_1 и t_2, већ само од разлике $t = t_2 - t_1$. Зато, једначина (2.16) постаје:

$$p_{ij}(0, t) = P\left(X_{t_1+t} = j \,\middle|\, X_{t_1} = i\right) = P\left(X_t = j \mid X_0 = i\right) = p_{ij}(t).$$

Поново потпуно аналогно са МЛДВ, дефинишу се *транзијентне вероватноће стања* МЛНВ у временском тренутку t; биће означене са $\pi_i(t)$.

Веза између транзијентних вероватноћа стања и вероватноћа преласка између стања у оквиру временског интервала $[t_1, t_2)$ је следећа:

$$\pi_j(t_2) = \sum_{i \in S} \pi_i(t_1)\, p_{ij}(t_1, t_2). \tag{2.17}$$

Увођењем матрице преласка $\mathbf{P}(t_1, t_2) = [p_{ij}(t_1, t_2)]_{|S| \times |S|}$, и вектора транзијентних вероватноћа стања $\boldsymbol{\pi}(t) = [\pi_0(t)\ \ \pi_1(t)\ \ \pi_2(t)\ \ ...]$, релација (2.17) се може записати у матричном облику:

$$\boldsymbol{\pi}(t) = \boldsymbol{\pi}(t_1)\, \mathbf{P}(t_1, t_2).$$

У случају хомогеног МЛНВ, може се ставити $t_1 = 0$, тј. важи:

$$\boldsymbol{\pi}(t) = \boldsymbol{\pi}(0)\, \mathbf{P}(0, t) = \boldsymbol{\pi}(0)\, \mathbf{P}(t).$$

Једначине Чепмен-Колмогорова за МЛНВ гласе:

$$p_{ik}(t_1, t_3) = \sum_{j \in S} p_{ij}(t_1, t_2) p_{jk}(t_2, t_3),\ 0 \leq t_1 \leq t_2 < t_3. \tag{2.18}$$

Међутим, оне у овом облику нису корисне као што је то био случај код МЛДВ. Уместо тога, једначине Чепмен-Колмогорова се трансформишу у систем диференцијалних једначина чијим решавањем можемо добити транзијентне вероватноће стања МЛНВ.

Најпре се уводе *интензитети преласка* из стања i у стање j у временском тренутку t помоћу следећих граничних вредности и већ дефинисаних вероватноћа преласка:

$$q_{ij}(t) = \lim_{\Delta t \to 0} \frac{p_{ij}(t, t + \Delta t)}{\Delta t},\ i \neq j, \tag{2.19}$$

$$q_{ii}(t) = \lim_{\Delta t \to 0} \frac{p_{ii}(t, t + \Delta t) - 1}{\Delta t}. \tag{2.20}$$

Ако граничне вредности (2.19) и (2.20) постоје, како је

$$\sum_{j\in S} p_{ij}(t, t+\Delta t) = 1,$$

то следи:

$$\sum_{j\in S} q_{ij}(t) = 0 \quad \text{и} \quad q_{ii}(t) = -\sum_{\substack{j\in S \\ \wedge \\ j\neq i}} q_{ij}(t),\ i \in S.$$

Стављањем $t_3+\Delta t$ уместо t_3 у (2.18), а затим одузимањем обе стране једначине (2.18) од добијене једначине, добија се:

$$p_{ij}(t_1, t_3+\Delta t) - p_{ij}(t_1, t_3) = \sum_{k\in S} p_{ik}(t_1, t_2)\left(p_{kj}(t_2, t_3+\Delta t) - p_{kj}(t_2, t_3)\right).$$

Дељењем обе стране горње једначине са Δt, узимањем лимеса када $\Delta t \to 0$ и $t_2 \to t_3$, добија се *систем директних Колмогоровљевих једначина*:

$$\frac{\partial p_{ij}(t_1, t_3)}{\partial t_3} = \sum_{k\in S} p_{ik}(t_1, t_3) q_{kj}(t_3),\ 0 \le t_1 < t_3. \tag{2.21}$$

Ако је МЛНВ хомоген, онда се систем директних Колмогоровљевих једначина може поједноставити. Стављањем $t = t_3 - t_1$ и коришћењем једначина (2.19) и (2.20) за рачунање временски независних интензитета преласка, добија се:

$$\frac{\mathrm{d}\pi_{ij}(t)}{\mathrm{d}t} = \sum_{k\in S} \pi_{ik}(t)\, q_{kj},\ \forall\, i, j \in S. \tag{2.22}$$

Диференцирањем обе стране једначине (2.17) добија се (2.23); коришћењем директних Колмогоровљевих једначина (2.22), (2.23) се трансформише у (2.24); још једном применом једначине (2.17) једначина (2.25) се трансформише у (2.26):

$$\begin{aligned}
\frac{\mathrm{d}\pi_j(t_2)}{\mathrm{d}t_2} &= \frac{\partial}{\partial t_2}\left(\sum_{i\in S} \pi_i(t_1)\, p_{ij}(t_1, t_2)\right) \\
&= \sum_{i\in S} \pi_i(t_1) \frac{\partial p_{ij}(t_1, t_2)}{\partial t_2} && (2.23)\\
&= \sum_{i\in S} \pi_i(t_1) \sum_{k\in S} p_{ik}(t_1, t_2) q_{kj}(t_2) && (2.24)\\
&= \sum_{k\in S} q_{kj}(t_2) \sum_{i\in S} p_{ik}(t_1, t_2) \pi_i(t_1) && (2.25)\\
&= \sum_{k\in S} q_{kj}(t_2) \pi_k(t_2),\ \forall j \in S. && (2.26)
\end{aligned}$$

Специјално, када је МЛНВ хомоген, интензитети преласка не зависе од времена, тј. $q_{ij}(t) = q_{ij}$, те једначина (2.26) постаје:

$$\frac{\mathrm{d}\pi_j(t)}{\mathrm{d}t} = \sum_{k\in S} q_{kj} \pi_k(t),\ \forall j \in S.$$

Ако за МЛНВ постоје јединствене равнотежне вероватноће стања, онда, знајући да је

$$\lim_{t\to\infty}\frac{\mathrm{d}\pi_j(t)}{\mathrm{d}t}=0,$$

систем диференцијалних једначина (2.26) се своди на следећи *систем линеарних једначина*:

$$0=\sum_{k\in S}q_{kj}\pi(k),\ \forall j\in S.$$

Увођењем *матрице интензитета преласка* $\mathbf{Q}=[q_{ij}]_{|S|\times|S|}$, где је q_{ij} дефинисан једначинама (2.19) и (2.20), горњи систем једначина се може записати у следећем облику:

$$\mathbf{0}=\boldsymbol{\pi}\mathbf{Q}. \tag{2.27}$$

По аналогији са МЛДВ дефинише се ергодичност у случају МЛНВ.

Дефиниција 2.3.17. МЛНВ за који постоји јединствени вектор равнотежних вероватноћа стања назива се **ергодичним**.

Јединствени вектор равнотежних вероватноћа стања се, уколико постоји, може добити решавањем једначине (2.27), уз поштовање нормализационог услова, тј.

$$\boldsymbol{\pi}\mathbf{1}=\sum_{i\in S}\pi(i)=1. \tag{2.28}$$

Слично хомогеним Марковљевим ланцима у дискретном времену, хомогени Марковљеви ланци у непрекидном времену се врло интуитивно представљају *дијаграмом преласка* чији су чворови стања, а гране означавају могуће преласке између стања. Једина разлика у односу на дискретни случај је да тежине грана представљају интензитете преласка између одговарајућих стања, а не вероватноће преласка.

Такође по аналогији са хомогеним Марковљевим ланцем у дискретном времену, дефинише се случајна променљива која представља *време боравка* хомогеног МЛНВ у стању i, означена са R_i. И у овом случају захтева се да расподела ове случајне променљиве има особину *одсуства меморије*, с тим што сада она мора бити непрекидна. Једина расподела која испуњава ова два услова јесте експоненцијална, те вероватноћа да се МЛНВ налази у стању i током временског интервала $[0,t)$ износи $\mathrm{e}^{q_{ii}t}$, где је q_{ii} дефинисан једначином (2.20). Дакле, време боравка у стању i има експоненцијалну расподелу са параметром $\lambda=-q_{ii}$, те очекивано време боравка у стању i износи:

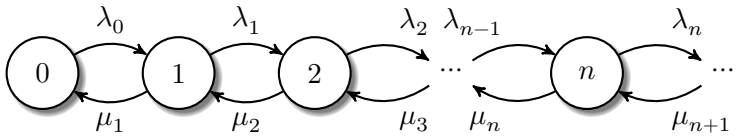
$$\mathrm{E}(R_i)=-\frac{1}{q_{ii}}.$$

2.3.3 Важни Марковљеви ланци

Веома значајна класа Марковљевих ланаца јесу процеси рађања и умирања чија је дефиниција дата у наставку.

Дефиниција 2.3.18. Процес рађања и умирања је Марковљев ланац у којем постоје преласци само између суседних стања [3].

У наставку се разматра *хомогени* процес рађања и умирања *у непрекидном времену* (лако се могу добити аналогни резултати за процес рађања и умирања у дискретном времену). Нека су стања процеса обележена бројевима $0, 1, 2, ...$; нека је λ_i стопа рађања, односно интензитет преласка из стања i у стање $i+1$ $(i = 0, 1, 2, ...)$, а μ_i стопа умирања, односно интензитет преласка из стања i у стање $i-1$ $(i = 1, 2, 3, ...)$, при чему важи $\lambda_i > 0$ и $\mu_i > 0$. Дијаграм преласка за описани процес рађања и умирања дат је на слици 2.3.



Слика 2.3: *Дијаграм преласка за процес рађања и умирања.*

Под условом да је процес ергодичан, јединствене равнотежне вероватноће стања добијају се решавањем једначине (2.27), при чему матрица интензитета преласка овог процеса изгледа на следећи начин:

$$\mathbf{Q} = \begin{bmatrix} -\lambda_0 & \lambda_0 & 0 & 0 & 0 & \cdots \\ \mu_1 & -(\lambda_1+\mu_1) & \lambda_1 & 0 & 0 & \cdots \\ 0 & \mu_2 & -(\lambda_2+\mu_2) & \lambda_2 & 0 & \cdots \\ 0 & 0 & \mu_3 & -(\lambda_3+\mu_3) & \lambda_3 & \cdots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

Матрица $\mathbf{Q}$ је у овом случају *тридијагонална*, те се производ $\mathbf{0} = \boldsymbol{\pi}\mathbf{Q}$ своди на следећи једноставан систем једначина:

$$0 = \lambda_0\pi_0 - \mu_1\pi_1, \tag{2.29}$$
$$0 = \lambda_{i-1}\pi_{i-1} - (\lambda_i + \mu_i)\,\pi_i + \mu_{i+1}\pi_{i+1},\ i \geq 1. \tag{2.30}$$

Решавањем једначине (2.29) по π_1 и уврштавањем добијеног резултата у (2.30) за $i = 1$, добија се:

$$\pi_1 = \frac{\lambda_0}{\mu_1}\pi_0,$$
$$\pi_2 = \frac{\lambda_0}{\mu_1}\frac{\lambda_1}{\mu_2}\pi_0.$$

Из горњих једнакости се наслућује, а лако се може доказати индукцијом, да важи следећа релација:

$$\pi_i = \frac{\lambda_0}{\mu_1}\frac{\lambda_1}{\mu_2}\cdots\frac{\lambda_{i-1}}{\mu_i}\pi_0 = \pi_0\prod_{j=0}^{i-1}\frac{\lambda_j}{\mu_{j+1}},\ i \geq 1. \tag{2.31}$$

Коришћењем нормализационог услова, $\sum_{i=0}^{\infty}\pi_i = 1$, добија се јединствена равнотежна вероватноћа да се процес налази у стању 0:

$$\pi_0 = \frac{1}{1 + \sum\limits_{k=1}^{\infty} \prod\limits_{i=0}^{k-1} \frac{\lambda_i}{\mu_{i+1}}}. \tag{2.32}$$

Јединствене равнотежне вероватноће свих осталих стања процеса добијају се увршта-вањем (2.32) у (2.31).
Услов конвергенције реда у имениоцу разломка на десној страни једначине (2.32) представља услов ергодичности процеса рађања и умирања и гласи:

$$\exists i_0 \in \mathbb{N} \left[\forall i \in \mathbb{N} \left(i > i_0 \Rightarrow \frac{\lambda_i}{\mu_{i+1}} < 1 \right) \right].$$

Још један веома битан стохастички процес јесте хомогени *Поасонов процес*, који се може дефинисати као посебан случај хомогеног процеса рађања и умирања.

Дефиниција 2.3.19. Хомогени **Поасонов процес** је хомогени процес рађања и умирања са стањима $0, 1, 2, ...$, при чему је стопа рађања $\lambda_i = \lambda > 0$ за $i \geq 0$, а стопа умирања $\mu_i = 0$ за све $i \geq 1$. Стопа рађања λ назива се *интензитет прилива*.

Поасонов процес се назива и *чистим процесом рађања* (енгл. *pure birth process*), а припада групи тзв. *бројачких процеса* (енгл. *counting processes*), јер се његово стање у временском тренутку t може тумачити као број догађаја који су се десили до тренутка t.

Треба приметити да са протоком времена број догађаја који су се десили може само да расте, те за Поасонов процес *не постоји* јединствени вектор равнотежних вероватноћа стања (тј. не може се користити једначина (2.27)) нити постоје стационарне вероватноће стања. Међутим, транзијентне вероватноће стања у временском тренутку t могу се израчунати решавањем система директних Колмогоровљевих једначина (2.22):

$$\begin{aligned} \frac{\mathrm{d}\pi_0(t)}{\mathrm{d}t} &= -\lambda \pi_0(t), \\ \frac{\mathrm{d}\pi_i(t)}{\mathrm{d}t} &= -\lambda \pi_i(t) + \lambda \pi_{i-1}(t), \; i \geq 1. \end{aligned}$$

Уз почетне услове $\pi_0(0) = 1$ и $\pi_i(0) = 0$ за $i \geq 1$, транзијентне вероватноће стања 0 и 1 у временском тренутку t имају следећи облик:

$$\begin{aligned} \pi_0(t) &= \mathrm{e}^{-\lambda t}, \\ \frac{\mathrm{d}\pi_1(t)}{\mathrm{d}t} = -\lambda \pi_1(t) + \lambda \mathrm{e}^{-\lambda t} &\Rightarrow \pi_1(t) = \lambda t \mathrm{e}^{-\lambda t}. \end{aligned}$$

Индукцијом се може доказати да важи:

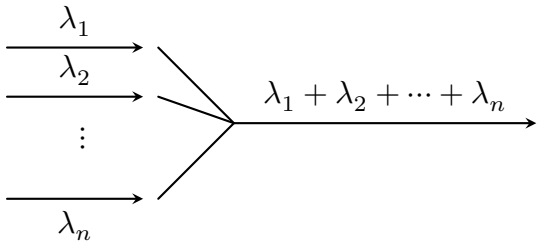
$$\pi_k(t) = \frac{(\lambda t)^k}{k!} \mathrm{e}^{-\lambda t}, \; k \geq 0.$$

Дакле, број догађаја који су се десили до тренутка t има Поасонову расподелу са параметром λt.

Већ је речено да време боравка у неком стању i МЛНВ има експоненцијалну расподелу са параметром $-q_{ii}$. Како је, за Поасонов процес, $q_{ii} = -\lambda$ за $i \geq 0$, време боравка у стању i ($i \geq 0$) има експоненцијалну расподелу са истим параметром λ. Алтернативна формулација ове важне везе између Поасоновог процеса и експоненцијалне расподеле гласи: време између два узастопна догађаја у Поасоновом процесу има експоненцијалну расподелу са параметром λ.

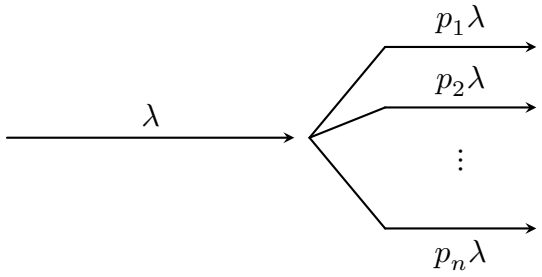
У наставку су дате две веома значајне особине Поасоновог процеса које је на елегантан и савремен начин доказао Кобајаши у [6]:

- *суперпозиција (спајање) Поасонових процеса* — нека је дато $n \geq 2$ токова догађаја описаних Поасоновим процесима са интензитетима приспећа $\lambda_1, \lambda_2, \ldots, \lambda_n$. Ако се сви ови токови комбинују у један ток догађаја, резултујући ток догађаја такође представља Поасонов процес са интензитетом приспећа $\lambda = \lambda_1 + \lambda_2 + \cdots + \lambda_n$;



Слика 2.4: *Суперпозиција Поасонових процеса.*

- *декомпозиција (раздвајање) Поасоновог процеса* — нека је дат Поасонов процес са интензитетом приспећа λ. Ако се сваки догађај овог процеса независно класификује у једну од $n \geq 2$ класа са вероватноћама $p_1, p_2, \ldots, p_n$, где је $p_1 + p_2 + \cdots + p_n = 1$, тада је ток догађаја који припада класи i Поасонов процес са интензитетом приспећа $\lambda_i = p_i\lambda$, за $i = 1, 2, \ldots, n$.



Слика 2.5: *Декомпозиција Поасоновог процеса.*

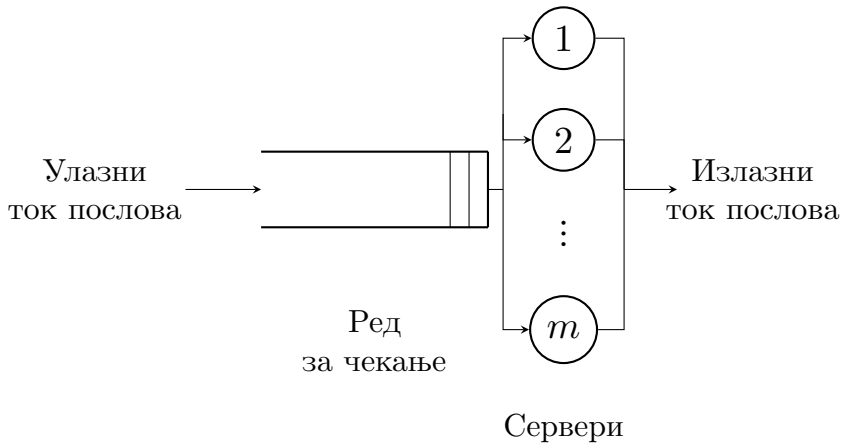
Поглавље 3

СИСТЕМИ СА ОПСЛУЖНИМ ЦЕНТРИМА

У овом поглављу биће дефинисани модели споменути у уводном поглављу — Марковљеви модели којима се моделују једноставнији системи и мреже са опслужним центрима којима се моделују сложенији системи. Биће дефинисани најважнији индикатори перформанси и образложене поједине аналитичке методе за њихово одређивање.

3.1 Системи са једним опслужним центром

Систем са једним опслужним центром приказан је на слици 3.1.



Слика 3.1: *Опслужни центар са m сервера.*

Опслужни центар чине ред за чекање ограниченог или неограниченог капацитета и један или више идентичних сервера. Овакав систем јесте градивни блок сложенијег модела — *мреже са опслужним центрима* — који ће бити обрађен касније и у том случају се једноставно зове *чвором мреже*.

Један сервер може у једном тренутку опслуживати (сервисирати, обрађивати) само један захтев, тј. сервер се у једном тренутку може налазити у једном од два стања — слободан (енгл. *idle*) или заузет (енгл. *busy*). Ако су у тренутку доласка захтева сви сервери заузети, новоприспели захтев чека у реду за чекање под условом да у њему има места (тј. да бафер, уколико је коначног капацитета, није попуњен до краја; у супротном

има више начина да се поступа, али ће се у овом раду сматрати да се такви послови одбацују) све док не дође тренутак да га преузме сервер који је постао слободан. Посао се из реда за чекање шаље на обраду на слободан сервер коришћењем унапред дефинисане дисциплине распоређивања (енгл. *queueing/scheduling discipline, service strategy*).

За потпуни опис система потребно је још познавати природу приспећа захтева у систем, као и природу обраде захтева на серверима. Доласке послова у систем можемо формално описати као случајан процес окарактерисан низом случајних променљивих $A_1, A_2, ...$ које представљају време које протекне између сукцесивних долазака захтева, при чему је уобичајено претпоставити да су ове случајне променљиве независне и да имају идентичну расподелу. Тада се процес долазака послова назива *процес обнављања* (енгл. *renewal process*). Функција расподеле времена између долазака послова у систем може бити дискретна или непрекидна, при чему ће се у наставку разматрати само случајеви када је она непрекидна. Интензитет приспећа послова у систем се конвенционално означава са λ и рачуна се као $\lambda = 1/\mathrm{E}\left(A_i\right), i = 1, 2, ...$ Најчешће коришћена функција расподеле времена између приспећа послова у систем јесте експоненцијална расподела, тј. процес долазака послова у систем је хомогени *Поасонов процес*. На сличан начин се дефинише низ случајних променљивих $S_1, S_2, ...$ које представљају време које је потребно да сервер обради посао, при чему је уобичајено претпоставити да су и ове случајне променљиве независне и да имају идентичну расподелу. Интензитет обраде послова на серверу се конвенционално означава са μ и рачуна се као $\mu = 1/\mathrm{E}\left(S_i\right), i = 1, 2, ...$

За компактно записивање информација које описују опслужни центар користи се Кендалова[1] нотација у облику $A/S/m/K$ - QD, где је:

- A — расподела времена између долазака послова у опслужни центар,
- S — расподела времена обраде послова на серверима опслужног центра,
- m — број сервера у опслужном центру,
- K — укупни капацитет опслужног центра (збир броја сервера и капацитета реда за чекање); ако је капацитет реда за чекање неограничен K се обично не наводи,
- QD — дисциплина распоређивања послова у реду за чекање опслужног центра.

Када је одговарајућа расподела експоненцијална, на позицијама A и S се користи слово M. Дисциплина распоређивања која ће бити најзначајнија у наставку овог поглавља је FIFO (енгл. *First-In, First-Out*) или FCFS (енгл. *First-Come, First-Served*) — послови се обрађују по редоследу доласка у систем. Уколико није експлицитно назначено, претпостављаће се да је дисциплина распоређивања FIFO.

3.1.1 Мере перформанси

Најбитније мере перформанси које ће се разматрати за моделе у наставку јесу:

- искоришћеност (енгл. *utilization*), U, представља део укупног времена које сервери опслужног центра проведу обрађујући послове; нема мерну јединицу и представља број у интервалу $[0, 1]$, тј. $0\% - 100\%$;

[1] David George Kendall (1918 — 2007) је био енглески статистичар и математичар.

- проток (енгл. *throughput*), X, представља просечан број захтева који опслужни центар обради у јединици времена;
- време одзива (енгл. *response time*), R, представља просечно време које је потребно да захтев прође кроз опслужни центар, тј. просечно време од приспећа захтева у опслужни центар до тренутка када је захтев обрађен и напушта опслужни центар;
- време чекања (енгл. *waiting time*), W, представља просечно време које захтев проводи чекајући у реду за чекање опслужног центра пре него што започне његова обрада на неком серверу опслужног центра;
- дужина реда за чекање (енгл. *queue length*), Q, представља просечан број захтева који се налази у реду за чекање опслужног центра;
- број послова (енгл. *number of jobs*), J, представља просечан број захтева који се налази у опслужном центру, укључујући и оне који се тренутно обрађују на серверима опслужног центра.

Једна од најважнијих релација у теорији система са редовима за чекање јесте Литлова[2] теорема или Литлов закон који повезује просечан број послова у систему са временом одзива система:

$$J = XR. \tag{3.1}$$

Литлов закон се може применити и на сам ред за чекање опслужног центра, тј.:

$$Q = XW. \tag{3.2}$$

Још једна важна релација која ће се користити у наставку јесте веза између времена одзива, времена чекања и интензитета обраде послова на серверима опслужног центра:

$$R = W + \frac{1}{\mu}. \tag{3.3}$$

3.1.2 M/M/1 систем

M/M/1 систем има ред за чекање бесконачног капацитета и тачно један сервер. Као такав, овај систем је најједноставнији за анализу. Математички се посматра као процес рађања и умирања (видети дефиницију 2.3.18), при чему:

- интензитет рађања је у сваком стању идентичан и једнак интензитету приспећа послова у систем, λ;
- интензитет умирања је у сваком стању идентичан и једнак интензитету опслуживања, μ.

[2]John Little (1928 — 2024) је био амерички научник који се истакао у области операционе анализе.

Под условом да је $\lambda < \mu$ или, еквивалентно, ако однос λ/μ означимо са ρ, $\rho < 1$, тј. ако је одговарајући МЛНВ ергодичан, односно систем стабилан, једначина (2.32) постаје:

$$\pi_0 = \frac{1}{1 + \sum\limits_{k=1}^{\infty} \prod\limits_{i=0}^{k-1} \frac{\lambda_i}{\mu_{i+1}}} = \frac{1}{1 + \sum\limits_{k=1}^{\infty} \left(\frac{\lambda}{\mu}\right)^k} = \frac{1}{1 + \dfrac{\lambda/\mu}{1 - \lambda/\mu}} = 1 - \frac{\lambda}{\mu} = 1 - \rho.$$

Из једначине (2.31) добијамо вероватноћу да се у систему налази i послова ($i \geq 0$):

$$\pi_i = \pi_0 \left(\frac{\lambda}{\mu}\right)^i = \left(1 - \frac{\lambda}{\mu}\right) \left(\frac{\lambda}{\mu}\right)^i = (1 - \rho)\rho^i.$$

Дакле, ако случајну промењљиву која представља број послова у систему означимо са N, важи $N \sim \mathrm{Geom}\,(1 - \rho)$, тј. број послова у систему има геометријску расподелу дефинисану помоћу (2.3) са вероватноћом успеха $p = 1 - \rho$.

Мере перформанси M/M/1 система рачунају се на следећи начин:

- искоришћеност је једнака односу интензитета приспећа послова у систем и интензитета опслуживања, тј. $U = \rho = \dfrac{\lambda}{\mu}$;
- проток је једнак интензитету приспећа послова у систем, тј. $X = \lambda$;
- просечан број послова је математичко очекивање броја послова у систему; коришћењем једначине (2.5) добија се:

$$J = \mathrm{E}\,(N) = \frac{1 - p}{p} = \frac{\rho}{1 - \rho} = \frac{\lambda}{\mu - \lambda};$$

- време одзива се добија коришћењем Литловог закона:

$$R = \frac{J}{X} = \frac{\rho}{\lambda(1 - \rho)} = \frac{1}{\mu - \lambda};$$

- време чекања се добија коришћењем релације (3.3):

$$W = R - \frac{1}{\mu} = \frac{\rho}{\mu(1 - \rho)} = \frac{\lambda}{\mu(\mu - \lambda)};$$

- дужина реда за чекање се добија коришћењем Литловог закона:

$$Q = XW = \lambda \cdot \frac{\rho}{\mu(1 - \rho)} = \frac{\rho^2}{1 - \rho} = \frac{\lambda^2}{\mu(\mu - \lambda)}.$$

3.1.3 M/M/m систем

M/M/m систем има ред за чекање бесконачног капацитета и m идентичних сервера сваки од којих обрађује захтеве интензитетом μ. Математички, оваквом систему одговара процес рађања и умирања (видети дефиницију 2.3.18), при чему:

- Интензитет рађања је у сваком стању идентичан и једнак интензитету приспећа послова у систем, λ.
- Интензитет умирања зависи од броја послова у систему — ако је број послова у систему, k, мањи или једнак броју сервера m, онда је интензитет умирања једнак $k\mu$, док су за $k \geq m$ сви сервери опслужног центра заузети независно од тога колико тачно послова има тренутно у систему, те је интензитет умирања једнак $m\mu$.

Дакле, важи:

$$\begin{aligned} \lambda_k &= \lambda, \quad k \geq 0, \\ \mu_k &= \begin{cases} k\mu, & 0 \leq k < m, \\ m\mu, & k \geq m. \end{cases} \end{aligned} \tag{3.4}$$

Услов стабилности система (тј. ергодичности одговарајућег МЛНВ) је $\lambda < m\mu$ или, увођењем $\rho = \lambda/(m\mu)$, $\rho < 1$. На основу релације (2.31) јединствене равнотежне вероватноће стања дате су са:

$$\pi_k = \begin{cases} \pi_0 \prod\limits_{i=0}^{k-1} \dfrac{\lambda}{(i+1)\mu} = \pi_0 \left(\dfrac{\lambda}{\mu}\right)^k \cdot \dfrac{1}{k!} = \pi_0 \dfrac{(m\rho)^k}{k!}, & 0 \leq k \leq m, \\ \pi_0 \prod\limits_{i=0}^{m-1} \dfrac{\lambda}{(i+1)\mu} \cdot \prod\limits_{i=m}^{k-1} \dfrac{\lambda}{m\mu} = \pi_0 \dfrac{\rho^k m^m}{m!}, & k \geq m. \end{cases}$$

На основу $\sum_{i=0}^{\infty} \pi_i = 1$ добија се:

$$\pi_0 = \left[\sum_{k=0}^{m-1} \frac{(m\rho)^k}{k!} + \frac{(m\rho)^m}{m!} \cdot \frac{1}{1-\rho} \right]^{-1}.$$

Вероватноћа да ће, након приспећа у систем, посао морати да чека у реду за чекање једнака је:

$$\begin{aligned} P(k \geq m) &= \sum_{k=m}^{\infty} \pi_k = \sum_{k=m}^{\infty} \pi_0 \frac{\rho^k m^m}{m!} = \pi_0 \frac{m^m}{m!} \cdot \sum_{k=m}^{\infty} \rho^k \\ &= \pi_0 \frac{m^m}{m!} \cdot \left(\sum_{k=0}^{\infty} \rho^k - \sum_{k=0}^{m-1} \rho^k \right) \\ &= \pi_0 \frac{m^m}{m!} \cdot \left(\frac{1}{1-\rho} - \frac{1-\rho^m}{1-\rho} \right) = \pi_0 \frac{(m\rho)^m}{m!(1-\rho)}. \end{aligned} \tag{3.5}$$

Нека је $N_q = \max(N - m, 0)$ случајна променљива која представља број послова који чекају у реду за чекање M/M/m система. Када је број послова у систему већи или

једнак броју сервера, m, M/M/m систем има *идентично* понашање као M/M/1 систем са интензитетом опслуживања $m\mu$ који је константан и не зависи од броја послова у систему, тј.

$$P\left(N_q = i \mid N \geq m\right) = (1-\rho)\rho^i, \quad i \geq 0.$$

На основу (2.5), добија се:

$$\mathrm{E}\left(N_q \mid N \geq m\right) = \frac{\rho}{1-\rho}. \tag{3.6}$$

Како је

$$\mathrm{E}\left(N_q\right) = \mathrm{E}\left(N_q \mid N \geq m\right) P\left(N \geq m\right) + \underbrace{\mathrm{E}\left(N_q \mid N < m\right)}_{0} P\left(N < m\right),$$

на основу (3.6) и (3.5) добија се:

$$\begin{aligned}\mathrm{E}\left(N_q\right) &= \frac{\rho}{1-\rho} P\left(N \geq m\right) = \frac{\rho}{1-\rho} \cdot \pi_0 \frac{(m\rho)^m}{m!(1-\rho)} \\ &= \pi_0 \frac{m^m \rho^{m+1}}{m!(1-\rho)^2}.\end{aligned} \tag{3.7}$$

На основу просечне дужине реда за чекање $Q = \mathrm{E}\left(N_q\right)$ лако се проналазе остале мере перформанси, као што ће се видети у наставку.

Мере перформанси M/M/m система рачунају се на следећи начин:

- искоришћеност је једнака односу интензитета приспећа послова у систем и укупног интензитета опслуживања, тј.
$$U = \rho = \frac{\lambda}{m\mu}; \tag{3.8}$$
- проток је једнак интензитету приспећа послова у систем, тј.
$$X = \lambda; \tag{3.9}$$
- дужина реда за чекање добија се коришћењем (3.7):
$$Q = \pi_0 \frac{m^m \rho^{m+1}}{m!(1-\rho)^2}; \tag{3.10}$$
- време чекања добија се коришћењем Литловог закона:
$$W = \frac{Q}{X} = \pi_0 \frac{m^m \rho^{m+1}}{m!(1-\rho)^2 \lambda}; \tag{3.11}$$
- време одзива добија се коришћењем релације (3.3):
$$R = W + \frac{1}{\mu} = \pi_0 \frac{m^m \rho^{m+1}}{m!(1-\rho)^2 \lambda} + \frac{1}{\mu}; \tag{3.12}$$

- број послова добија се коришћењем Литловог закона:

$$J = XR = \pi_0 \frac{m^m \rho^{m+1}}{m!(1-\rho)^2} + \frac{\lambda}{\mu} = \pi_0 \frac{m^m \rho^{m+1}}{m!(1-\rho)^2} + m\rho. \tag{3.13}$$

Треба приметити да се за $m = 1$ све горе наведене формуле своде на одговарајуће формуле за M/M/1 систем.

3.2 Мреже са опслужним центрима

Мреже са опслужним центрима су сложенији модели који се састоје од два или више међусобно повезаних опслужних центара, а користе се за моделовање комплекснијих система са већим бројем ресурса. У наставку ће се када год се помене реч *мрежа* подразумевати мрежа са опслужним центрима, док ће се уместо термина *опслужни центар* користити реч *чвор*.

Основна класификација мрежа са опслужним центрима је на *отворене* и *затворене* мреже.

Отворене мреже су оне у којима послови могу да улазе и излазе из мреже, односно број послова у мрежи није ограничен. Прецизније, посао улази у мрежу тако што се додаје у ред за чекање неког од чворова у мрежи или, ако на том чвору постоји слободан сервер, одмах почиње да се обрађује. Након тога посао наставља свој пут кроз мрежу у складу са вероватноћама које су додељене свакој од путања којима је чвор на коме се тренутно посао налази повезан са осталим чворовима у мрежи.

За разлику од тога, у затвореним мрежама циркулише фиксан број послова, тј. нови послови не могу да улазе у мрежу, а постојећи послови не могу да напусте мрежу. Понекада се дозвољава да послови напуштају мрежу, али се у том случају претпоставља да за сваки посао који напусти мрежу у њу улази нови посао, те се број послова у мрежи ефективно не мења.

У наставку ће се користити следећа нотација:

- N — број чворова у мрежи;
- K — број послова у затвореној мрежи (константа);
- k_i — стање i-тог чвора, односно број послова у i-том чвору мреже ($1 \leq i \leq N$);
- $\mathbf{k} = (k_1, k_2, \ldots, k_N)$ — стање мреже, односно уређена N-торка коју чине бројеви послова који се налазе у појединим чворовима мреже;
- $\pi(k_1, k_2, \ldots, k_N)$ — стационарна вероватноћа стања мреже;
- $\pi_i(k)$ — тзв. маргинална вероватноћа стања мреже; представља вероватноћу да се у i-том чвора мреже налази тачно k послова ($1 \leq i \leq N$, $k \geq 0$; $k \leq K$ ако је мрежа затворена); рачуна се као збир одговарајућих стационарних вероватноћа стања мреже:
  - ако је мрежа отворена:

$$\pi_i(k) = \sum_{k_i = k} \pi(k_1, k_2, \ldots, k_N);$$

– ако је мрежа затворена:

$$\pi_i(k) = \sum_{\substack{\sum_{j=1}^{N} k_j = K \\ \wedge \\ k_i = k}} \pi(k_1, k_2, \ldots, k_N);$$

- m_i — број паралелних сервера у i-том чвору мреже ($1 \leq i \leq N$, $m_i \geq 1$);
- μ_i — интензитет опслуживања на серверу у оквиру i-тог чвора мреже;
- p_{0i} — вероватноћа да посао уђе у мрежу преко i-тог чвора ($1 \leq i \leq N$, $0 \leq p_{0i} \leq 1$);
- p_{ij} — вероватноћа преласка посла са i-тог чвора на j-ти чвор мреже ($1 \leq i, j \leq N$, $0 \leq p_{ij} \leq 1$);
- p_{i0} — вероватноћа да посао напусти отворену мрежу након обраде на i-том чвору ($1 \leq i \leq N$, $0 \leq p_{i0} \leq 1$);
- λ_i — интензитет тока послова кроз i-ти чвор мреже ($1 \leq i \leq N$);
- α_i — интензитет улазног тока послова кроз i-ти чвор отворене мреже ($1 \leq i \leq N$).

Интензитет тока послова кроз i-ти чвор отворене мреже једнак је збиру интензитета улазних токова послова кроз i-ти чвор и збиру токова који потичу од осталих чворова у мрежи кроз i-ти чвор (тзв. *унутрашњи* или *интерни* токови), тј.

$$\lambda_i = \alpha_i + \sum_{j=1}^{N} p_{ji}\lambda_j, \quad 1 \leq i \leq N. \tag{3.14}$$

Једначине (3.14) називају се *једначине баланса токова* (енгл. *traffic equations*). Ове једначине се могу трансформисати у еквивалентну компактнију матричну једначину која је погоднија за рачунање интензитета токова на рачунару. Нека важи:

$$\boldsymbol{\lambda} = \begin{bmatrix} \lambda_1 \\ \lambda_2 \\ \vdots \\ \lambda_N \end{bmatrix}, \quad \boldsymbol{\alpha} = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_N \end{bmatrix}, \quad \mathbf{P} = \begin{bmatrix} p_{11} & p_{12} & \cdots & p_{1N} \\ p_{21} & p_{22} & \cdots & p_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ p_{N1} & p_{N2} & \cdots & p_{NN} \end{bmatrix}.$$

Тада се једначине (3.14) могу записати као:

$$\boldsymbol{\lambda} = \boldsymbol{\alpha} + \mathbf{P}^{\top}\boldsymbol{\lambda}. \tag{3.15}$$

Решавањем по $\boldsymbol{\lambda}$ добија се:

$$\boldsymbol{\lambda} = (\mathbf{I} - \mathbf{P}^{\top})^{-1}\boldsymbol{\alpha}, \tag{3.16}$$

Треба приметити да матрица $\mathbf{P}$ *није стохастичка*, тј. збир вероватноћа преласка у редовима матрице $\mathbf{P}$ који одговарају оним чворовима са којих послови могу да напусте систем мањи је од 1 и износи $1 - p_{i0}$, $i = 1, 2, \ldots, N$. Ова чињеница је важна, јер утиче на то да $(\mathbf{I} - \mathbf{P}^{\top})^{-1}$ постоји, тј. да матрица $(\mathbf{I} - \mathbf{P}^{\top})$ није сингуларна.

Затворена мрежа нема улазних токова послова, те се једначине (3.14) своде на:

$$\lambda_i = \sum_{j=1}^{N} p_{ji}\lambda_j, \quad 1 \le i \le N.$$

или, ако се поново искористи матрична нотација:

$$\boldsymbol{\lambda} = \mathbf{P}^\top \boldsymbol{\lambda}. \tag{3.17}$$

У овом случају матрица $\mathbf{P}$ *јесте стохастичка*, тј. $\sum_{j=1}^{N} p_{ij} = 1$ за свако $i = 1, 2, \ldots, N$. Систем једначина (3.17) има бесконачно много решења, јер је број линеарно независних једначина $N - 1$, а број непознатих N. Зато се уводи још један битан параметар који квантификује кретање послова у мрежи — *просечан број посета чвору* или *релативни проток кроз чвор*:

$$v_i = \frac{\lambda_i}{\lambda}, \quad 1 \le i \le N. \tag{3.18}$$

Код отворених мрежа λ представља укупни проток послова кроз мрежу који је, када се мрежа налази у равнотежи, једнак укупном интензитету улазног тока послова, тј. $\lambda = \sum_{i=1}^{N} \alpha_i$. Дељењем обе стране једначине (3.14) са λ добија се:

$$v_i = p_{0i} + \sum_{j=1}^{N} p_{ji} v_j, \quad 1 \le i \le N. \tag{3.19}$$

или, увођењем матрица

$$\mathbf{p} = \begin{bmatrix} p_{01} \\ p_{02} \\ \vdots \\ p_{0N} \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_N \end{bmatrix},$$

добија се:

$$\mathbf{v} = \mathbf{p} + \mathbf{P}^\top \mathbf{v},$$

што, решавањем по $\mathbf{v}$, даје:

$$\mathbf{v} = (\mathbf{I} - \mathbf{P}^\top)^{-1} \mathbf{p}. \tag{3.20}$$

Код затворених мрежа, вредност λ у једначини (3.18) се обично бира тако да је једнака протоку кроз један унапред изабрани чвор у мрежи који се назива *референтни чвор*. Нека је то, једноставности ради, а без губитка општости, први чвор у мрежи, односно чвор којем одговара индекс 1; тада је $\lambda = \lambda_1$, односно $v_1 = 1$. Средњи број посета осталим чворовима у мрежи добија се на основу следећег система једначина:

$$v_i = p_{1i} v_1 + p_{2i} v_2 + \cdots + p_{Ni} v_N = p_{1i} + \sum_{j=2}^{N} p_{ji} v_j, \quad 2 \le i \le N, \tag{3.21}$$

Нека је

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_N \end{bmatrix}, \quad \mathbf{p} = \begin{bmatrix} p_{12} \\ p_{13} \\ \vdots \\ p_{1N} \end{bmatrix}.$$

Увођењем подматрица

$$\mathbf{v}_{\text{sub}} = \begin{bmatrix} v_2 \\ \vdots \\ v_N \end{bmatrix}, \quad \mathbf{P}_{\text{sub}} = \begin{bmatrix} p_{22} & p_{23} & \cdots & p_{2N} \\ p_{32} & p_{33} & \cdots & p_{3N} \\ \vdots & \vdots & \ddots & \vdots \\ p_{N2} & p_{N3} & \cdots & p_{NN} \end{bmatrix},$$

добија се следећа матрична једначина која је еквивалентна једначини (3.21):

$$\mathbf{v}_{\text{sub}} = \mathbf{p} + \mathbf{P}_{\text{sub}}^{\top}\mathbf{v}_{\text{sub}}. \tag{3.22}$$

Треба приметити да матрица $\mathbf{P}_{\text{sub}}$ *није стохастичка*. Решавањем једначине (3.22) по аналогији са (3.16) добија се:

$$\mathbf{v}_{\text{sub}} = (\mathbf{I} - \mathbf{P}_{\text{sub}}^{\top})^{-1}\mathbf{p}. \tag{3.23}$$

Дељењем просечног број посета i-том опслужном центру са брзином којом исти обрађује послове добија се *временска потражња* за i-тим опслужним центром:

$$d_i = \frac{v_i}{\mu_i}, \quad 1 \leq i \leq N. \tag{3.24}$$

Ако се временске потражње за чвором у затвореној мрежи поделе са интензитетом опслуживања референтног чвора добија се *нормализована временска потражња* за чвором или *релативна искоришћеност* чвора:

$$x_i = \frac{d_i}{\mu_1}, \quad 1 \leq i \leq N. \tag{3.25}$$

Дакле, важи $x_1 = 1$, при чему је 1 индекс референтног чвора у затвореној мрежи. Лако се доказује да је однос искоришћености два сервисна центра једнак односу њихових релативних искоришћености:

$$\frac{U_i}{U_j} = \frac{x_i}{x_j}, \quad 1 \leq i, j \leq N.$$

3.2.1 Мере перформанси

Мере перформанси за сваки чвор у мрежи дефинишу се на идентичан начин као што је то учињено у 3.1.1, с тим што ће се за означавање чвора користити индекс уз ознаку саме мере перформанси; тако, на пример, ако се говори о времену одзива, за први чвор мреже користиће се ознака R_1, за други чвор R_2, итд.

Корисно је дефинисати и три мере перформанси на нивоу читаве мреже за чије означавање се неће користити индекс и по томе ће се разликовати од мера перформанси појединачних чворова мреже:

- проток:
  - ако је мрежа отворена, онда је проток интензитет којим послови напуштају отворену мрежу; за мрежу која се налази у равнотежи, проток је једнак укупном интензитету улазних токова послова у мрежу, тј.

$$X = \sum_{i=1}^{N} \alpha_i; \tag{3.26}$$

  - ако је мрежа затворена, онда је проток мреже једнак протоку кроз референтни чвор мреже; ако је то чвор са индексом 1, онда важи

$$X = X_1; \tag{3.27}$$

- време одзива:
  - ако је мрежа отворена, онда време одзива представља просечно време потребно да посао напусти мрежу; према томе:

$$R = \sum_{i=1}^{N} v_i R_i; \tag{3.28}$$

  - ако је мрежа затворена, онда се поново може користити релација 3.28, с тим што у овом случају резултујућа вредност представља просечно време које протекне између две сукцесивне посете посла референтном чвору мреже;
- број послова — просечан број послова у мрежи;
  - ако је мрежа отворена, онда је то збир просечних бројева послова у сваком чвору мреже:

$$J = \sum_{i=1}^{N} J_i; \tag{3.29}$$

  - ако је мрежа затворена, онда је број послова унапред дефинисана константа K, тј.

$$J = K. \tag{3.30}$$

Понашање многих мрежа са опслужним центрима може се моделовати коришћењем МЛНВ, те се стационарне вероватноће стања мреже могу добити решавањем једначина (2.27) и (2.28) које се називају *једначинама глобалне равнотеже*. Међутим, као што ће у наставку постати јасно, постоје мреже са опслужним центрима код којих се вероватноће стања мреже могу израчунати на знатно једноставнији начин.

3.2.2 Мреже са решењем у облику производа

За i-ти чвор мреже се каже да има *својство локалне равнотеже* ако за свако стање мреже $\mathbf{k} \in S$ важи следеће: интензитет којим се из стања $\mathbf{k}$ прелази у било које друго стање мреже $\mathbf{k}' \in S$ ($\mathbf{k}' \neq \mathbf{k}$) као последица тога што посао *напушта* i-ти чвор мреже једнак је интензитету којим се из стања $\mathbf{k}'$ прелази у стање $\mathbf{k}$ као последица тога што

посао *пристиже* у i-ти чвор мреже. Ако сваки чвор мреже има својство локалне равнотеже, онда се за *целу мрежу* каже да има својство локалне равнотеже. За мреже које имају својство локалне равнотеже важи следеће тврђење: вероватноћа било ког стања мреже може се изразити као производ константе и производа маргиналних вероватноћа стања, односно

$$\pi\left(k_1, k_2, \ldots, k_N\right) = \frac{1}{G}\prod_{i=1}^{N}\pi_i\left(k_i\right), \quad \left(k_1, k_2, \ldots, k_N\right) \in S. \tag{3.31}$$

Константа G се назива *нормализациона константа*.

Џексонове мреже

Прави пробој у анализи мрежа са опслужним центрима донели су научни радови Џејмса Џексона[3] [7, 8] који је успео да пронађе решење у облику производа за значајну класу *отворених* мрежа. Његов резултат наведен је у облику следеће теореме:

Теорема 3.2.1 (Џексонова теорема)**.** *Нека отворену мрежу чини N -/M/m_i/∞-FCFS чворова и нека су сви улазни токови послова Поасонови са интензитетима приспећа α_i, $i = 1, 2, \ldots, N$. Ако је сваки чвор мреже стабилан (или, еквивалентно, одговарајући МЛНВ ергодичан), тј. ако важи $\rho_i < 1$, где је $\rho_i = \lambda_i/(m_i\mu_i)$, при чему се λ_i рачуна коришћењем једначине* (3.16)*, онда се стационарне вероватноће стања мреже могу изразити као производ одговарајућих маргиналних вероватноћа стања мреже:*

$$\pi\left(k_1, k_2, \ldots, k_N\right) = \prod_{i=1}^{N}\pi_i\left(k_i\right). \tag{3.32}$$

Треба приметити да је релација (3.32) посебан случај раније уведене општије релације (3.31), при чему је нормализациона константа једнака 1. Џексон је показао да је маргинална вероватноћа стања мреже $\pi_i\left(k\right)$ једнака вероватноћи стања одговарајућег изолованог M/M/m_i-FIFO система са улазним током послова интензитета λ_i, те се мере перформанси појединачних чворова мреже рачунају коришћењем формула (3.8)-(3.13). Перформансе које се односе на целу мрежу рачунају се формулама (3.26), (3.28) и (3.29).

Гордон-Њуелове мреже и Бјузенов алгоритам

Гордон[4] и Њуел[5] су разматрали затворене мреже за које су увели исте претпоставке као што је увео Џексон за отворене мреже, сем што нема улазних нити излазних токова послова, тј. број послова у мрежи $K \geq 1$ представља константу:

$$K = \sum_{i=1}^{N} k_i.$$

[3]James R. Jackson (1924 — 2011) је био амерички математичар.

[4]William J. Gordon је био амерички математичар и инжењер.

[5]Gordon F. Newell (1925 — 2001) је био амерички математичар и физичар, професор емеритус саобраћајног инжењерства.

Број стања мреже је *коначан* (што није био случај код Џексонових отворених мрежа) и једнак биномном коефицијенту

$$\binom{K+N-1}{N-1} = \binom{K+N-1}{K},$$

тј. израчунава се као број начина да се K послова распореди у N чворова мреже. Гордон и Њуел су показали да се стационарне вероватноће стања мреже могу рачунати на следећи начин [9]:

$$\pi(k_1, k_2, \ldots, k_N) = \frac{1}{G(K)} \prod_{i=1}^{N} F_i(k_i). \tag{3.33}$$

У горњој једначини број $G(K)$ се, као и у једначини (3.31), назива *нормализациона константа* и за њу важи:

$$G(K) = \sum_{\sum_{i=1}^{N} k_i = K} \prod_{i=1}^{N} F_i(k_i).$$

Функције $F_i(k_i)$ $(1 \leq i \leq N,\ 0 \leq k_i \leq K)$ имају различит облик за сваки чвор затворене мреже који зависи од броја сервера у чвору, m_i, и нормализоване временске потражње за чвором (видети (3.25)), x_i:

$$F_i(k_i) = \frac{x_i^{k_i}}{\beta_i(k_i)}, \quad 1 \leq i \leq N,\ 0 \leq k_i \leq K. \tag{3.34}$$

Функција $\beta_i(k_i)$ је дефинисана на следећи начин:

$$\beta_i(k_i) = \begin{cases} 1, & m_i = 1, \\ k_i!, & k_i \leq m_i, \\ m_i! m_i^{k_i - m_i}, & k_i > m_i. \end{cases}$$

Међутим, често се користи општији облик функције (3.34) у којој се користи просечан број посета чвору и чињеница да се чвор са више од једног сервера може третирати као чвор са једним сервером и интензитетом опслуживања који зависи од броја послова у чвору (видети (3.4)):

$$F_i(k_i) = \frac{v_i^{k_i}}{A_i(k_i)}, \quad 1 \leq i \leq N,\ 0 \leq k_i \leq K, \tag{3.35}$$

где је

$$A_i(k_i) = \begin{cases} 1, & k_i = 0, \\ \prod_{j=1}^{k_i} \mu_i(j), & k_i > 0. \end{cases}$$

Дакле, вероватноћа стања $(k_1, k_2, \ldots, k_N)$ затворене мреже дата са (3.33) сразмерна је производу степена нормализованих временских потражњи за чворовима (видети (3.34)), односно производу степена просечног броја посета чворовима (видети (3.35)). На основу вероватноћа стања мреже даље се могу рачунати одговарајуће мере перформанси.

Очигледан проблем са директном применом Гордон-Њуелове методе јесте велика временска сложеност рачунања константе $G(K)$, јер је потребно израчунати суму

$$\binom{K+N-1}{K}$$

производа. Зато, у пракси се користи Бјузенов алгоритам који је био један од првих ефикасних алгоритама за израчунавање нормализационе константе $G(K)$ само на основу функција $F_i(k_i)$.

Уводи се помоћна функција $G_n(k)$ дефинисана на следећи начин:

$$G_n(k) = \sum_{\sum_{i=1}^{n} k_i = k} \prod_{i=1}^{n} F_i(k_i), \quad n = 1, 2, \ldots, N. \tag{3.36}$$

За $n > 1$ једначина (3.36) се може трансформисати на следећи начин:

$$\begin{aligned} G_n(k) &= \sum_{j=0}^{k} \sum_{\substack{\sum_{i=1}^{n} k_i = k \\ \wedge \\ k_n = j}} \prod_{i=1}^{n} F_i(k_i) \\ &= \sum_{j=0}^{k} F_n(j) \sum_{\sum_{i=1}^{n-1} k_i = k-j} \prod_{i=1}^{n-1} F_i(k_i) \\ &= \sum_{j=0}^{k} F_n(j) G_{n-1}(k-j), \quad n = 1, 2, \ldots, N. \end{aligned}$$

Коришћењем вектора

$$\mathbf{G}_n = \begin{bmatrix} G_n(0) \\ G_n(1) \\ \vdots \\ G_n(K) \end{bmatrix} \quad \text{и} \quad \mathbf{F}_n = \begin{bmatrix} F_n(0) \\ F_n(1) \\ \vdots \\ F_n(K) \end{bmatrix}, \quad n = 1, 2, \ldots, N,$$

горње једначине се могу записати на следећи начин:

$$\mathbf{G}_n = \mathbf{F}_n \otimes \mathbf{G}_{n-1}, \quad n = 2, 3, \ldots, N. \tag{3.37}$$

Оператор $\otimes$ искоришћен у релацији (3.37) је сличан оператору конволуције две функције, с тим што се, уместо уобичајених $2K+1$ елемената резултујућег вектора, користи само првих $K+1$ елемената, јер толико елемената има низ могућих бројева послова у мрежи: $\{0, 1, 2, \ldots, K\}$. Због овога се у литератури Бјузенов алгоритам назива и *конволуционим алгоритмом* [3]. Треба приметити да једначина (3.37) није дефинисана за $n = 1$. Уместо ње, за израчунавање вектора $\mathbf{G}$ у том (иницијалном) случају користи се $\mathbf{G}_1 = \mathbf{F}_1$. Како је, по дефиницији, $F_n(0) = 1$ за $n = 1, 2, \ldots, N$, то је $G_n(0) = 1$ за $n = 1, 2, \ldots, N$. Лако се показује да је нормализациона константа $G(K)$ једнака $G_N(K)$.

Временска сложеност Бјузеновог алгоритма је $\Theta(NK)$ (за сваки од N чворова у мрежи реализује се оператор $\otimes$ над векторима дужине $K+1$), док је просторна сложеност $\Theta(K)$ (ако се користи тзв. *in-place* ажурирање низа $\mathbf{G}$ који има $K+1$ елемената, при

чему се индекс петље декрементира, тј. редом узима вредности $K, K-1, K-2, \ldots, 0$), јер, као што ће у наставку постати јасно, за одређивање свих мера перформанси довољно је коришћење само вектора $\mathbf{G}_N$, док се сви остали вектори могу одбацити. Због тога се N обично не наглашава експлицитно у индексу, тј. пише се $G(k)$ уместо $G_N(k)$, при чему $k = 0, 1, \ldots, K$.

За рачунање мера перформанси чворова мреже потребно је, у општем случају, познавати одговарајуће маргиналне вероватноће стања мреже:

$$\begin{aligned}
\pi_i(k) &= \sum_{\substack{\sum_{j=1}^{N} k_j = K \\ \wedge \\ k_i = k}} \pi(k_1, k_2, \ldots, k_N) \\
&= \sum_{\substack{\sum_{j=1}^{N} k_j = K \\ \wedge \\ k_i = k}} \frac{1}{G(K)} \prod_{j=1}^{N} F_j(k_j) \\
&= \frac{F_i(k)}{G(K)} \sum_{\substack{\sum_{j=1}^{N} k_j = K-k \\ \wedge \\ j \neq i}} \prod_{\substack{j=1 \\ \wedge \\ j \neq i}}^{N} F_j(k_j) \\
&= \frac{F_i(k)}{G(K)} G_N^{(i)}(K-k), \quad 1 \leq i \leq N,\ 0 \leq k \leq K. \qquad (3.38)
\end{aligned}$$

Вредност $G_N^{(i)}(k)$ представља нормализациону константу у затвореној мрежи са k послова у којој је i-ти чвор мреже *уклоњен*. Како је, по дефиницији,

$$G_{N-1}(k) = G_N^{(N)}(k), \quad k = 0, 1, \ldots, K,$$

специјално, за $i = N$ релација (3.38) постаје:

$$\pi_N(k) = \frac{F_N(k)}{G(K)} G_{N-1}(K-k), \quad 1 \leq i \leq N,\ 0 \leq k \leq K.$$

Маргиналне вероватноће које имају индекс различит од N могу се израчунати коришћењем уопштене рекурзивне формуле за израчунавање вредности $G_N^{(i)}(k)$ за $1 \leq i \leq N$. Наиме, како збир маргиналних вероватноћа мора бити једнак 1, то се коришћењем једначине (3.38) добија:

$$\sum_{k=0}^{K} \pi_i(k) = \sum_{k=0}^{K} \frac{F_i(k)}{G(K)} G_N^{(i)}(K-k) = 1,$$

односно

$$G(K) = \sum_{j=0}^{K} F_i(j) G_N^{(i)}(K-j), \quad 1 \leq i \leq N.$$

Коришћењем ових једначина може се извести следећа рекурзивна формула за израчунавање $G_N^{(i)}(k)$:

$$G_N^{(i)}(k) = G(k) - \sum_{j=1}^{k} F_i(j) G_N^{(i)}(k-j), \quad 1 \le i \le N,\, 0 \le k \le K, \tag{3.39}$$

са иницијалним условом $G_N^{(i)}(0) = G(0) = 1$ за $1 \le i \le N$.

Мере перформанси i-тог опслужног центара у затвореној мрежи ($i = 1, 2, \ldots, N$) рачунају се на следећи начин:

- проток:
$$X_i = v_i \frac{G(K-1)}{G(K)}; \tag{3.40}$$

- искоришћеност:
$$U_i = \frac{X_i}{m_i \mu_i} = \frac{v_i}{m_i \mu_i} \frac{G(K-1)}{G(K)}; \tag{3.41}$$

- дужина реда за чекање:

  – уколико опслужни центар има тачно један сервер ($m_i = 1$) може се користи следећа упрошћена једначина:
$$Q_i = \begin{cases} 0, & K = 1; \\ \frac{1}{G(K)} \sum_{k=2}^{K} \left(\frac{v_i}{\mu_i}\right)^k G(K-k), & K \ge 2; \end{cases} \tag{3.42}$$

  – уколико опслужни центар има $m_i \ge 2$ сервера, морају се користити одговарајуће маргиналне вероватноће стања мреже:
$$Q_i = \begin{cases} 0, & K \le m_i; \\ \sum_{k=m_i+1}^{K} (k - m_i) \pi_i(k), & K > m_i; \end{cases} \tag{3.43}$$

- време чекања добија се коришћењем Литловог закона:
$$W_i = \frac{Q_i}{X_i}; \tag{3.44}$$

- време одзива добија се коришћењем формуле (3.3):
$$R_i = W_i + \frac{1}{\mu_i}; \tag{3.45}$$

- број послова добија се коришћењем Литловог закона:
$$J_i = R_i X_i; \tag{3.46}$$

Перформансе које се односе на целу мрежу рачунају се формулама (3.27), (3.28) и (3.30).

Поглавље 4

СИМУЛАЦИЈА ЗАСНОВАНА НА ДИСКРЕТНИМ ДОГАЂАЈИМА

Симулација заснована на дискретним догађајима (енгл. *discrete event simulation* или, кратко, DES) је веома често коришћена симулациона метода за евалуацију мера перформанси система који се моделују мрежама са опслужним центрима. Ова техника подразумева да се током симулације обрађују *догађаји* (биће означени са $e_1, e_2, ...$) који се дешавају у одређеним *временским тренуцима* ($t_1, t_2, ...$), при чему обрада сваког догађаја изазива *промену стања симулиране мреже*. Време у симулацији напредује *скоковито* (са t_i на t_{i+1} када треба да се обради догађај e_{i+1}, $i = 1, 2, ...$), а догађаји се обрађују у редоследу у коме су се десили (другим речима, увек важи $t_{i+1} \geq t_i$, $i = 1, 2, ...$). Постоје три типа догађаја који се обрађују током симулације:

- *долазак новог посла у мрежу* — овај тип догађаја се користи само код отворених мрежа. Реализација овог догађаја подразумева да се у ред за чекање неког од чворова мреже дода нови посао или, ако на том чвору постоји слободан сервер, одмах започне његова обрада;

- *почетак обраде посла* — овај догађај се дешава када сервер у неком чвору мреже почиње да обрађује посао; посао долази директно из остатка мреже или се узима из реда за чекање. У овом случају, посао се уклања из реда за чекање и додаје се у листу послова који се тренутно обрађују на том чвору;

- *завршетак обраде посла* — овај догађај се дешава када сервер у неком чвору мреже заврши обраду посла. У овом случају, посао се уклања из листе послова који се тренутно обрађују на том чвору и, у зависности од тога која је путања изабрана у складу са стратегијом рутирања на том чвору, посао се или додаје у ред за чекање неког од других чворова мреже или се уклања из мреже (ако је мрежа отворена).

DES има две веома битне предности у односу на друге симулационе методе:

- *јединствен ток контроле* — с обзиром да се догађаји обрађују секвенцијално у петљи која се извршава у оквиру једне нити или процеса, избегавају се класични проблеми конкурентног програмирања (*race condition*, *deadlock*, *livelock* итд.) и потреба за коришћењем сложених механизама синхронизације који, између осталог, инхерентно имају негативан утицај на брзину извршавања симулације. Симулациони код је, самим тим, једноставан, стабилан и лак за одржавање;

- *поновљивост* — с обзиром да се због јединственог тока контроле без проблема може користити глобални (заједнички, дељени) генератор псеудо-случајних бројева, симулација се може покретати више пута са истом иницијализационом вредношћу (енгл. *seed*) при чему ће се, под условом да је модел непромењен, увек добити исте вредности мера перформанси, тј. симулација се извршава у потпуности детерминистички.

4.1 Мере перформанси

Мере перформанси дефинисане у 3.1.1 и 3.2.1 оцењују се статистичком анализом узорака прикупљених током симулације. Формат прикупљених узорака који ће бити наведен у наставку сличан је оном који је коришћен у алату JMT (видети [10]). Овај формат је пожељан јер се, као што ће се видети у наставку, његовим коришћењем омогућава *униформна статистичка обрада* података без обзира о којој мери перформанси је реч.

За сваку меру перформанси генерише се низ уређених тројки облика

$$(t_i, v_i, w_i), \quad i = 1, 2, \ldots, N, \tag{4.1}$$

где је t_i временски тренутак у коме се догађај e_i десио, v_i — вредност која се бележи у симулацији, а која је релевантна за одговарајућу меру перформанси, а w_i одговарајућа тежина (пондер[1]). N је обим узорка генерисаног током симулације. Узорачка средина мере перформанси M, означена са $\hat{M}$, израчунава се као пондерисана средина узорка $(v_1, v_2, \ldots, v_N)$ коришћењем тежина $(w_1, w_2, \ldots, w_N)$ на један од следећа два начина:

$$\hat{M} = \frac{\sum_{i=1}^{N} w_i v_i}{\sum_{i=1}^{N} w_i} \tag{4.2}$$

или

$$\hat{M} = \left[\frac{\sum_{i=1}^{N} w_i v_i}{\sum_{i=1}^{N} w_i} \right]^{-1}. \tag{4.3}$$

У следећим табелама наведено је шта тачно представљају v_i и w_i приликом генерисања узорка који се користи за оцену одговарајуће мере перформанси чвора у мрежи:

[1] Француска реч *pondère* значи тежина или важност. Води порекло из латинског *pondus*, што значи тежина.

Табела 4.1: *Генерисање узорка и тежина за оцену мера перформанси чворова у мрежи.*

Мера перформанси	v_i	w_i	Формула
Искоришћеност	n_{busy}	Δt_i	(4.2)
Проток	Δt_i	1	(4.3)
Број послова	k	Δt_i	(4.2)
Време одзива	$t_{\text{departure}} - t_{\text{arrival}}$	1	(4.2)
Време чекања	$t_{\text{start service}} - t_{\text{arrival}}$	1	(4.2)
Дужина реда за чекање	q	Δt_i	(4.2)

У табели изнад:

- n_{busy} је тренутни број заузетих сервера на датом чвору;
- Δt_i је време које је протекло од последњег евидентирања вредности за дату меру перформанси на датом чвору које се десило у неком тренутку t_j, при чему је $j < i$; уколико се ради о првој вредности, узима се тренутак реализације догађаја, t_i:

$$\Delta t_i = \begin{cases} t_i - t_j, & \text{постоји претходно регистрована вредност;} \\ t_i, & \text{иначе;} \end{cases}$$

- k је тренутни број послова у чвору мреже,
- q је тренутни број послова у реду за чекање;
- t_{arrival} је време доласка посла у чвор;
- $t_{\text{start service}}$ је време почетка обраде посла на чвору;
- $t_{\text{departure}}$ је време када посао напушта чвор.

На сличан начин се генеришу узорак и тежине које се користе за оцену мера перформанси мреже у целини:

Табела 4.2: *Генерисање узорка и тежина за оцену мера перформанси мреже.*

Мера перформанси	v_i	w_i	Формула
Проток	Δt_i	1	(4.3)
Број послова	k	Δt_i	(4.2)
Време одзива	$t_{\text{departure}} - t_{\text{arrival}}$	1	(4.2)

У табели изнад:

- Δt_i је време које је протекло од последњег евидентирања вредности за дату меру перформанси које се десило у неком тренутку t_j, при чему је $j < i$; уколико се ради о првој вредности, узима се тренутак реализације догађаја, t_i:

$$\Delta t_i = \begin{cases} t_i - t_j, & \text{постоји претходно регистрована вредност;} \\ t_i, & \text{иначе;} \end{cases}$$

треба нагласити да се, у случају евидентирања узорка за оцену протока користи референти чвор мреже ако је мрежа затворена.

- k је тренутни број послова у мрежи (у случају затворене мреже, узорак је увек дужине 1, јер се број послова у мрежи не мења);
- t_{arrival} је време доласка посла у мрежу (ако је мрежа отворена) или у референтни чвор мреже (ако је мрежа затворена);
- $t_{\text{departure}}$ је време када посао напушта мрежу (ако је мрежа отворена) или референтни чвор мреже (ако је мрежа затворена).

Како би се елиминисала зависност између појединачних мерења у оквиру узорка (што је услов за примену централне граничне теореме), симулација се понавља R пута са различитом иницијализационом вредношћу, при чему се редом генеришу узорци обима $N_1, N_2, \ldots, N_R$. За сваки узорак рачуна се узорачка средина $\hat{M}_i$ $(i = 1, 2, \ldots, R)$ коришћењем једначина (4.2) или (4.3). Овако добијене узорачке средине *jecy* независне и имају идентичну расподелу, те се може применити (2.8). На основу средње вредности узорачких средина

$$\hat{M} = \frac{1}{R} \sum_{i=1}^{R} \hat{M}_i$$

и оцене стандардне девијације

$$s = \sqrt{\frac{1}{R-1} \sum_{i=1}^{R} (\hat{M}_i - \hat{M})^2}$$

добија се интервална оцена било које мере перформанси:

$$\left[\hat{M} - q_{R-1,1-\alpha/2} \frac{s}{\sqrt{R}}, \hat{M} + q_{R-1,1-\alpha/2} \frac{s}{\sqrt{R}} \right]. \tag{4.4}$$

Како би се смањио утицај транзијентних стања система на резултат, пожељно је одређени број елемената узорка генерисаних на почетку симулације одбацити, тј. не узимати у обзир приликом израчунавања интервалне оцене (4.4).

Поглавље 5

АНАЛИЗА ЗАХТЕВА

Ради једноставности, када се у остатку овог поглавља помене реч *апликација* мислиће се на апликацију која је реализована у оквиру овог рада. *Циљни корисници* апликације јесу студенти који се први пут сусрећу са процесом моделовања система и евалуације мера перформанси система на основу модела. *Намена апликације* је да корисницима омогући лако креирање модела и олакша разумевање појединих метода којима се долази до индикатора перформанси система.

Имајући ово у виду, у наставку су најпре дати захтеви који се односе на графички кориснички интерфејс апликације; затим, кроз опис одговарајућих случајева употребе (енгл. *use cases*), дати су функционални захтеви (енгл. *functional requirements*) које апликација треба да испуни; случајеви употребе су тесно повезани са графичким корисничким интерфејсом апликације и активирају се интеракцијом корисника са графичким корисничким интерфејсом; коначно, наведени су и нефункционални захтеви (енгл. *non-functional requirements*).

Независно којој групи припада, сваком захтеву додељен је јединствен идентификатор чиме се омогућава његово референцирање у остатку рада. Идентификатор захтева је у облику REQ-T-N, где је T тип захтева (FN — функционални захтев, GUI — захтев који се односи на графички кориснички интерфејс, NFN — нефункционални захтев), а N редни број захтева.

У спецификацији захтева у овом раду користе се следећи термини:

Табела 5.1: *Значење термина у спецификацији захтева.*

Термин	Значење
МОРА	Захтев је обавезан и мора бити испуњен.
НЕ СМЕ	Захтев садржи апсолутну забрану и не сме бити прекршен.
МОЖЕ, НЕ МОРА	Захтев обухвата аспект чије испуњење није обавезно.

5.1 Графички кориснички интерфејс

REQ-GUI-01: Општи принципи графичког корисничког интерфејса апликације:

- Ако у неком тренутку нека функционалност у апликацији није доступна, елемент графичког корисничког интерфејса којим се таква функционалност иницира МОРА бити онемогућен, што се визуелно истиче променом боје или нијансе елемента.

Корисник НЕ СМЕ бити у могућности да приступи функционалности која није доступна.

- Уколико се притиском на неко дугме или одабиром неке ставке менија не извршава одмах нека радња, већ се отвара нови прозор у оквиру којег се од корисника очекује неки улаз, тада одговарајуће дугме или ставка менија МОРА садржати три тачке на крају свог натписа, под условом да натпис постоји.

- Дугме или ставка менија која има натпис МОЖЕ, додатно, имати и иконицу (енгл. *icon*) која визуелно представља радњу која се извршава одабиром одговарајућег елемента и олакшава његово препознавање од стране корисника. Дугме или ставка менија која нема натпис МОРА имати најмање иконицу и, додатно, помоћни натпис (енгл. *tooltip*) који се приказује када корисник задржи курсор миша изнад одговарајућег елемента графичког корисничког интерфејса.

- Ставке менија за које се очекује да ће бити коришћене често МОРАЈУ имати одговарајуће пречице на тастатури (енгл. *accelerators*), односно јединствене комбинације тастера којима се приступа одговарајућим функционалностима без отварања менија.

REQ-GUI-02: Главни прозор апликације МОРА имати следеће елементе:

- наслов прозора (енгл. *title bar*) са називом апликације,
- главни мени (енгл. *main menu*),
- трака са алаткама (енгл. *tool bar*),
- главни панел са картицама (енгл. *main tab pane*).

Корисник МОРА бити у могућности да у било ком тренутку, под условом да није отворен неки модални прозор у оквиру апликације, минимизује или максимизује главни прозор апликације. Такође, корисник МОРА бити у могућности да на произвољан начин промени величину главног прозора апликације. Притом, сви елементи графичког корисничког интерфејса МОРАЈУ бити прилагођени тренутним димензијама прозора, односно МОРАЈУ бити функционални и конзистентно распоређени.

REQ-GUI-03: Главни мени апликације МОРА садржати следеће ставке (у заградама су наведене одговарајуће пречице):

- Датотека са следећим ставкама:
  - Нова мрежа (Ctrl+N),
  - Учитај мрежу (Ctrl+O),
  - Недавно учитане мреже,
  - Сачувај (Ctrl+S),
  - Сачувај као (Ctrl+⇧+S),
  - Поништи све измене,

  - Сачувај слику као (Ctrl+E),
  - Излаз (Ctrl+Q);
- Уреди са следећим ставкама:
  - Изабрани елемент мреже са следећим ставкама:
    * Ротирај (Ctrl+R) (уколико изабрани елемент мреже *није* путања; видети [REQ-GUI-05]),
    * Својства (Ctrl+P) (уколико изабрани елемент мреже *није* путања; видети [REQ-GUI-05]),
    * Тип (уколико изабрани елемент мреже *јесте* путања; видети [REQ-GUI--05]) са следећим ставкама:
      · Права линија,
      · Безјеова крива,
      · Више сегмената;
    * Уклони (⌫);
  - Својства мреже... (Ctrl+⇧+P);
- Подешавања са следећом ставком:
  - Језик са следећим ставкама:
    * Српски,
    * English;
- Помоћ са следећим ставкама:
  - О програму.

REQ-GUI-04: Трака са алаткама апликације МОРА садржати три јасно визуелно издвојене групе дугмади:

- група Елементи мреже која садржи:
  - прекидачко дугме (енгл. *toggle button*) Улаз,
  - прекидачко дугме Опслужни центар,
  - прекидачко дугме Излаз,
  - прекидачко дугме Путања;
- група Симулација која садржи:
  - дугме Параметри...,
  - дугме за покретање симулације са падајућом листом из које се бира тип симулације:
    * Аутоматска симулација,
    * Интерактивна симулација;

  - дугме за заустављање симулације,
  - дугме Извези узорке,
  - дугме Мере перформанси...,
  - дугме Шта-ако...;
- група Аналитичко решавање која садржи:
  - дугме Покрени,
  - дугме Мере перформанси...,
  - дугме Шта-ако....

REQ-GUI-05: Једна картица главног панела апликације МОРА садржати и јасно приказивати елементе једне мреже, при чему панел НЕ СМЕ имати софтверско ограничење у погледу максималног броја картица, односно максималног броја тренутно отворених мрежа. Панел може бити празан, односно не мора садржати ни једну картицу. У сваком тренутку МОРА бити активна највише једна картица, при чему НЕ СМЕ постојати ограничење у погледу редоследа којим се картице активирају. Наслов картице у сваком тренутку МОРА бити једнак називу мреже. Корисник МОРА бити у могућности да у сваком тренутку затвори тренутно активну картицу мреже.

Елементи мреже су:

- *улаз* којим корисник дефинише улазне токове послова;
- *опслужни центар* којим корисник дефинише ресурс у мрежи;
- *излаз* којим корисник дефинише излазне токове послова;
- *путања* која повезује два елемента мреже и дефинише смер кретања послова у мрежи.

Улази, опслужни центри и излази МОРАЈУ бити визуелно репрезентовани тако да се лако разликују. Ова три типа елемената мреже МОРАЈУ имати натпис са називом. Додатно, референтни опслужни центар у затвореној мрежи МОРА имати јасну визуелну индикацију у облику иконе круне поред свог назива.

Уколико улаз, опслужни центар или излаз није повезан тако да је топологија мреже исправна, МОРА бити приказана јасан визуелни индикатор у облику црвене иконе упозорења поред назива елемента мреже.

Путање МОРАЈУ бити визуелно репрезентоване као линије које повезују два елемента мреже. МОРАЈУ бити подржана три типа линија: права линија, Безјеова крива и линија коју чини више сегмената. Одабир праве линије и Безјеове криве МОРА бити онемогућен када би њихово исцртавање подразумевало да путања пресече изворишни елемент.

У једном тренутку може бити *изабран* (енгл. *selected*) највише један елемент мреже, при чему се избор врши притиском левог тастера миша док се курсор налази изнад елемента мреже. Изабрани елемент мреже МОРА се разликовати визуелно од осталих елемената у мрежи.

REQ-GUI-06: Модални прозор Својства МОРА садржати панел са хармоником (енгл. *accordion*) са следећим насловљеним панелима (енгл. *titled pane*):

- Опште — садржи:
  - текстуално поље (енгл. *text field*) са натписом (енгл. *label*) Назив за унос назива чвора;
  - контролу за избор боје (енгл. *color picker*) са натписом Боја попуњавања;
  - контролу за избор боје са натписом Боја линије;
  - контролу типа *text area* са натписом Опис и помоћним натписом (енгл. *prompt text*) који подстиче корисника да унесе кратак опис чвора;
- Ред за чекање — садржи:
  - групу радио-дугмади (енгл. *radio button group*) са натписом Капацитет, која садржи:
    - радио-дугме (енгл. *radio button*) Бесконачан, које онемогућава унос капацитета реда за чекање;
    - радио-дугме Коначан, које омогућава бројачку контролу (енгл. *spinner*) за унос целобројне вредности која представља капацитете реда за чекање (минимална дозвољена вредност треба бити 1);
  - падајућу листу (енгл. *choice box*) са натписом Дисциплина за избор дисциплине реда, са следећим опцијама:
    - По редоследу доласка (FIFO),
    - Последњи у реду (LIFO),
    - Случајан избор (SIRO);
- Сервис — садржи:
  - бројачку контролу са натписом Број сервера за унос целобројне вредности која представља број сервера у опслужном центру (минимална дозвољена вредност треба бити 1);
  - падајућу листу са натписом Расподела времена опслуживања за избор расподеле, са следећим опцијама:
    - Експоненцијална,
    - Ерлангова,
    - Паретова,
    - Униформна,
    - Детерминистичка,
    - Хиперекспоненцијална.
  - дугме са натписом ... које отвара модални дијалог за унос параметара изабране расподеле;
- Рутирање — садржи:
  - падајућу листу са натписом Стратегија за избор стратегије рутирања, са следећим опцијама:

        * Избор заснован на вероватноћама,
        * Случајан избор,
        * Кружан избор.
    - табелу (енгл. *table view*) са две колоне:
        * колону са натписом Одредиште, која садржи назив сваког повезаног чвора,
        * колону са натписом Вероватноћа, у којој се уноси нумеричка вредност која представља вероватноћу рутирања ка датом одредишту.
    - поље за потврду (енгл. *check box*) са натписом Процентуална вероватноћа; када је активно, вредности у колони Вероватноћа се приказују као реалан број у интервалу $[0, 100]$ после којег долази знак процента (%); када није активно, вредности у колони Вероватноћа приказују се као реалан број у интервалу $[0, 1]$.

Када је изабрани елемент мреже *улаз*, модални прозор Својства МОРА променити свој садржај на следећи начин:

- наслovљени панел Ред за чекање се уклања;
- наслovљени панел Сервис се мења у Доласци послова и садржи:
    - падајућу листу са натписом Расподела времена између долазака за избор расподеле, са следећим опцијама:
        * Експоненцијална,
        * Ерлангова,
        * Паретова,
        * Униформна,
        * Детерминистичка,
        * Хиперекспоненцијална.
    - дугме са натписом ... које отвара модални дијалог за унос параметара изабране расподеле.

Када је изабрани елемент мреже *излаз*, модални прозор Својства МОРА садржи само наслovљени панел Опште.

Уколико нема изабраног елемента мреже, ставка менија Својства МОРА бити онемогућена.

REQ-GUI-07: Модални прозор Својства мреже МОРА омогућити кориснику измену релевантних својстава тренутно активне мреже. Прозор МОРА садржати следеће елементе:

- текстуално поље са натписом Назив за унос назива мреже;
- контролу типа енгл. *text area* са натписом Опис за унос описа мреже и помоћним натписом који подстиче корисника да унесе кратак опис мреже;
- бројачку контролу са натписом Укупан број послова за унос укупног броја послова у мрежи; ово поље МОРА бити онемогућено ако је активна мрежа отворена;

- падајућу листу са натписом Референтни опслужни центар за избор једног опслужног центра који ће бити референтан приликом израчунавања мера перформанси затворене мреже; ово поље МОРА бити онемогућено ако је активна мрежа отворена.

REQ-GUI-08: Модални прозор О програму приказује основне информације о апликацији и њеној лиценци. Прозор садржи следеће елементе:

- натпис са називом и верзијом апликације;
- натпис са кратким описом апликације;
- натпис са именом аутора;
- натпис са именом ментора;
- хипервезу (енгл. *hyperlink*) са натписом Званични Git репозиторијум која садржи адресу репозиторијума са изворним кодом апликације; МОРА бити омогућено да се притиском на ову хипервезу отвори веб-страница на поменутој адреси у подразумеваном веб-прегледачу корисничког рачунара;
- контролу типа *text area* са натписом Лиценца која садржи текст лиценце под којом се дистрибуира апликација;
- натпис Лиценце коришћених алата и библиотека који означава почетак одељка са панелом са картицама у којем се за све коришћене алате и библиотеке налази по једна картица са контролом типа *text area* у оквиру које се налази текст лиценце одговарајућег алата или библиотеке.

Ни једна контрола типа *text area* НЕ СМЕ омогућити измену садржаја и МОРА имати могућност померања садржаја (енгл. *scrolling*).

REQ-GUI-09: Модални прозор Параметри симулације МОРА омогућити кориснику унос и измену свих релевантних параметара симулације засноване на дискретним догађајима (детаљније описане у поглављу 4), као и дефинисање иницијалног броја послова у сваком опслужном центру. Прозор МОРА садржати следеће елементе:

- бројачку контролу са натписом Број покретања за унос целобројне вредности која одређује број независних покретања симулације;
- текстуално поље са натписом Максимално трајање за унос реалне вредности која представља највеће дозвољено трајање једног покретања, као и падајућу листу за избор временске јединице;
- текстуално поље са натписом Максимално симулирано време за унос реалне вредности која представља највеће дозвољено симулирано време, као и падајућу листу за избор временске јединице;
- бројачку контролу са натписом Максимални обим узорка за унос максималног броја мерења која ће бити евидентирана током једног покретања, за сваку меру перформанси;

- бројачку контролу са натписом Максимални број догађаја за унос горње границе броја догађаја који могу бити обрађени у оквиру једног покретања;
- падајућу листу са натписом Генератор псеудо-случајних бројева за избор алгоритма за генерисање случајних бројева, уз дугме са иконом која асоцира на „информације", а које отвара веб-страницу са детаљним описом одабраног генератора у подразумеваном веб-прегледачу корисничког рачунара;
- бројачку контролу са натписом Иницијализациона вредност за унос почетне вредности генератора псеудо-случајних бројева, као и дугме са иконом која асоцира на „освежавање", а које омогућава насумично генерисање нове вредности;
- текстуално поље са натписом Проценат одбачених мерења за унос процента мерења који неће бити узети у обзир приликом израчунавања интервалних оцена мера перформанси (видети једначину (4.4));
- текстуално поље са натписом Ниво поверења за унос нивоа поверења;
- натпис Иницијални број послова у опслужним центрима после којег следи табела са следећим колонама:
  - Опслужни центар – текстуална колона која приказује назив сваког опслужног центра у мрежи,
  - Број послова – нумеричка колона у којој корисник уноси иницијалан број послова за одговарајући опслужни центар.

REQ-GUI-10: Модални прозор Интерактивна симулација МОРА омогућити кориснику да ручно управља извршавањем симулације корак по корак. Прозор МОРА садржати следеће елементе:

- контролу типа *progress bar* која приказује проценат комплетираности симулације;
- натпис Протекло време у чијем се наставку налази натпис који приказује тренутно време извршавања у формату <дани>:<часови>:<минуте>:<секунде>.<милисекунде>;
- панел са картицама у којем свака картица представља једно покретање симулације и има натпис облика Симулација <редни број>; унутар сваке картице налази се:
  - натпис Симулирано време у чијем се наставку налази натпис са тренутним временом у симулираном систему у формату <дани> d <часови> h <минуте> min <секунде> s <милисекунде> ms <микросекунде> μs;
  - натпис Догађај у чијем се наставку налази натпис са описом тренутног догађаја који је обрађен у формату <Елемент мреже>: <Тип догађаја> (нпр. Улаз 1: Креирање новог посла).
- дугме са натписом Следећи догађај за обраду наредног догађаја у симулацији;
- дугме са натписом Прескочи које омогућава прескакање више догађаја одједном;

- бројачка контрола која одређује број догађаја који ће бити прескочени након притиска на дугме Прескочи;
- дугме са натписом Заврши симулацију које омогућава да се симулација заврши без интеракције са корисником, односно да се обраде сви преостали догађаји у текућем покретању симулације.

REQ-GUI-11: Модални прозор Симулација МОРА омогућити кориснику минимални увид у ток извршавања симулације. Прозор МОРА садржати следеће елементе:

- контролу типа *progress bar* која приказује проценат комплетираности симулације;
- натпис Протекло време у чијем се наставку налази натпис који приказује тренутно време извршавања у формату <дани>:<часови>:<минуте>:<секунде>.<милисекунде>.

REQ-GUI-12: Прозор Мере перформанси приказује резултате израчунавања мера перформанси за све опслужне центре у мрежи, било да су добијени симулацијом или аналитичким поступком, као и мере перформанси које се односе на систем као целину. Прозор садржи следеће елементе:

- панел са картицама у горњем делу прозора са две картице са следећим натписима:
  - Графикон,
  - Табела.
- У картици са натписом Графикон налази се угнежђени панел који садржи следеће картице: Искоришћеност, Проток, Дужина реда за чекање, Број послова, Време чекања, Време одзива; свака картица садржи стубичасти дијаграм (енгл. *bar chart*) који визуелно приказује вредности одговарајуће мере за сваки опслужни центар:
  - на хоризонталној оси приказани су називи опслужних центара,
  - на вертикалној оси приказане су нумеричке вредности одговарајуће мере перформанси.

  Сваки стуб у стубичастом дијаграму МОРА бити обојен на начин који одговара боји попуњавања опслужног центра на који се односи. Помоћни натпис при задржавању курсора изнад сваког стуба у поменутом стубичастом дијаграму МОРА приказивати вредност одговарајуће мере перформанси у формату <центар> [± <половина дужине интервала поверења>], где је <центар> очекивана вредност мере перформанси за дати опслужни центар, а <половина дужине интервала поверења> је опциона и коришћена само ако су мере перформанси добијене симулацијом.
- У картици са натписом Табела налази се табела са следећим колонама:
  - Назив – текстуална колона која приказује назив сваког опслужног центра у мрежи или садржи реч Мрежа, ако се ред табеле односи на мере перформанси које су дефинисане на нивоу читаве мреже,
  - Искоришћеност,

– Проток,
– Дужина реда за чекање,
– Број послова,
– Време чекања,
– Време одзива.

Редови табеле приказују вредности свих мера перформанси за сваки опслужни центар у мрежи. Последњи ред табеле приказује вредности оних мера перформанси које су дефинисане на нивоу мреже. Вредности мера перформанси у табели МОРАЈУ бити приказане у истом формату као и вредности које се приказују у помоћном натпису стубова у стубичастом дијаграму.

- У доњем делу прозора налази се дугме са натписом Извоз које отвара дијалог за извоз приказаних резултата у текстуалну датотеку у једном од следећих формата:
  – Comma-Separated Values (.csv),
  – Microsoft Excel Workbook (.xlsx),
  – Microsoft Excel 97–2003 Workbook (.xls),
  – JavaScript Object Notation (.json).

REQ-GUI-13: Прозор Шта-ако МОРА омогућити кориснику увид у то како систематична промена неког параметра у моделу утиче на вредности мера перформанси опслужних центара и мреже као целине. У зависности од интервала у коме се параметар мења, реализује се више израчунавања мера перформанси симулационом (ако је прозор отворен притиском на дугме Шта-ако... из групе Симулација на траци са алаткама) или аналитичком методом (ако је прозор отворен притиском на дугме Шта-ако... из групе Аналитичко решавање на траци са алаткама). Резултати сваког израчунавања се бележе и на крају се приказују у виду линијских графикона (енгл. *line chart*) који показују како се мере перформанси мењају у зависности од вредности параметра који се варира за сваки опслужни центар у мрежи, као и за целу мрежу.

Прозор садржи хоризонтално подељени панел (енгл. *split pane*). Леви угнежђени панел садржи следеће елементе:

- групу радио-дугмади која садржи:
  – радио-дугме Чвор у мрежи које у наставку има падајућу листу за избор произвољног *улаза* или *опслужног центра* у мрежи;
  – радио-дугме Мрежа које је могуће изабрати само ако је активна мрежа затворена;
- хоризонталну линију раздвајања (енгл. *horizontal separator*);
- групу контрола за подешавање параметра који ће се мењати, а која садржи:
  – падајућу листу са натписом Променљива за избор параметра који ће бити вариран; садржај ове листе зависи од изабраног радио-дугмета у претходно поменутој групи радио-дугмади и може садржати следеће ставке:

* Расподела времена између долазака послова (ако је изабрано радио-дугме Чвор у мрежи, а из падајуће листе је изабран елемент мреже чији је тип *улаз*);
* Време опслуживања или Број сервера (ако је изабрано радио-дугме Чвор у мрежи, а из падајуће листе је изабран елемент мреже чији је тип *опслужни центар*);
* Укупан број послова (ако је изабрано радио-дугме Мрежа);

– текстуално поље са натписом Почетна вредност за унос почетне вредности изабране променљиве;

– текстуално поље са натписом Крајња вредност за унос завршне вредности изабране променљиве;

– бројачку контролу са натписом Број корака за унос броја тачака у којима ће бити израчунате мере перформанси у оквиру задатог интервала вредности изабране променљиве у случају да променљива није целобројног типа; ако је променљива целобројног типа, ова контрола МОРА бити онемогућена, а број корака се МОРА аутоматски рачунати на основу почетне и крајње вредности променљиве.

Пре завршетка израчунавања мера перформанси, десни угнежђени панел МОРА садржати следећа два елемента:

- дугме са натписом Покрени; притиском на ово дугме корисник започиње израчунавање мера перформанси одговарајућом аналитичком или симулационом методом;
- контролу типа *progress bar* која кориснику приказује напредак извршавања анализе или симулације.

Након завршетка израчунавања, десни угнежђени панел МОРА садржати панел који садржи посебну картицу за сваки опслужни центар у мрежи, као и картицу са натписом Мрежа за мере перформанси које се односе на целу мрежу. Унутар сваке од ових картица налази се угнежђени панел са следећим картицама: Искоришћеност, Проток, Дужина реда за чекање, Број послова, Време чекања и Време одзива. Свака од ових картица садржи линијски графикон који приказује зависност одговарајуће мере перформанси од вредности вариране променљиве. На хоризонталној оси приказане су вредности вариране променљиве, а на вертикалној оси приказане су нумеричке вредности одговарајуће мере перформанси.

REQ-GUI-14: Прозор Анализа мора омогућити кориснику увид у поступак аналитичког решавања модела мреже, а отвара се притиском на дугме Покрени из групе Аналитичко решавање у оквиру траке са алаткама. Прозор садржи следеће елементе:

- контролу типа *progress bar* која приказује проценат комплетираности аналитичке методе;
- натпис Протекло време у чијем се наставку налази натпис који приказује тренутно време извршавања у формату <дани>:<часови>:<минуте>:<секунде>.<милисекунде>;

- хоризонтално подељени панел (енгл. *split pane*) са два угнежђена панела:
  - садржај левог угнежђеног панела зависи од тога која аналитичка метода је примењена за решавање модела:
    * уколико се ради о Џексоновој отвореној мрежи, овај панел садржи све кораке у израчунавању протока кроз опслужне центре мреже, односно примену једначине (3.16);
    * уколико се ради о Гордон-Њуеловој затвореној мрежи, овај панел приказује поступак рачунања просечног броја посета опслужним центрима на основу једначине (3.23) и, испод тога, рачунање вектора **G** коришћењем Бјузеновог алгоритма, односно примену једначине (3.37);
  - десни угнежђени панел садржи:
    * падајућу листу са натписом Опслужни центар за избор опслужног центра у мрежи;
    * панел са хармоником са следећим насловљеним панелима: (1) Рачунање мера перформанси – приказује кораке у израчунавању мера перформанси за изабрани опслужни центар; уколико се ради о Џексоновој отвореној мрежи, МОРАЈУ бити приказане једначине (3.8)-(3.13); уколико се ради о Гордон-Њуеловој затвореној мрежи, МОРАЈУ бити приказане једначине (3.40)-(3.46); (2) Вероватноће стања – приказује вероватноће стања изабраног опслужног центра помоћу стубичастог дијаграма. Све једначине, као и стубичасти дијаграм МОРАЈУ се ажурирати сваки пут када корисник промени изабрани опслужни центар у падајућој листи.

5.2 Функционални захтеви

За све случајеве употребе описане у наставку подразумева се да је актер (енгл. *actor*) случаја употребе корисник апликације (студент или наставник који демонстрира рад у апликацији). Сви случајеви употребе у наставку подразумевају да је апликација успешно инсталирана и покренута на корисничком рачунару.

5.2.1 Основне операције

REQ-FN-01: *Креирање нове мреже.*

Предуслови: Нема.

Активација: Корисник бира Датотека » Нова мрежа (Ctrl + N).

Главни ток:
1. Систем креира модел празне мреже са подразумеваним параметрима.
2. Отвара се нова картица у главном панелу са картицама; картица добија аутоматски генерисан јединствен назив мреже.
3. Новододата картица постаје активна; приказ мреже је празан и спреман за унос елемената.

Постуслови: Нова, празна мрежа је креирана и спремна за даље измене.

REQ-FN-02: *Учитавање мреже из датотеке.*

Предуслови: У фајл систему корисничког рачунара постоји JSON датотека која садржи модел мреже.

Активација: Корисник бира Датотека » Учитај мрежу... (Ctrl + O)

Главни ток:

1. Отвара се уобичајени дијалог за избор датотеке у фајл систему корисничког рачунара са филтером подешеним тако да се приказују само JSON датотеке. Корисник овде може да одустане од ове операције.
2. Уколико је корисник прихватио операцију и изабрао датотеку, систем верификује формат датотеке.
3. Ако је верификација успешна, отвара се нова картица са садржајем учитане мреже и она постаје активна.

Постуслови: Мрежа је успешно учитана и спремна за даље измене. Назив мреже додаје се као ставка подменија Датотека » Недавно учитане мреже; помоћни натпис ове ставке садржи стазу датотеке из које је мрежа учитана.

Алт. токови: Уколико верификација не успе, систем приказује поруку о грешци са описом узрока.

REQ-FN-03: *Учитавање недавно учитане мреже.*

Предуслови: Корисник је већ учитао најмање једну мрежу у текућој сесији апликације.

Активација: Корисник бира мрежу коју жели да учита из подменија Датотека » Недавно учитане мреже.

Главни ток:

1. Систем проверава да ли датотека и даље постоји у фајл систему корисничког рачунара на стази која је приказана у помоћном натпису изабране ставке. Ако датотека постоји, систем верификује формат датотеке.
2. Ако је верификација успешна, отвара се нова картица са садржајем учитане мреже и она постаје активна.

Постуслови: Мрежа је успешно учитана и спремна за даље измене.

Алт. токови: Уколико датотека не постоји или верификација не успе, систем приказује поруку о грешци са описом узрока.

REQ-FN-04: *Снимање мреже у нову датотеку.*

Предуслови: Картица са мрежом коју корисник жели да сними је активна.

Активација: Корисник бира Датотека » Сачувај као... (Ctrl + ⇧ + S).

Главни ток:

1. Отвара се уобичајени дијалог за избор стазе до фајла у који ће се снимити мрежа. Корисник овде може да одустане од ове операције.
2. Уколико је корисник прихватио операцију и изабрао стазу до фајла, систем снима садржај мреже у текстуалном JSON формату.

3. Систем ажурира придружену стазу активне датотеке.

Постуслови: Мрежа је снимљена у нову датотеку; свако будуће снимање иницирано помоћу Датотека » Сачувај користи ту датотеку (видети [REQ-FN--05]).

Алт. токови: Уколико снимање не успе, систем приказује поруку о грешци са описом узрока.

REQ-FN-05: *Снимање већ снимљене мреже.*

Предуслови: Картица са мрежом коју корисник жели да сними је активна и корисник је у текућој сесији апликације ту мрежу најмање једном већ снимио.

Активација: Корисник бира Датотека » Сачувај (Ctrl+S).

Главни ток: Систем снима садржај мреже ажурирајући садржај већ познате (придружене) датотеке, а након тога приказује поруку о успешном снимању.

Постуслови: Датотека придружена мрежи садржи актуелно стање мреже.

Алт. токови: Уколико снимање не успе, систем приказује поруку о грешци са описом узрока.

REQ-FN-06: *Поништавање свих измена мреже.*

Предуслови: Картица са мрежом чије измене корисник жели да поништи је активна, при чему је дата мрежа снимљена најмање једном у текућој сесији апликације.

Активација: Корисник бира Датотека » Поништи све измене.

Главни ток:
1. Систем тражи експлицитну потврду корисника приказивањем модалног дијалога.
2. Након добијања потврде, систем поново учитава последњу снимљену верзију мреже из датотеке у активну картицу.

Постуслови: Све неснимљене измене су одбачене; стање мреже је идентично последњем снимљеном.

REQ-FN-07: *Извоз мреже у облику слике.*

Предуслови: Картица са мрежом коју корисник жели да сачува као слику је активна.

Активација: Корисник бира Датотека » Сачувај слику као... (Ctrl+E).

Главни ток:
1. Систем отвара уобичајени дијалог за избор стазе до фајла у који ће се снимити слика у PNG формату. Корисник овде може да одустане од ове операције.
2. Уколико је корисник прихватио операцију и изабрао стазу до фајла, систем снима садржај мреже у облику слике у PNG формату.

Постуслови: Фајл са сликом постоји на систему и верно репрезентује текући приказ мреже.

REQ-FN-08: *Измена својстава мреже.*

Предуслови: Картица са мрежом чија својства корисник жели да измени је активна.

Активација: Корисник бира Уреди › Својства мреже... (Ctrl+⇧+P).

Главни ток:
1. Систем отвара модални прозор Својства мреже (видети захтев [REQ-GUI-07]).
2. Корисник уноси жељене измене и затвара прозор.

Постуслови: Својства активне мреже су ажурирана.

Алт. токови: Уколико је било које својство мреже измењено на начин који није дозвољен, систем приказује поруку о грешци са описом узрока, одговарајуће својство мреже остаје непромењено, а прозор Својства мреже остаје отворен.

REQ-FN-09: *Преглед информација о апликацији.*

Предуслови: Нема.

Активација: Корисник бира Помоћ › О програму.

Главни ток: Систем приказује модални прозор О програму (видети захтев [REQ--GUI-08]).

Постуслови: Корисник је информисан о верзији апликације коју тренутно користи, њеној лиценци и адреси репозиторијума са изворним кодом и најновијим издањем апликације ради евентуалног преузимања новијег издања.

REQ-FN-10: *Излаз из апликације.*

Предуслови: Нема.

Активација: Корисник бира Датотека › Излаз (Ctrl+Q) или затвара главни прозор.

Главни ток:
1. Систем приказује модални дијалог који тражи потврду корисника да жели да изађе из апликације.
2. Ако корисник потврди, апликација се затвара, док у супротном остаје отворена и са непромењеним стањем.

Постуслови: Апликација је затворена.

5.2.2 Измена мреже

REQ-FN-11: *Додавање нових елемената у оквиру постојеће мреже.*

Предуслови: Активна је картица која садржи мрежу коју корисник жели да измени.

Активација: Корисник активира режим додавања елемента мреже чији је тип *улаз*, *опслужни центар* или *излаз* притиском на одговарајуће дугме на траци са алаткама у оквиру групе Елементи мреже.

Главни ток:

1. Систем приказује *preview* елемента мреже на позицији на којој се налази курсор миша. Елемент визуелно указује да ли је позиција дозвољена (нема преклапања са постојећим елементима) или није дозвољена (постоји преклапање).
2. Ако се изабере опслужни центар, систем омогућава кориснику да промени број сервера унутар тог опслужног центра помоћу точкића миша, пре него што потврди додавање.
3. Корисник једноструким притиском левог тастера миша док се елемент налази на некој дозвољеној позицији унутар панела са мрежом потврђује додавање елемента. Систем генерише јединствен назив елемента у оквиру мреже и поставља елемент на ту позицију. Режим додавања остаје активан ради евентуалног додавања још елемената истог типа. Уколико корисник покуша да дода елемент на недозвољену позицију, систем игнорише ову акцију и наставља да функционише у режиму додавања[1].
4. Корисник може променити тип елемента мреже (улаз, опслужни центар, излаз) који се тренутно додаје притиском на одговарајуће дугме у групи Елементи мреже на траци са алаткама; систем ажурира тип елемента мреже и наставља да функционише у режиму додавања.
5. Корисник излази из режима додавања притиском на већ изабрано дугме на траци са алаткама, отварањем неке ставке главног менија или притиском десног тастера миша унутар панела са мрежом.

Постуслови: Мрежа је измењена у складу са радњама које је корисник предузео и сви новододати елементи су видљиви и имају одговарајући аутоматски генерисан назив.

REQ-FN-12: *Повезивање постојећих елемената мреже (додавање путање).*

Предуслови: Активна је картица која садржи мрежу коју корисник жели да измени.

Главни ток:

1. Корисник активира режим додавања путање притиском на дугме Путања у оквиру групе Елементи мреже на траци са алаткама.
2. Све постојеће путање у мрежи и сви елементи мреже који не могу бити изворишни елемент путање треба да промене свој изглед како би се визуелно истакло да је апликација у режиму додавања путање и да у обзир за повезивање долазе само они елементи који нису променили свој изглед.

[1]На овај начин систем не „кажњава” корисника када се деси случајан притисак левог тастера миша.

3. Корисник бира изворишни елемент путање; ако је избор валидан, систем га визуелно истиче; сви елементи који не могу бити одредиште мењају изглед на исти начин као у кораку 2.
4. Корисник бира одредишни елемент; ако је избор валидан, систем додаје путању између изворишта и одредишта. Режим остаје активан ради додавања још путања.
5. Корисник излази из режима додавања путање притиском на дугме Путања, отварањем неке ставке главног менија или притиском десног тастера миша унутар панела са мрежом.

Постуслови: Сви изабрани парови елемената мреже су повезани новим путањама.

Алт. токови: Ако у кораку 2 корисник изабере недозвољено изворište, или у кораку 3 недозвољено одредиште, систем прекида режим додавања путање.

REQ-FN-13: *Промена позиције елемента мреже.*

Предуслови: Нема.

Активација:
1. Корисник покреће операцију премештања елемента мреже (*улаза*, *опслужног центра* или *излаза*) притиском левог тастера миша док се курсор налази изнад изабраног елемента мреже и превлачи елемент до нове позиције унутар панела са мрежом. Током превлачења, систем визуелно указује да ли је нова позиција дозвољена (нема преклапања са постојећим елементима) или није дозвољена (постоји преклапање). Систем ажурира све путање којима је елемент мреже повезан са остатком мреже у реалном времену током превлачења, под условом да је нова позиција дозвољена.
2. Корисник завршава операцију премештања елемента мреже пуштањем левог тастера миша. Ако је позиција на којој се елемент налази након пуштања дозвољена, систем ажурира позицију елемента и све повезане путање; у супротном, елемент се враћа на последњу дозвољену позицију током превлачења.

Главни ток: Систем ажурира позицију изабраног елемента и повезане путање ако оне постоје.

Постуслови: Изабрани елемент мреже је премештен на нову позицију.

REQ-FN-14: *Ротирање елемента мреже.*

Предуслови: Нема.

Активација: Корисник (1) бира Уреди › Изабрани елемент мреже › Ротирај (Ctrl + R) уколико је изабрани елемент мреже онај који он жели да ротира и притом је његов тип *улаз*, *опслужни центар* или *излаз*, или (2) отвара контекстни мени (енгл. *context menu*) притиском десног тастера миша док се курсор налази изнад одговарајућег елемента мреже и бира ставку Ротирај (Ctrl + R).

Главни ток: Систем ротира изабрани елемент мреже за угао од 180°, уз ажурирање повезаних путања ако оне постоје.

Постуслови: Изабрани елемент мреже је ротиран за 180°.

REQ-FN-15: *Промена типа путање.*

Предуслови: Нема.

Активација: Корисник (1) бира Уреди » Изабрани елемент мреже » Тип уколико је путања чији тип он жели да промени тренутно изабрани елемент мреже или (2) отвара контекстни мени притиском десног тастера миша док се курсор налази изнад одговарајуће путање у мрежи и бира ставку Тип.

Главни ток:
1. Корисник бира једну од ставки Права линија, Безјеова крива или Више сегмената (МОРАЈУ бити омогућени само они типови путања који су дозвољени ограничењем наведеним у [REQ-GUI-05]).
2. Систем ажурира приказ путање у складу са изабраним типом.

Постуслови: Путања изгледа на нови начин, у складу са изабраним типом, при чему крајње тачке остају непромењене.

REQ-FN-16: *Измена својстава елемента мреже.*

Предуслови: Нема.

Активација: Корисник (1) бира Уреди » Изабрани елемент мреже » Својства... (Ctrl+P) уколико је елемент мреже чија својства он жели да измени тренутно изабран или (2) отвара контекстни мени притиском десног тастера миша док се курсор налази изнад одговарајућег елемента мреже и бира ставку Својства (Ctrl+P).

Главни ток:
1. Систем отвара модални прозор Својства чији садржај зависи од типа елемента мреже (видети захтев [REQ-GUI-06]).
2. Корисник мења жељена својства и затвара прозор.
3. Систем верификује унете вредности и, уколико су оне валидне, ажурира својства елемента мреже.

Постуслови: Својства елемента су ажурирана; приказ елемента мреже је ажуриран у складу са реализованим променама.

Алт. токови: Уколико је било које својство елемента измењено на начин који није дозвољен, систем приказује поруку о грешци са описом узрока, одговарајуће својство елемента мреже остаје непромењено и прозор Својства остаје отворен.

REQ-FN-17: *Брисање елемента мреже.*

Предуслови: Нема.

Активација: Корисник (1) бира Уреди » Изабрани елемент мреже » Уклони (⌫) уколико је елемент мреже који он жели да обрише тренутно изабран или

(2) отвара контекстни мени притиском десног тастера миша док се курсор налази изнад одговарајућег елемента мреже и бира ставку Уклони (⌫).

Главни ток:
1. Систем тражи потврду брисања одговарајућим дијалогом.
2. Ако добије потврду, систем уклања елемент мреже; додатно, ако је уклоњени елемент мреже типа *улаз*, *опслужни центар* или *излаз*, све путање којима је он повезан са остатком мреже такође се уклањају.

Постуслови: Елемент мреже (и повезане путање, ако постоје) су уклоњени из мреже.

5.2.3 Симулација

REQ-FN-18: *Измена параметара симулације.*

Предуслови: Активна је картица са мрежом над којом корисник жели да покрене симулацију и мрежа је валидна.

Активација: Корисник притисне дугме Параметри... на траци са алаткама.

Главни ток:
1. Систем отвара модални прозор Параметри симулације (видети захтев [REQ-GUI-09]).
2. Корисник подешава вредности и затвара прозор.
3. Систем верификује унете вредности и, уколико су оне валидне, ажурира параметре симулације.

Постуслови: Параметри симулације и иницијалне поставке чворова су ажурирани.

Алт. токови: Уколико је било који параметар симулације измењен на начин који није дозвољен, систем приказује поруку о грешци са описом узрока, одговарајући параметар остаје непромењен и прозор Параметри симулације остаје отворен.

REQ-FN-19: *Покретање интерактивне симулације.*

Предуслови: Активна је картица са мрежом над којом корисник жели да покрене симулацију и мрежа је валидна; параметри симулације су дефинисани у складу са жељама корисника.

Активација: Корисник у падајућој листи дугмета за покретање симулације бира Интерактивна симулација.

Главни ток:
1. Систем отвара модални прозор Интерактивна симулација (видети захтев [REQ-GUI-10]) и омогућава извршавање корак по корак, прескакање више догађаја и завршетак симулације без даље интеракције са симулацијом.
2. У сваком тренутку током симулације модел мреже у главном панелу са картицама приказује тренутно стање мреже, односно број послова у сваком реду за чекање и статус сваког сервера (слободан/заузет).

3. Сваки прелазак посла са једног елемента мреже на други (укључујући и улазак у систем и излазак из система) је представљен кориснику коришћењем анимације.
4. Након комплетирања аутоматске симулације, графичка репрезентација сваког опслужног центра мора да укључи следеће:
   - *искоришћеност* сваког сервера помоћу дводимензионалног кружног дијаграма (енгл. *pie chart*) која се налази унутар круга који представља сервер;
   - *релативна дужина реда за чекање* сваког опслужног центра (нормализација се врши у односу на збир дужина редова за чекање свих опслужних центара у мрежи) којом корисник стиче осећај о нивоу загушења у појединим деловима мреже; релативна дужина реда за чекање представља се попуњеним правоугаоником унутар правоугаоника који представља ред за чекање опслужног центра, при чему је однос ширине два правоугаоника једнак релативној дужини реда за чекање.

Постуслови: Симулација је успешно извршена; генерисани су узорци потребни за оцену мера перформанси система; корисник је стекао увид у динамику система током симулације.

Алт. токови: Корисник може у било ком тренутку да затвори прозор Интерактивна симулација и настави рад у апликацији, при чему се узорци прикупљени до тог тренутка одбацују; не врши се визуелизација описана у кораку 4.

REQ-FN-20: *Покретање аутоматске симулације.*

Предуслови: Активна је картица са мрежом над којом корисник жели да покрене симулацију и мрежа је валидна; параметри симулације су дефинисани у складу са жељама корисника.

Активација: Корисник у падајућој листи дугмета за покретање бира Аутоматска симулација.

Главни ток:
1. Систем извршава симулацију без било какве интеракције са корисником; напредак симулације и протекло време приказани су у оквиру модалног прозора Симулација у складу са захтевом [REQ-GUI-11].
2. По завршетку симулације реализује се визуелизација описана у кораку 4 из случаја употребе [REQ-FN-19].

Постуслови: Симулација је завршена; генерисани су узорци потребни за оцену мера перформанси система.

Алт. токови: Корисник може у било ком тренутку да затвори прозор Симулација и настави рад у апликацији, при чему се узорци прикупљени до тог тренутка одбацују; не врши се визуелизација поменута у кораку 2.

REQ-FN-21: *Извоз узорака генерисаних током симулације.*

Предуслови: Комплетирана је интерактивна или аутоматска симулација.

Активација: Корисник притисне дугме Извези узорке из групе Симулација на траци са алаткама.

Главни ток:

1. Систем отвара уобичајени дијалог за избор стазе до директоријума у који ће се снимити узорци. Корисник овде може да одустане од ове операције.
2. Уколико је корисник прихватио операцију и изабрао стазу до директоријума, систем снима узорке у Microsoft Excel Workbook (`.xlsx`) формату на следећи начин: креира се онолико директоријума колико је покретања симулације корисник подесио у оквиру прозора Параметри симулације, при чему сваки директоријум носи назив `Симулација X` где је `X` редни број покретања симулације (почевши од 1); унутар сваког од тих директоријума креира се по једна `.xlsx` датотека за сваки опслужни центар мреже (при чему датотека носи назив `Y.xlsx` где је `Y` назив опслужног центра у мрежи) и, додатно, једна `.xlsx` датотека за мрежу као целину (при чему датотека носи назив `Z.xlsx` где је `Z` назив мреже); свака од тих датотека садржи радне листове (енгл. *sheet*) са називима одговарајућих мера перформанси, при чему сваки радни лист садржи по једну табелу са следећим колонама: `Време`, `Вредност`, `Тежина` у складу са форматом (4.1).

Постуслови: Датотеке са узорцима су снимљене на изабрану локацију.

Алт. токови: Уколико снимање не успе, систем приказује поруку о грешци са описом узрока.

5.2.4 Аналитичко решавање

REQ-FN-22: *Покретање аналитичког решавања.*

Предуслови: Активна је картица са мрежом коју корисник жели да реши аналитички и мрежа је валидна.

Активација: Корисник притисне дугме Покрени из групе Аналитичко решавање на траци са алаткама.

Главни ток:

1. Систем аутоматски препознаје коју аналитичку методу треба да примени и отвара модални прозор Анализа са корацима и формулама (видети захтев [REQ-GUI-14]).
2. По завршетку аналитичке методе, реализује се визуелизација описана у кораку 4 из случаја употребе [REQ-FN-19].

Постуслови: Израчунате мере перформанси су доступне за приказ и даљу употребу.

Алт. токови: Уколико корисник затвори прозор Анализа пре завршетка аналитичког решавања, резултати се не чувају и не приказују, а корисник наставља рад у апликацији. Не врши се визуелизација поменута у кораку 2.

5.2.5 Мере перформанси

REQ-FN-23: *Преглед мера перформанси.*

Предуслови: Активна је картица са мрежом чије мере перформанси корисник жели да погледа, а претходно је покренута симулација или аналитичко решавање модела и сачекан је завршетак.

Активација: Корисник притисне дугме Мере перформанси... из групе Симулација или Аналитичко решавање на траци са алаткама.

Главни ток: Систем приказује прозор Мере перформанси са графичким и табеларним приказима (видети захтев [REQ-GUI-12]).

Постуслови: Корисник има увид у све тражене мере перформанси појединачних опслужних центара у мрежи и мреже као целине.

REQ-FN-24: *Извоз мера перформанси.*

Предуслови: Отворен је прозор Мере перформанси са попуњеним резултатима.

Активација: Корисник притисне дугме Извоз.

Главни ток: Систем отвара уобичајени дијалог за избор стазе до фајла у који ће бити извезени резултати и избор формата (видети захтев [REQ-GUI--12]).

Постуслови: Резултати су снимљени у изабрани фајл у жељеном формату.

5.2.6 „Шта-ако" анализа

REQ-FN-25: *Покретање „Шта-ако" анализе.*

Предуслови: Активна је картица са мрежом над којом корисник жели да покрене „Шта-ако" анализу и мрежа је валидна.

Активација: Корисник притисне дугме Шта-ако... из групе Симулација или Аналитичко решавање на траци са алаткама.

Главни ток:

1. Систем приказује прозор Шта-ако (видети захтев [REQ-GUI-13]) са избором опслужног центра, улаза или мреже, затим променљиве, интервала и броја корака.
2. Корисник подешава параметре анализе и потврђује свој избор притиском на дугме Покрени.
3. Систем покреће израчунавање мера перформанси одговарајућом методом и, по завршетку, приказује линијске дијаграме описане у захтеву [REQ-GUI-13].

Постуслови: Корисник стиче увид у то како промена изабране променљиве утиче на мере перформанси свих елемената мреже и мреже као целине.

5.3 Нефункционални захтеви

REQ-NFN-01: Сви артефакти који су потребни за покретање апликације (извршно окружење, извршне датотеке, библиотеке, сви ресурси у облику текстуалних и бинарних

датотека итд.) МОРАЈУ се налазити у оквиру јединственог фајла чијим покретањем корисник покреће или чаробњака (енгл. *wizard*) за инсталацију апликације на оперативном систему корисника у неколико једноставних корака или саму апликацију. Од корисника се НЕ СМЕ захтевати инсталација било каквог додатног софтвера.

REQ-NFN-02: Апликација МОРА бити извршива у складу са [REQ-NFN-01] на најмање два оперативна система: Windows и Linux (у обзир долазе само дистрибуције које имају графичко корисничко окружење). Апликација МОЖЕ да подржи и macOS. Скуп функционалности апликације МОРА бити идентичан на свим подржаним платформама, без потребе за изменама у изворном коду. Графички кориснички интерфејс апликације МОРА бити конзистентан на свим платформама.

REQ-NFN-03: Текст који се приказује на графичком корисничком интерфејсу апликације МОРА да подржи најмање следећа два писма: ћирилично писмо српског језика и писмо енглеског језика. Промена језика МОРА бити омогућена кроз графички кориснички интерфејс апликације при чему се НЕ СМЕ захтевати од корисника да изађе из апликације да би промена језика била видљива, већ се промена свих текстуалних елемената које корисник види МОРА десити у току извршавања апликације и то непосредно након што корисник ту промену иницира.

REQ-NFN-04: Покретање симулације или аналитичког решавања мреже НЕ СМЕ да резултује „замрзавањем" или неадекватним радом графичког корисничког интерфејса апликације. Аутоматска симулација увек има два или више покретања која МОРАЈУ бити независни токови контроле различити од главне нити апликације. Интерактивна симулација увек има два или више покретања која се МОРАЈУ извршавати секвенцијално због природе интеракције са корисником. Аналитичко решавање мреже увек представља независан ток контроле различит од главне нити апликације. Уколико корисник покрене „Шта-ако" анализу, сва израчунавања МОРАЈУ бити независни токови контроле; једно израчунавање у овом контексту представља два или више покретања аутоматске симулације или аналитичку методу за решавање мреже.

REQ-NFN-05: Апликација МОРА омогућити кориснику да конфигурише следеће аспекте њеног рада и изгледа:

- подешавања прозора: иконица апликације, иницијално стање (максимизован или не), величина прозора (у случају да прозор није максимизован) и активну тему графичког корисничког интерфејса (при чему треба подржати најмање једну светлију и једну тамнију тему);
- језик корисничког интерфејса (при чему треба подржати најмање српски и енглески језик);
- иницијални скуп мрежа које ће бити учитане при покретању апликације;
- параметри за нумеричке прорачуне: број децималних цифара, груписање цифара и начин заокруживања;
- параметри за генератор случајних бројева, укључујући тип генератора и иницијализациону вредност.

Све наведене ставке МОРАЈУ бити дефинисане у конфигурационој датотеци и морају се аутоматски применити при поновном покретању апликације.

Поглавље 6

ПРОЈЕКТОВАЊЕ И ИМПЛЕМЕНТАЦИЈА АПЛИКАЦИЈЕ

Имајући у виду да оно што чини софтверски алат који је реализован у оквиру овог рада вредним у педагошком смислу јесу, пре свега, његове *презентационе могућности* и *интерактивност*, именован је *Visual and Interactive Performance Evaluation Tool* или, скраћено, VIPET, што ће се користити у наставку.

6.1 Пројектовање и коришћени пројектни обрасци

Приликом пројектовања апликације VIPET примењен је скуп принципа објектно оријентисаног пројектовања познат под акронимом SOLID [11]:

- S — *Single Responsibility Principle* (принцип једне одговорности) — увек треба да постоји само један разлог за измену неке класе, односно класа треба да има само једну јасно дефинисану одговорност у систему;
- O — *Open/Closed Principle* (принцип отворености за наслеђивање и затворености за измене) — класе треба да буду отворене за наслеђивање али затворене за модификације; другим речима, приликом додавања нових функционалности не сме се захтевати модификација постојећих класа, већ само додавање нових класа које наслеђују постојеће класе или имплементирају одговарајуће интерфејсе; овај принцип подразумева интензивно коришћење полиморфизма;
- L — *Liskov Substitution Principle* (принцип замене Барбаре Лисков) — објекти изведених класа морају се понашати као објекти основних класа, без нарушавања исправности програма ако се искористе као замена за објекте основних класа;
- I — *Interface Segregation Principle* (принцип раздвајања интерфејса) — боље је имати више специфичних интерфејса него један општи интерфејс;
- D — *Dependency Inversion Principle* (принцип инверзије зависности) — класе вишег нивоа не треба да зависе од класа нижег нивоа, већ од одговарајућих апстракција (интерфејса); на овај начин се смањује повезаност модула и омогућава замена једне имплементације другом без утицаја на остале делове система.

Одлука да се за реализацију алата VIPET користи програмски језик Java и радни оквир (енгл. *framework*) JavaFX за креирање графичког корисничког интерфејса утицала је на процес пројектовања апликације. Основу апликације VIPET чини пројектни образац *Model-View-Controller* (MVC) који омогућава одвајање логике апликације од корисничког интерфејса, чиме се постиже добра организација кода, лако одржавање и могућност поновног коришћења компоненти. У овом обрасцу:

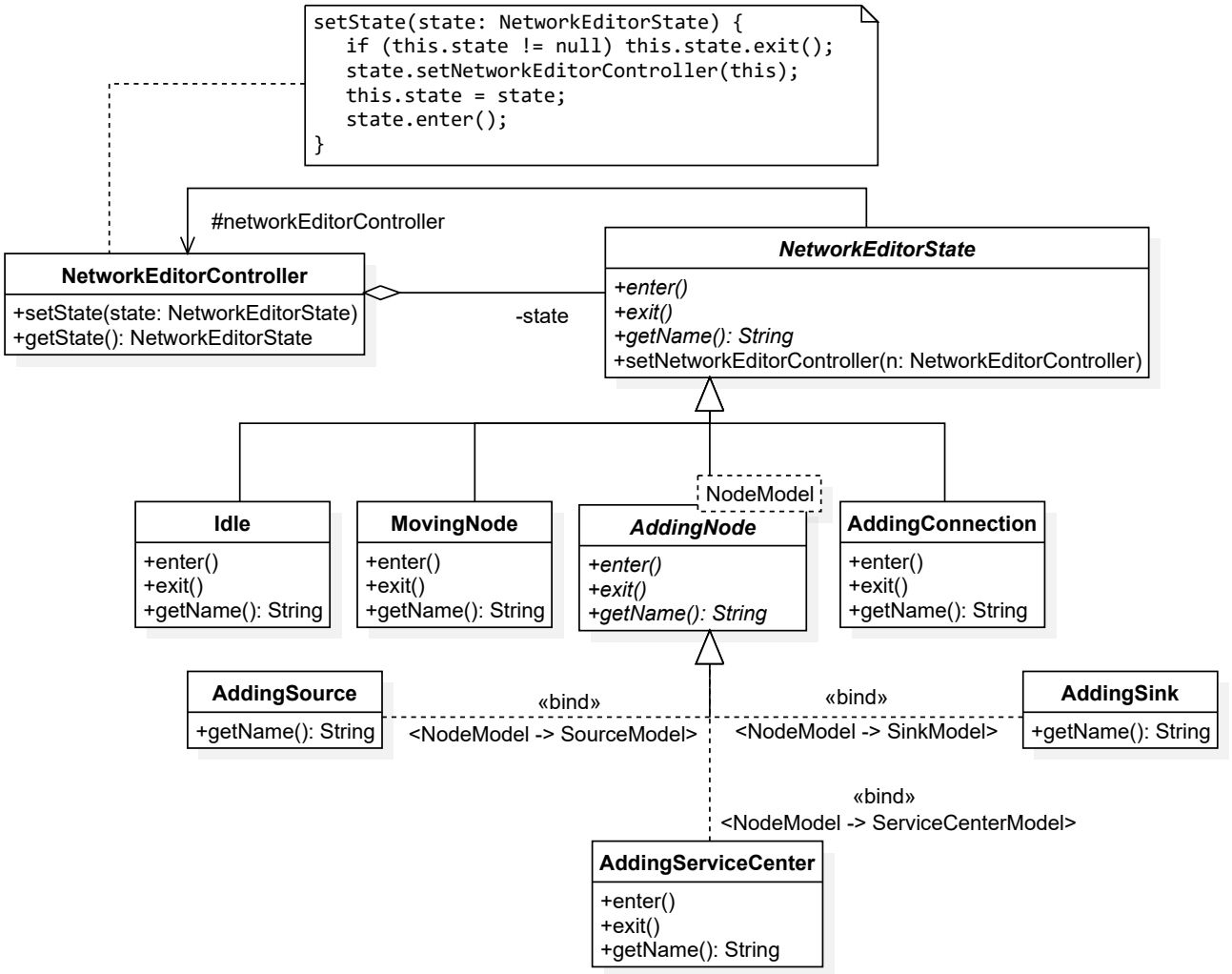
- *Model* (модел) обухвата класе које представљају мреже са опслужним центрима и све њихове елементе, као и класе које енкапсулирају све податке који су потребни за покретање симулација и аналитичких метода; овде треба нагласити две мање модификације у односу на то шта представља модел у класичном смислу: (1) одлучено је да се имплементација алгоритама за симулацију и аналитичко решавање мрежа издвоји у облику *сервиса*, посебних класа које нису модели; (2) дозвољено је да модел има својства (енгл. *properties*), а не само поља „обичних" типова, чиме се омогућава директно везивање (енгл. *binding*) са својствима графичких компоненти у приказу, што олакшава и убрзава ажурирање приказа на основу промена у моделу и обрнуто;

- *View* (приказ) је одговоран за приказивање података кориснику и интеракцију са њим; сваки прозор и свака компонента која се користи на више места у графичком корисничком интерфејсу имају декларативну спецификацију у облику текстуалне датотеке у формату FXML, који представља адаптацију формата XML (енгл. *eXtensible Markup Language*); приказ дефинише улазне и излазне контроле (натписи, дугмад, текстуална поља итд.), њихов распоред, положај и изглед (уз напомену да се, када идентичну стилизацију треба применити на више места, користе CSS (енгл. *Cascading Style Sheets*) датотеке које се могу реферисати из FXML датотека);

- *Controller* (контролер) делује као посредник између модела и приказа; у радном оквиру JavaFX контролер се реализује као класа придружена FXML датотеци, која садржи поља и методе означене анотацијом `@FXML` (поља су референце на одговарајуће контроле дефинисане у приказу, односно FXML датотеци, док методе служе за обраду догађаја (енгл. *event handlers*)); улога контролера обухвата: (1) успостављање једносмерног (енгл. *unidirectional*) или двосмерног (енгл. *bidirectional*) везивања између својстава модела и својстава одговарајућих графичких компоненти приказа у иницијализационој методи контролера (`initialize`); (2) динамичка измена приказа на основу модела када то није могуће учинити једноставним везивањем својстава, при чему контролер преузима потребне податке из модела и на основу њих ажурира приказ; (3) реакција на догађаје које генеришу корисничке интеракције са приказом; (4) позивање метода одговарајућих сервиса.

Како се у захтеву [REQ-FN-11] помињу *режими додавања различитих елемената мреже*, природно се намеће коришћење пројектног обрасца *Стање* (енгл. *State*). На слици 6.1 приказан је стандардни UML (енгл. *Unified Modeling Language*) дијаграм класа овог пројектног обрасца који је прилагођен потребама апликације.

У наставку су наведени још неки обрасци коришћени у фази пројектовања апликације:



Слика 6.1: *Пројектни образац Стање у контексту апликације.*

- већи број класа које имају суфикс `Util` у свом имену (на пример, `UIUtil`) представљају *Фасаду* (енгл. *Facade*) која пружа поједностављен интерфејс сложенијим подсистемима апликације; на овај начин се смањује повезаност између модула и поједностављује коришћење сложенијих функционалности (на пример, у случају већ поменуте класе `UIUtil`, те функционалности би биле отварање нових прозора, креирање дијалога итд.);

- *Шаблонска метода* (енгл. *Template Method*) `takeMeasurement` дефинисана је у апстрактној класи `SampleGenerator` на следећи начин:

```
public void takeMeasurement(final double time) {
  currentTime = time;

  if (sample.size() < maxSampleSize && shouldTakeMeasurement()) {
    sample.add(new Measurement(currentTime, getValue(), getWeight()));
  }
}
```

Конкретне поткласе (`UtilizationServiceCenterSampleGenerator`, `NumberOfJobsServiceCenterSampleGenerator` итд.) имплементирају апстрактне методе `shouldTakeMeasurement`, `getValue` и `getWeight` које се позивају у оквиру ове шаблонске методе; на овај начин је обезбеђена јасна структура алгоритма за евидентирање мерења током симулације, док се поједини кораци делегирају конкретним потклasама;

- за потребе локализације апликације користи се класа `LocalizationManager` која представља *Уникат* (енгл. *Singleton*), јер је потребно да постоји само једна инстанца ове класе која ће бити доступна из свих делова апликације;
- за стратегије рутирања послова у мрежи приликом извршавања симулације, односно избора опслужног центра или излаза којем се прослеђује посао након што се заврши његова обрада на серверу тренутног опслужног центра, искоришћен је пројектни образац *Стратегија* (енгл. *Strategy*) који ефикасно решава овакав тип проблема.

6.2 Програмски код и његова организација

Као што је већ наведено, за реализацију алата VIPET коришћен је програмски језик Java у верзији 21 која је, у време писања овог рада, претпоследња LTS (енгл. *Long-Term Support*) верзија овог језика. Java је одабрана из следећих разлога:

- *платформска независност* — изворни код у програмском језику Java се преводи у Java *бајткод*, који се извршава на Java виртуелној машини (енгл. *Java Virtual Machine*, JVM); JVM је доступна за велики број платформи, што омогућава да се Java апликације покрећу на различитим оперативним системима без потребе за модификацијама у изворном коду;
- *велики број доступних и поузданих библиотека и алата* — Java има велику стандардну библиотеку, која покрива широк спектар функционалности, као и велики број додатних библиотека, које значајно олакшавају и убрзавају развој комплексних апликација; такође, постоји и значајан број алата који олакшавају и убрзавају развој, тестирање и одржавање апликација написаних у овом програмском језику;
- *популарност и подршка заједнице* — Java је један од најпознатијих програмских језика на свету, што значи да постоји велика заједница програмера који могу допринети развоју пројеката у којима се користи Java, постоји обиље квалитетне и широко доступне документације и ресурса за учење.

Програмски језик Java је настао 1995. године у америчкој компанији Sun Microsystems. Године 2006. ова компанија је објавила прву референтну имплементацију стандардне едиције овог програмског језика (енгл. *Java Standard Edition*, Java SE) као пројекат отвореног кода[1] под називом OpenJDK[2]. Компанија Sun Microsystems је 2010. године

[1]Пројекат је лиценциран коришћењем GNU GPL лиценце у верзији 2.0 са специјалним додатком (тзв. *Classpath Exception*) који дозвољава да се библиотеке из OpenJDK могу повезивати са другим модулима, укључујући и модуле затвореног кода, без наметања GPL лиценце за резултујући софтверски производ. Управо је ово омогућило комерцијалну употребу OpenJDK.

[2]Званични репозиторијум пројекта OpenJDK се може наћи на следећој адреси: `https://github.com/openjdk/jdk`.

постала део компаније Oracle Corporation која је 2019. године променила услове коришћења своје дистрибуције Java SE, што је изазвало велику забринутост у заједници програмера и довело до појаве више алтернативних дистрибуција Java SE које су базиране на OpenJDK, са мањим или већим изменама самог изворног кода. За развој апликације у овом раду коришћена је дистрибуција Eclipse Temurin[3], која је једна од најпознатијих и најактивније одржаваних дистрибуција Java SE заснованих на OpenJDK. Развојем руководи европска организација Eclipse Adoptium Working Group.

Као што и име сугерише, OpenJDK представља оно што се у контексту Java програмирања назива *Java Development Kit* (JDK) или *Java Software Development Kit* (Java SDK), те укључује све што је потребно за развој и извршавање Java програма:

- преводилац (`javac`), дебагер (`jdb`), алат за креирање архива (`jar`), алат за креирање документације (`javadoc`) и друге алате;
- *Java Runtime Environment* (JRE) — окружење за извршавање Java програма, које укључује JVM, стандардне библиотеке и друге компоненте потребне за покретање Java програма.

Због жеље да се VIPET учини што приступачнијим за даљи развој, измене и унапређења, донете су следеће одлуке:

1. сви идентификатори у изворном коду су без изузетка на енглеском језику; тежило се писању што читљивијег и разумљивијег кода, уз минимално коришћење коментара, пратећи познату максиму да је *добар код сам себи најбоља документација* [12]; коментари, такође на енглеском језику, коришћени су само у случајевима када се сматрало да намера која стоји иза одређеног дела кода није очигледна;
2. апликација ће бити јавно доступна као софтвер отвореног кода (енгл. *open source*) на адреси `https://github.com/danijelaskov/vipet` у оквиру платформе GitHub која је, у време писања овог рада, водећа платформа за јавни кооперативни развој и управљање софтверским пројектима; биће лиценцирана под GNU GPL лиценцом у верзији 3.0 која обезбеђује заштиту ауторства гарантујући притом да сва евентуална будућа побољшања и измене остану доступна заједници; ово је у складу са академским принципима слободног приступа знању, транспарентности, сарадње и могућности провере и репродукције резултата;
3. одлука у 2 утицала је на то да се као алат за управљање верзијама изворног кода (енгл. *Version Control System*, VCS) користи Git; овај избор је оправдан и због чињенице да је Git постао *de facto* стандард у индустрији у овој категорији алата због своје флексибилности, ефикасности и филозофије дистрибуираног развоја.

Како је за иоле озбиљнији развој софтвера потребан и одговарајући алат за управљање зависностима и аутоматизацију процеса превођења, тестирања и паковања апликације, донета је одлука да се користи систем Gradle[4]. Сам алат не мора бити инсталиран

[3]Званични веб-сајт дистрибуције Temurin налази се на следећој адреси: `https://adoptium.net/temurin/releases/?version=21`. Иначе, реч *temurin* је анаграм речи *runtime*.

[4]Званични веб-сајт алата Gradle налази се на следећој адреси: `https://gradle.org/`.

на рачунару на коме се развија апликација, јер је довољно покренути омотачку скрипту (енгл. *wrapper script*; на оперативном систему Windows то је `gradlew.bat`, док је на оперативним системима Linux и macOS то `gradlew`) која ће аутоматски преузети и инсталирати одговарајућу верзију алата.

Сви задаци (енгл. *tasks*) који се тичу поменутих аспеката развоја апликације дефинисани су у скрипти `build.gradle.kts` у кореном директоријуму пројекта. Ова скрипта садржи програмски код у коме се користи доменски специфичан језик (енгл. *Domain Specific Language*, DSL) који поштује синтаксна правила програмског језика Kotlin[5]. Kotlin је једноставан и читљив до те мере да је чак и људима који га не познају формално веома лако да испрате шта постиже поменута скрипта; истовремено довољно је моћан да омогући флексибилну конфигурацију целокупне логике превођења, тестирања и паковања апликације.

Gradle подразумевано користи директоријумску структуру која је постала практично стандардна када је реч о развоју Java пројеката и лако је препознатљива сваком искусном програмеру. Ова хијерархија уведена је још у алату Apache Maven[6], који је претходио алату Gradle, а корисна је због тога што се њеном употребом избегавају *ad hoc* решења која углавном негативно утичу на брзину којом програмери који први пут виде пројекат могу да се укључе у његов развој. Ова структура подразумева постојање директоријума `src` који садржи све датотеке које чине апликацију, при чему:

- поддиректоријум `main` садржи све датотеке које су потребне за *извршавање* апликације; овај поддиректоријум садржи два поддиректоријума:
  - `java` садржи сав изворни код апликације у облику текстуалних датотека са екстензијом `.java` распоређених у директоријуме који одговарају пакетима којима припадају одговарајуће класе;
  - `resources` садржи све артефакте који нису програмски код, а потребни су за рад апликације (иконицу апликације, FXML датотеке које дефинишу графички кориснички интерфејс, CSS датотеке које дефинишу стил графичког корисничког интерфејса, `.properties` датотеке које се користе за локализацију апликације (`messages.properties` садржи све стрингове који се приказују на графичком корисничком интерфејсу на енглеском језику, док `messages_sr.properties` садржи њихове преводе на српски језик; видети [REQ-NFN-03]), подешавање логовања у апликацији (`log4j2.properties`) и подешавања која се односе на саму апликацију (`application.properties`; видети [REQ-NFN-05]));
- поддиректоријум `test` садржи све артефакте који су потребни за јединично тестирање (енгл. *unit testing*) апликације; слично поддиректоријуму `main` и овај поддиректоријум садржи два поддиректоријума:
  - `java` садржи сав изворни код јединичних тестова;
  - `resources` садржи све артефакте који нису програмски код, а потребни су за извршавање јединичних тестова.

[5]Овакви доменски специфични језици који користе синтаксу других програмских језика се називају уграђеним (енгл. *embedded*) доменски специфичним језицима.

[6]Званични веб-сајт алата Apache Maven налази се на следећој адреси: `https://maven.apache.org/`.

Поред наведених директоријума, корени директоријум пројекта садржи следеће:

- директоријум `build`, који се аутоматски креира током процеса превођења и садржи све артефакте који настају током тог процеса, као што су преведене класе (`.class` датотеке), JAR датотеке и извештаје о тестирању; овај директоријум се не прати у систему за управљање верзијама;
- директоријум `logs`, који се аутоматски креира током извршавања апликације и садржи лог фајлове које апликација генерише током извршавања; овај директоријум се такође не прати у систему за управљање верзијама;
- директоријум `licenses`, који садржи текстове лиценци свих библиотека које се користе у оквиру апликације;
- датотеку `LICENSE`, која садржи текст лиценце под којом је апликација објављена;
- директоријум `gradle` који садржи:
  - поддиректоријум `wrapper`, који садржи JAR датотеку и пратећи конфигурациони фајл који се користе за аутоматско преузимање и инсталацију алата Gradle када се покрене већ поменута омотачка скрипта;
  - датотеку `libs.versions.toml`, која представља централни текстуални конфигурациони фајл за дефиницију зависности коришћењем TOML (енгл. *Tom's Obvious, Minimal Language*) синтаксе са три секције: (1) `versions` у којој се дефинишу верзије свих зависности, (2) `libraries` у којој се дефинишу све библиотеке које апликација користи, и (3) `plugins` у којој се дефинишу сви додаци (енгл. *plugins*) које алат Gradle користи; све библиотеке се дохватају из јавног репозиторијума Maven Central[7], који је, у време писања овог рада, највећи, најпознатији и најкоришћенији репозиторијум Java библиотека; сви додаци се дохватају из јавног репозиторијума Gradle Plugin Portal[8].

Професионална израда софтвера подразумева коришћење алата за аутоматско форматирање изворног кода како би се обезбедила доследност у стилу писања кода, што олакшава његово читање и одржавање. За аутоматско форматирање изворног кода апликације VIPET коришћен је додатак за алат Gradle који се назива Spotless, а подразумевани стил писања кода који се користи је Google Java Style Guide[9].

За писање изворног кода апликације коришћено је интегрисано развојно окружење (енгл. *Integrated Development Environment*, IDE) IntelliJ IDEA[10], које је једно од најпознатијих и најнапреднијих интегрисаних развојних окружења за програмски језик Java. Ово окружење пружа бројне функционалности које олакшавају и убрзавају развој софтвера, али сам код апликације није ни на који начин везан за ово окружење (нпр. коришћењем неких специфичних формата датотека или функционалности које су доступне само у

[7]Овај репозиторијум налази се на следећој адреси: `https://repo.maven.apache.org/maven2/`.

[8]Овај репозиторијум налази се на следећој адреси: `https://plugins.gradle.org/m2`.

[9]Водич за овај стил може се наћи на следећој адреси: `https://google.github.io/styleguide/javaguide.html`.

[10]Званични веб-сајт интегрисаног развојног окружења IntelliJ IDEA налази се на следећој адреси: `https://www.jetbrains.com/idea/`.

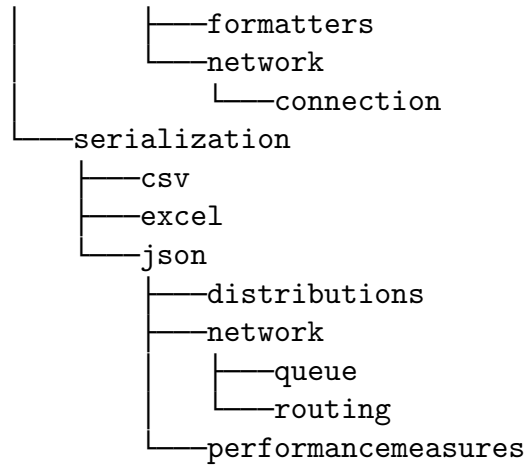
оквиру овог окружења), те се може уређивати помоћу било ког другог окружења или чак и помоћу обичног текстуалног уређивача.

За генерисање FXML датотека коришћен је алат Scene Builder[11] који омогућава креирање и уређивање FXML датотека по принципу WYSIWYG (*What You See Is What You Get*), што знатно убрзава и олакшава рад у односу на ручно писање ових датотека.

Како аутор овог рада има власништво над доменом `askov.dev`, корени пакет апликације је, у складу са конвенцијама у индустрији, назван `dev.askov.vipet`. У наставку су наведени сви потпакети овог пакета:

```
dev
└── askov
    └── vipet
        ├── common
        ├── configuration
        ├── core
        │   ├── analyzer
        │   │   └── exceptions
        │   ├── performancemeasures
        │   └── simulator
        │       ├── discrete
        │       └── sampling
        │           ├── analysis
        │           └── generation
        │               ├── network
        │               └── servicecenter
        ├── localization
        ├── mvc
        │   ├── common
        │   ├── controllers
        │   │   ├── analysis
        │   │   ├── converters
        │   │   ├── distributions
        │   │   └── networkeditor
        │   │       └── states
        │   ├── models
        │   │   ├── analyzer
        │   │   ├── distributions
        │   │   │   └── custom
        │   │   ├── network
        │   │   │   ├── queue
        │   │   │   └── routing
        │   │   └── simulation
        │   └── views
        │       ├── components
```

[11] Званични веб-сајт алата Scene Builder налази се на следећој адреси: `https://gluonhq.com/products/scene-builder/`.



У наставку следи кратак преглед најважнијих библиотека које се користе у оквиру апликације VIPET:

- Ikonli — библиотека која обезбеђује приступ великом броју икона;
- Apache Commons Math — библиотека која пружа широк спектар математичких функција и имплементације квалитетних генератора псеудо-случајних бројева;
- JLaTeXMath — библиотека која омогућава приказивање математичких формула написаних у LaTeXсинтакси;
- Gson — библиотека за серијализацију и десеријализацију Java објеката коришћењем формата JSON;
- OpenCSV — библиотека за рад са CSV датотекама;
- Apache POI — библиотека за рад са Microsoft Excel датотекама;
- SLF4J — фасада за различите библиотеке за логовање у Java апликацијама;
- Log4j — библиотека за логовање;
- JUnit — библиотека за јединично тестирање.

6.3 CI/CD

Континуална интеграција и континуална испорука (енгл. *Continuous Integration / Continuous Delivery*, CI/CD) представља скуп техника и приступа у изради софтвера у оквиру којих се изворни код након сваке промене *аутоматски* преводи, тестира, и пушта у продукцију (тј. испоручује корисницима). Циљ ових метода је да се обезбеди *брзо* и *поуздано* испоручивање нових верзија софтвера корисницима, уз минималан ризик од грешака и проблема.

У случају апликације VIPET, успостављен је процес континуалне интеграције и континуалне испоруке (енгл. *CI/CD pipeline*) коришћењем услуге GitHub Actions која је део платформе GitHub. Дефинисана су два радна тока (енгл. *workflows*) који се покрећу на основу одређених догађаја у репозиторијуму:

- `ci.yml` — овај радни ток се покреће на сваки *push* на главну грану (енгл. *main branch*) као и на сваки *pull request* који циља главну грану; обухвата кораке као што су преузимање изворног кода, инсталација JDK, покретање Gradle `build` задатка и постављање (енгл. *upload*) резултата тестирања за сваки од следећих оперативних система: Windows, Ubuntu и macOS;

- `release.yml` — овај радни ток се покреће када се креира нова ознака (енгл. *tag*) у репозиторијуму у формату

  `v<major>.<minor>.<patch>`[12];

  овај радни ток обухвата све што је наведено у оквиру радног тока `ci.yml`, и, под условом да су сви кораци успешно завршени, креира се ново издање апликације (енгл. *release*) на платформи GitHub; издање апликације обухвата:

  – садржај репозиторијума компресован у оквиру `.zip` архиве; овај формат је уобичајено коришћен на оперативном систему Windows;

  – садржај репозиторијума компресован у оквиру `.tar.gz` архиве; овај формат је уобичајено коришћен на оперативним системима Linux и macOS;

  – стандардни Microsoft инсталациони програм (`.msi`) који омогућава веома лаку инсталацију апликације на оперативном систему Windows; двоструким притиском на ову датотеку покреће се чаробњак (енгл. *wizard*) са графичким корисничким интерфејсом који води корисника кроз процес инсталације;

  – стандардни Debian пакет (`.deb`) који омогућава инсталацију апликације на различитим дистрибуцијама оперативног система Linux које користе овај формат пакета; инсталација се врши помоћу одговарајућег алата за управљање пакетима (енгл. *package manager*; на пример `apt`) и обично захтева администраторске привилегије;

  – `AppImage` фајл који омогућава покретање апликације без претходне инсталације на великом броју дистрибуција оперативног система Linux; довољно је да се фајл учини извршним (на пример, командом `chmod +x <appImage>`) и да се двоструким притиском на датотеку покрене апликација; овај формат се често назива *преносивим* (енгл. *portable*), јер се апликација може покретати са било ког места у фајл систему, па чак и са преносивих медија као што су USB флеш дискови.

Радни ток `release.yml` се ослања на Gradle задатак `jpackage` који најпре (1) искористи алат `jlink` за креирање *прилагођеног* извршног окружења које садржи минималан скуп модула потребних за рад апликације (чиме се смањује величина извршне дистрибуције и елиминише потреба да корисник има претходно инсталиран JRE на свом систему), а затим (2) покрене истоимени алат (`jpackage`) који на основу резултата рада

[12] `<major>`, `<minor>` и `<patch>` су бројеви чије је значење дефинисано у оквиру документа „Семантичко верзионисање” које се може наћи на следећој адреси: `https://semver.org/lang/sr/`; аутор ове спецификације је већ поменути Том Престон-Вернер.

алата `jlink` аутоматски генерише `.msi` и `.deb` датотеке за инсталацију апликације. `AppImage` фајл се креира коришћењем алата `appimagetool`[13], тако што се прво прилагоди директоријумска структура коју генерише `jlink` тако да она одговара структури коју очекује алат `appimagetool`, а затим се позове и сам алат. На овај начин су задовољени захтеви [REQ-NFN-01] и [REQ-NFN-02].

[13]Званични репозиторијум овог алата налази се на следећој адреси: `https://github.com/AppImage/appimagetool`.

Поглавље 7

ВЕРИФИКАЦИЈА И ВАЛИДАЦИЈА АПЛИКАЦИЈЕ

Верификација, односно провера да ли је апликација исправно реализована у односу на захтеве дефинисане у поглављу 5, извршена је мануелном провером сваког захтева. Сви захтеви су реализовани, а у наставку су приложене слике екрана (енгл. *screenshots*) које илуструју неке од важнијих случајева употребе апликације.

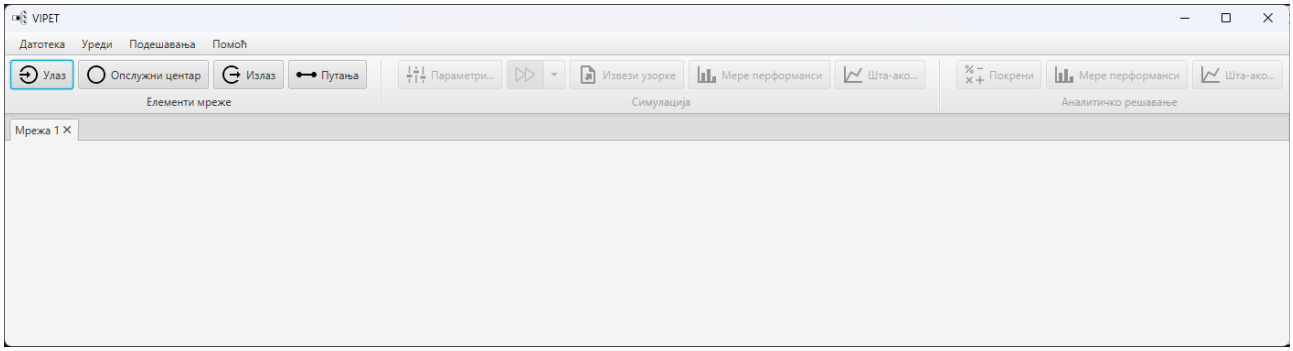
Валидација, односно провера да ли апликација заиста испуњава своју сврху, извршена је у оном делу који се односи на тачност резултата добијених реализованим аналитичким методама и симулацијом заснованом на дискретним догађајима. За валидацију су коришћене мреже са опслужним центрима за које су познати тачни резултати добијени аналитичким методама. JSON спецификације ових мрежа налазе се у директоријуму `src/test/resources/dev/askov/vipet/serialization/json/networks`, док се одговарајући аналитички резултати налазе у директоријуму `src/test/resources/dev/askov/vipet/serialization/json/performancemeasures`. Пакет `dev.askov.vipet.core` у директоријуму `src/test/java` садржи:

- пакет `analyzer` који садржи:
  - класу `JacksonAnalyzerTest` којом се тестира аналитичко решавање Џексонових отворених мрежа;
  - класу `GordonNewellAnalyzerTest` којом се тестира аналитичко решавање Гордон–Њуелових затворених мрежа;
- пакет `simulator.discrete` који садржи класу `DiscreteEventSimulatorTest` којом се тестира симулација отворених и затворених мрежа.

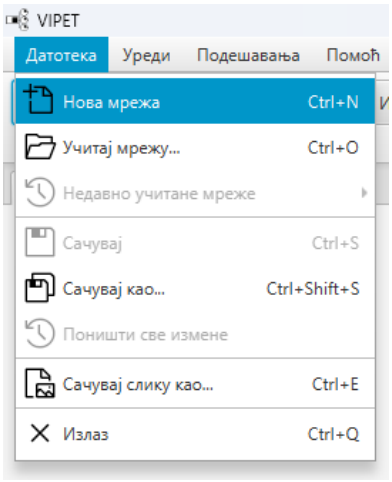
Мере перформанси добијене аналитичким методама се упоређују директно са очекиваним вредностима, док се резултати добијени симулацијом упоређују тако што се проверава да ли 99% интервали поверења којима се оцењују мере перформанси садрже очекиване вредности. Симулација за сваку мрежу се покреће са довољно дугим временом извршавања и довољан број пута како би се обезбедило да се очекиване вредности налазе у добијеним интервалима поверења.

7.1 Креирање нове мреже

Главни прозор апликације који се кориснику приказује непосредно након покретања апликације приказан је на слици 7.1. Прозор има све елементе наведене у [REQ-GUI--02] и [REQ-GUI-04]. Корисник може *одмах* да започне додавање жељених елемената у мрежу коришћењем дугмади из групе Елементи мреже на траци са алаткама. Међутим, уколико је корисник завршио са изменама тренутног модела и жели да креира нову мрежу без затварања тренутно активне, то може започети, у складу са [REQ-FN-01], избором ставке Нова мрежа из менија Датотека или коришћењем комбинације тастера Ctrl + N (видети слику 7.2). Изглед апликације након додавања нове мреже приказан је на слици 7.3.



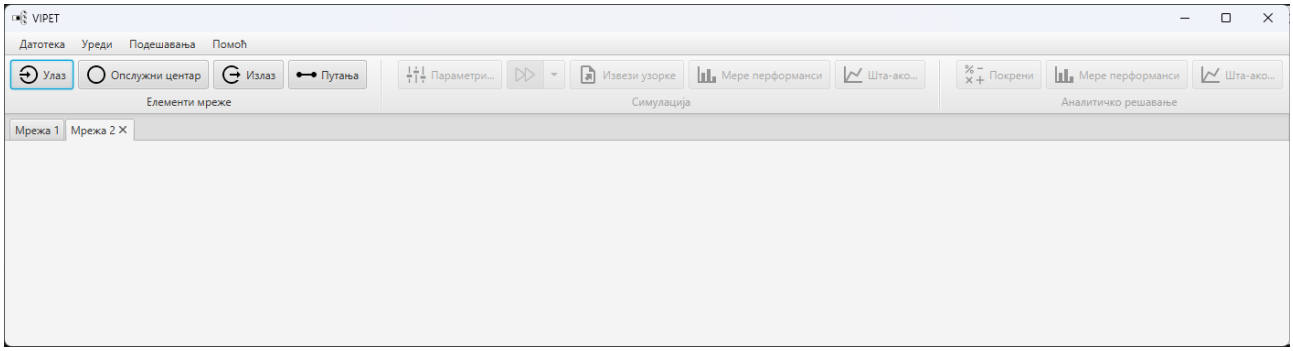
Слика 7.1: *Главни прозор апликације.*



Слика 7.2: *Избор ставке Нова мрежа из менија Датотека.*

7.2 Додавање новог елемента мреже

На слици 7.4а приказан је изглед апликације након додавања неколико елемената у мрежу са слике 7.3. На слици 7.4б приказан је изглед опслужног центра Опслужни центар 3 док се апликација налази у режиму додавања елемента мреже и, истовремено, курсор миша се налази на позицији на којој није дозвољено додавање елемента мреже. На слици



Слика 7.3: *Изглед апликације након избора ставке* Нова мрежа *из менија* Датотека.

7.4в приказан је исти опслужни центар, али је курсор миша померен на позицији на којој јесте дозвољено додавање елемента мреже. На слици 7.4г приказан је изглед апликације након што је нови елемент мреже додат и након што је апликација изашла из режима додавања елемента мреже; курсор миша се налази изнад новододатог елемента мреже, те је видљив и помоћни натпис за овај елемент мреже. Овакво понашање јесте у складу са [REQ-FN-11].

7.3 Измена постојећег елемента мреже

На слици 7.5а приказан је главни прозор апликације са активним контекстним менијем који је последица притиска десног тастера миша док се курсор налази изнад елемента Опслужни центар 2. На слици 7.5б може се видети измењена мрежа након избора ставке Ротирај из претходно приказаног контекстног менија (видети [REQ-FN-14]).

На слици 7.6а приказан је модални дијалог за измену својстава опслужног центра Опслужни центар 2 након избора ставке Својства... у контекстном менију приказаном на слици 7.5а (видети [REQ-FN-08]). На слици 7.6б приказан је исти модални дијалог након отварања картице Сервис. На слици 7.6в приказан је угнежђени модални дијалог за измену параметра расподеле времена опслуживања након притиска на дугме ... у претходно приказаном модалном дијалогу.

7.4 Покретање аналитичког решавања

На слици 7.7а приказан је главни прозор апликације након што је корисник изменио називе опслужних центара, дефинисао боје линије и боје попуњавања свих елемената мреже Мрежа 2 (чиме је сваком од њих дефинисан визуелни идентитет) и повезао све елементе мреже путањама. На слици 7.7б приказан је главни прозор апликације након завршетка аналитичког решавања модела које је корисник покренуо притиском на дугме Покрени у оквиру групе Аналитичко решавање на траци са алаткама (видети [REQ-FN-22]). На слици 7.8 приказан је модални прозор Аналитичко решавање који се појављује након завршетка аналитичког решавања модела (паралелно са већ поменутом визуелизацијом са слике 7.7б), а који изгледа у складу са [REQ-GUI-14]. Као што се може видети, у левом делу прозора је приказан начин на који се долази до протока кроз све опслужне центре у мрежи, док садржај на десној страни прозора зависи од тренутно изабраног опслужног центра (подразумевано је изабран опслужни центар Опслужни центар

1), а укључује рачунање свих мера перформанси које су дефинисане у [REQ-GUI-14]. На слици 7.9 приказан је поново исти модални прозор, али након што је корисник изабрао насловљени панел Вероватноће стања.

На слици 7.10 приказан је модални прозор Аналитичко решавање, али након што је корисник изабрао опслужни центар Улазно-излазни уређај из падајуће листе; на слици 7.11 приказан је исти модални прозор, али након што је корисник изабрао насловљени панел Вероватноће стања.

На слици 7.12 приказан је модални прозор Аналитичко решавање, али након што је корисник изабрао опслужни центар Диск из падајуће листе; на слици 7.13 приказан је исти модални прозор, али након што је корисник изабрао насловљени панел Вероватноће стања.

7.5 Преглед резултата аналитичког решавања

На слици 7.14 приказан је модални прозор за преглед мера перформанси који се може отворити у било ком тренутку након затварања модалног прозора приказаног на слици 7.8 (видети [REQ-FN-23]), уз услов да корисник не промени мрежу на начин који би утицао на мере перформанси модела (додавање или брисање елемента мреже, промена броја сервера у оквиру неког постојећег сервисног центра, промена параметара расподела времена између долазака послова или времена опслуживања, итд.). Подразумевано је активна картица Графикон која омогућава лако упоређивање исте метрике за све елементе мреже.

На слици 7.15 приказан је исти модални прозор као на слици 7.14, али након што је корисник изабрао картицу Табела.

7.6 Покретање „Шта-ако" анализе базиране на аналитичком решавању мреже

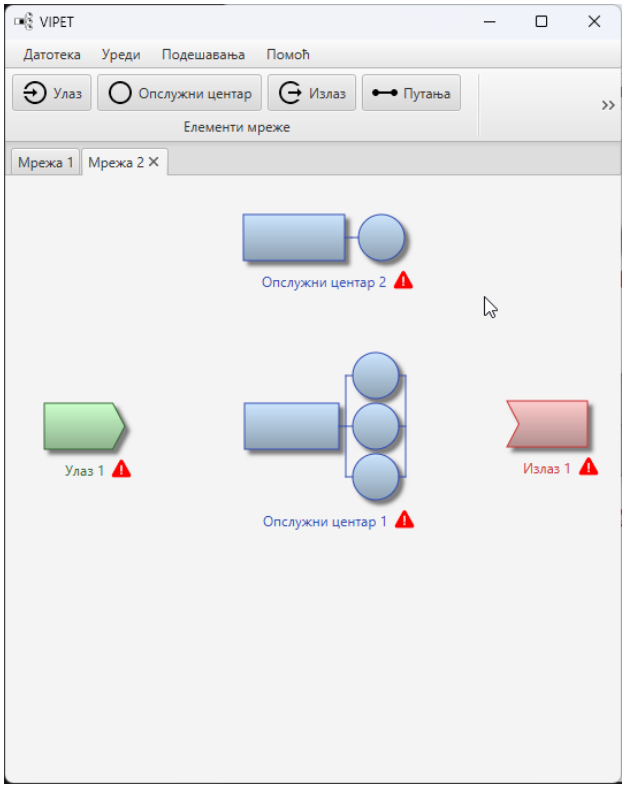
На слици 7.16 приказан је модални прозор за „шта-ако" анализу (видети [REQ-FN--25]). На слици 7.17 приказан је изглед модалног прозора за „шта-ако" анализу након што је корисник изабрао Процесор у оквиру падајуће листе Чвор у мрежи, Број сервера у оквиру падајуће листе Променљива, 3 у оквиру текстуалног поља Почетна вредност, 10 у оквиру текстуалног поља Крајња вредност и након притиска корисника на дугме Покрени.

7.7 Покретање интерактивне симулације

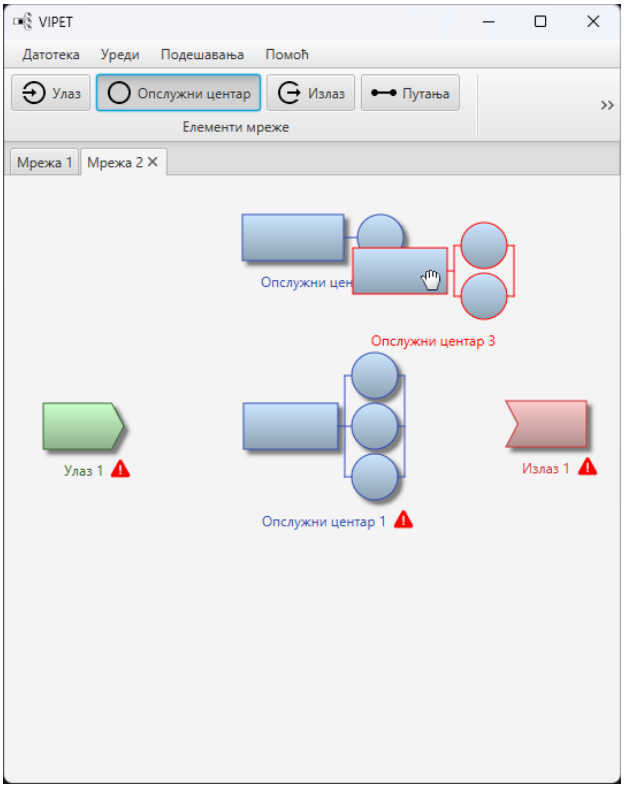
На слици 7.18 приказан је прозор Параметри симулације који се појављује након притиска на дугме Параметри... из групе Симулација у оквиру траке са алаткама (видети [REQ-FN-18]). Улазне контроле приказане на слици попуњене су подразумеваним вредностима.

На слици 7.19 приказан је главни прозор апликације након покретања интерактивне симулације (видети [REQ-FN-19]) у тренутку када је тренутно време симулације 943 ms 139 µs. На слици 7.20 приказан је главни прозор апликације у неком тренутку након првог наредног притиска на дугме Следећи догађај, а за време трајања анимације преласка

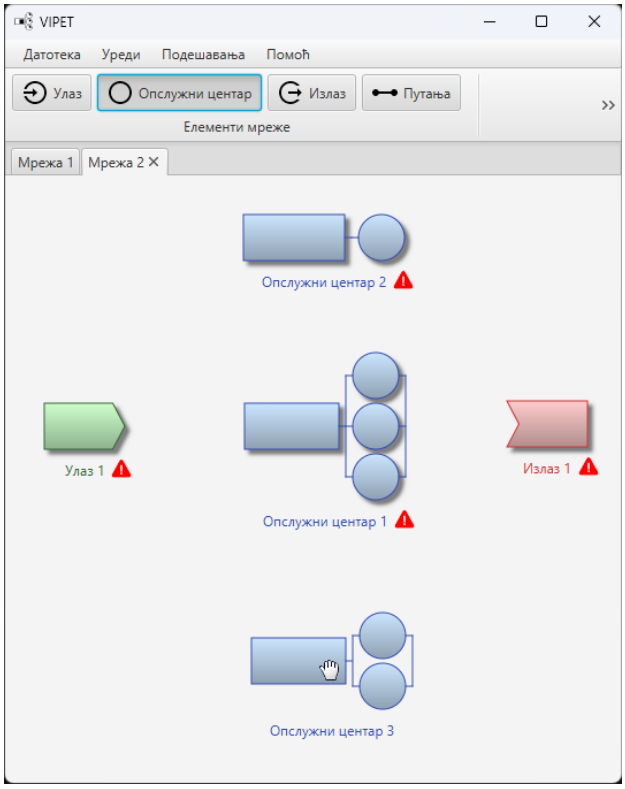
посла са елемента мреже Диск на елемент мреже Процесор. На слици 7.21 приказан је главни прозор апликације након завршетка поменуте анимације. Као што се може видети, број послова у реду за чекање елемента мреже Процесор се повећао за један, док је сервер елемента мреже Диск на коме се посао обрађивао постао слободан.



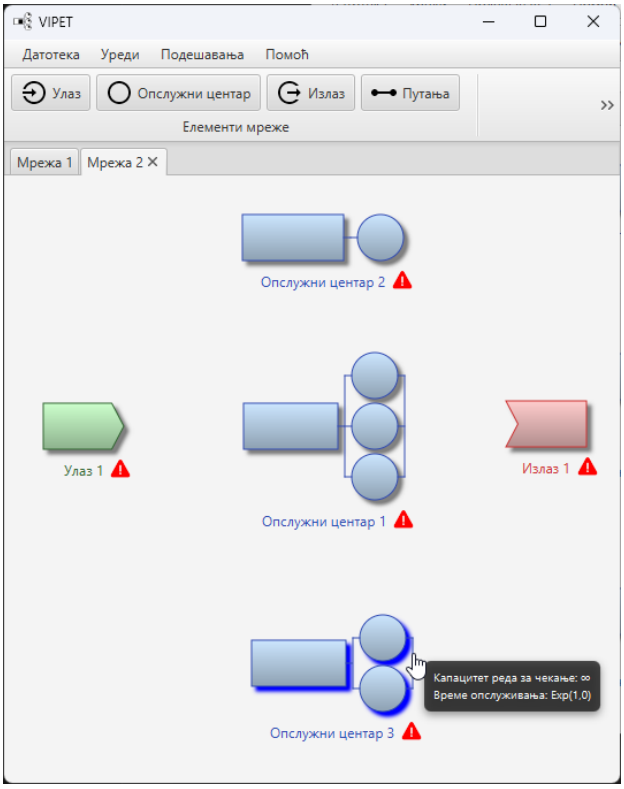
(а) *Режим чекања на кориснички унос (енгл. idle).*



(б) *Режим додавања елемента мреже, при чему одредишна позиција није дозвољена.*

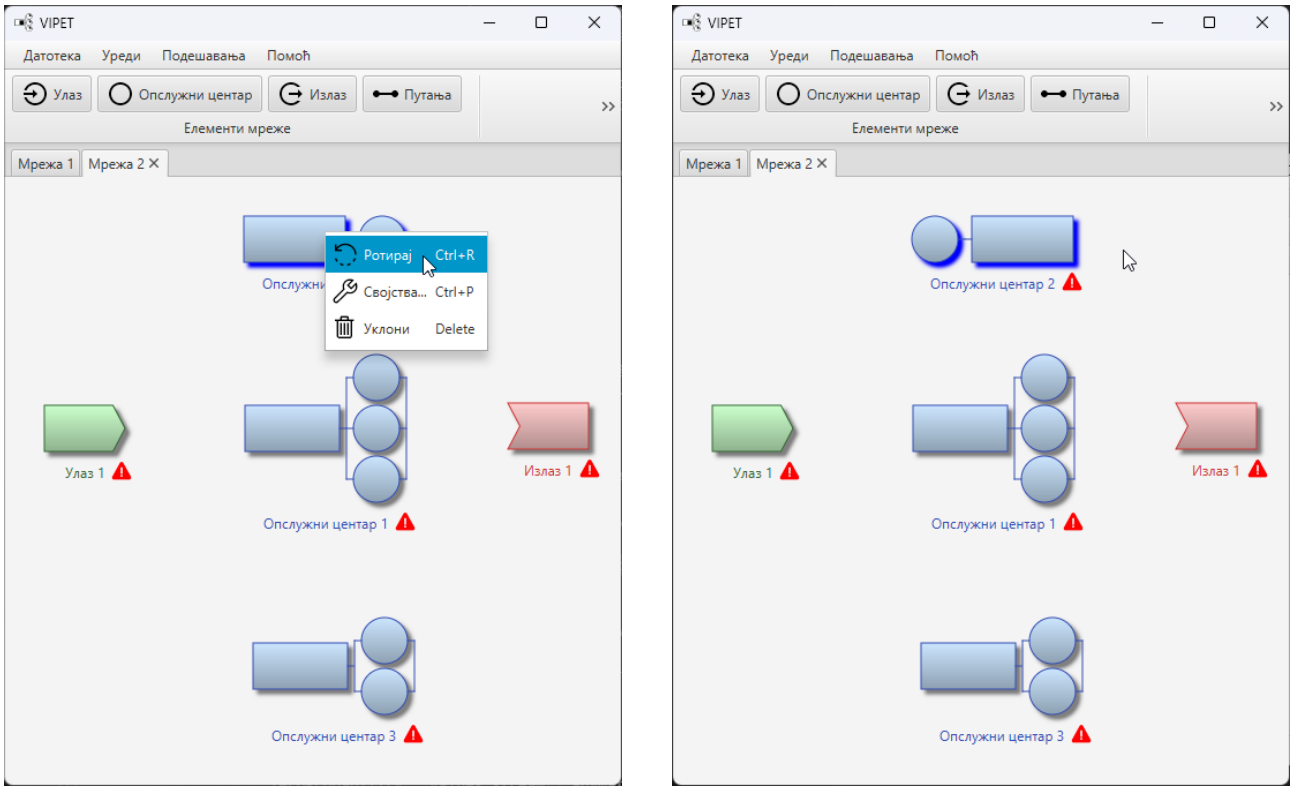


(в) *Режим додавања елемента мреже, при чему одредишна позиција јесте дозвољена.*



(г) *Режим чекања на кориснички унос, при чему је курсор миша изнад елемента мреже.*

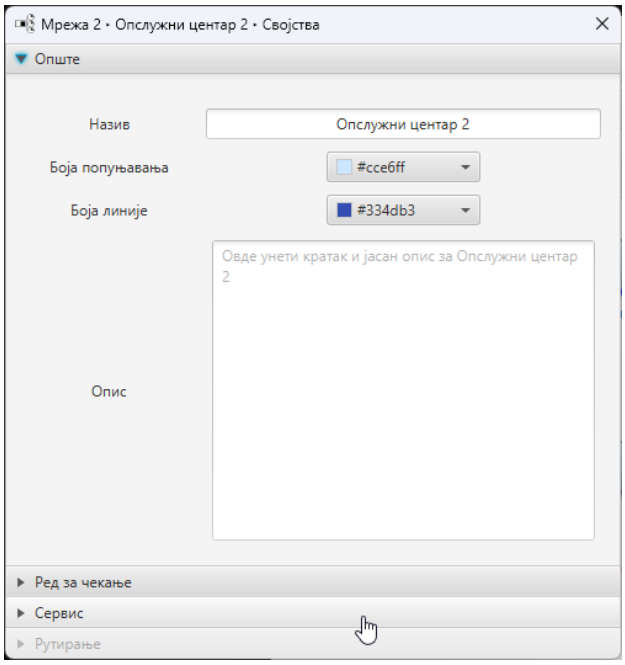
Слика 7.4: *Додавање новог елемента мреже.*



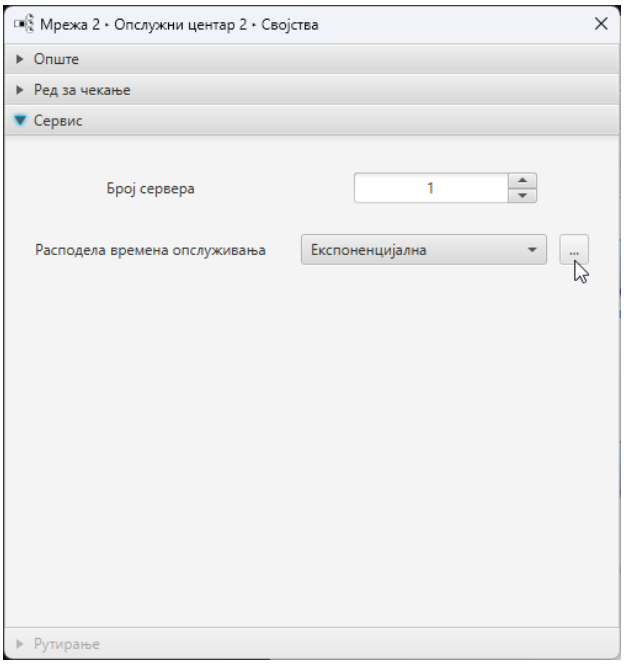
(а) *Активан контекстни мени изнад елемента мреже.*

(б) *Измењена мрежа након избора ставке* Ротирај *у контекстном менију.*

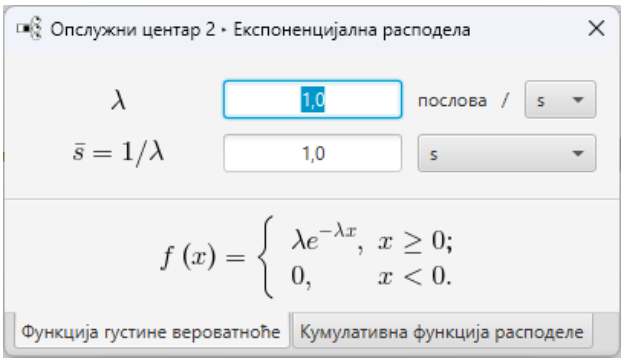
Слика 7.5: *Промена оријентације елемента мреже.*



(а) *Модални дијалог за измену својстава опслужног центра.*

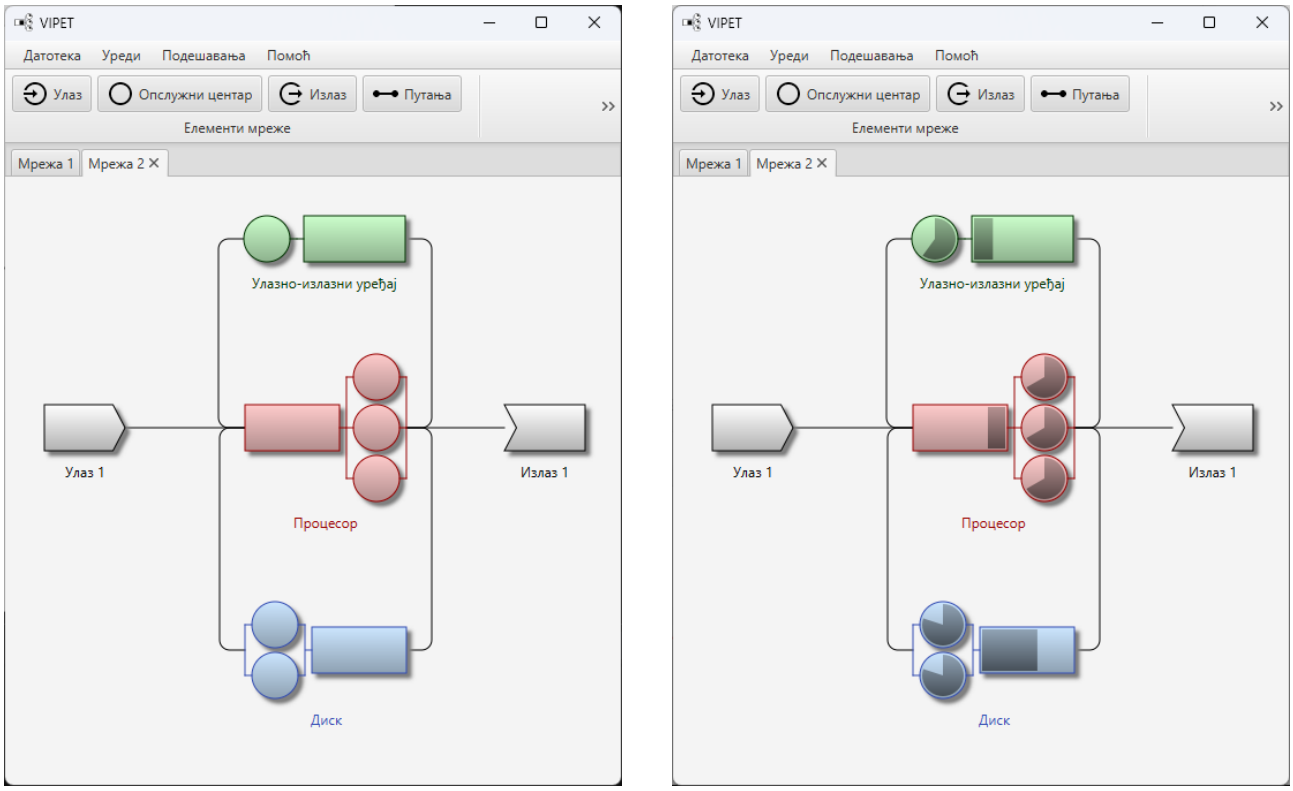


(б) *Модални дијалог за измену својстава опслужног центра након отварања картице* Сервис.



(в) *Уграђени модални дијалог за измену параметра расподеле времена опслуживања.*

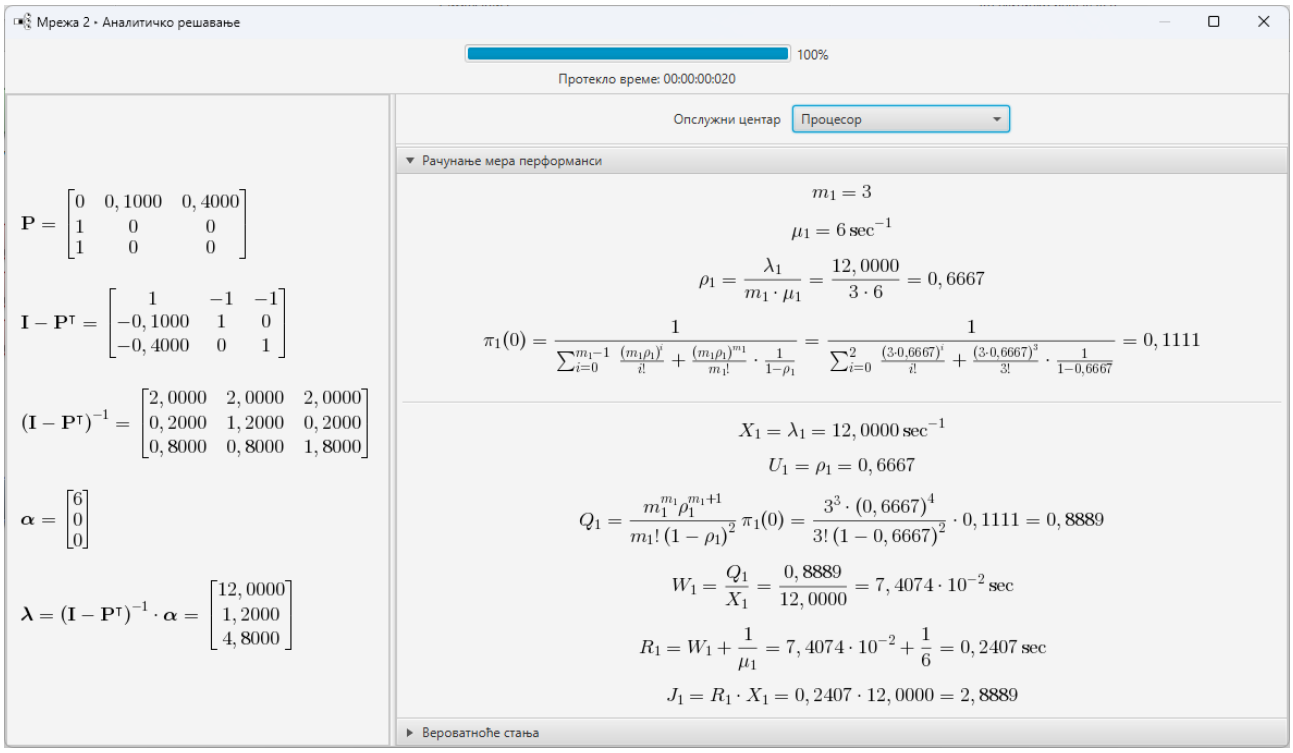
Слика 7.6: *Измена параметра расподеле времена опслуживања сервисног центра.*



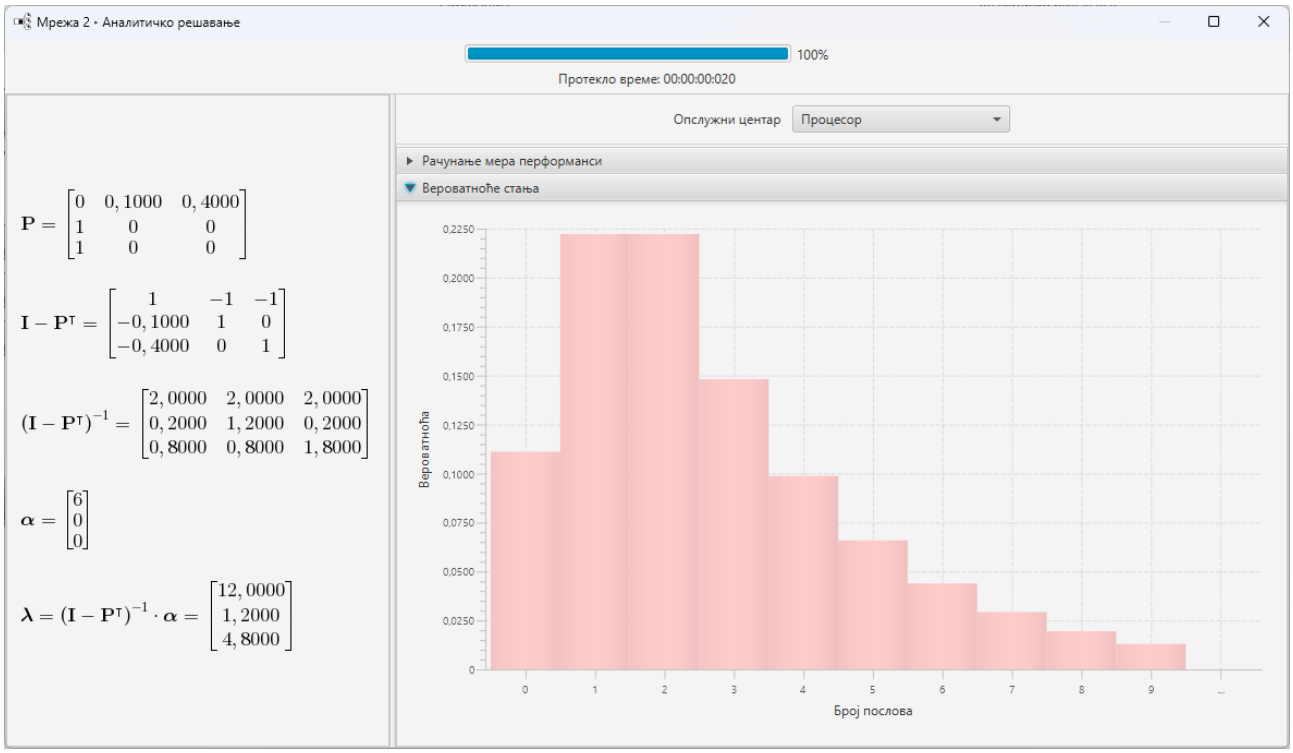
(а) *Главни прозор апликације са измењеним визуелним идентитетом елемената мреже.*

(б) *Главни прозор апликације након завршетка аналитичког решавања модела.*

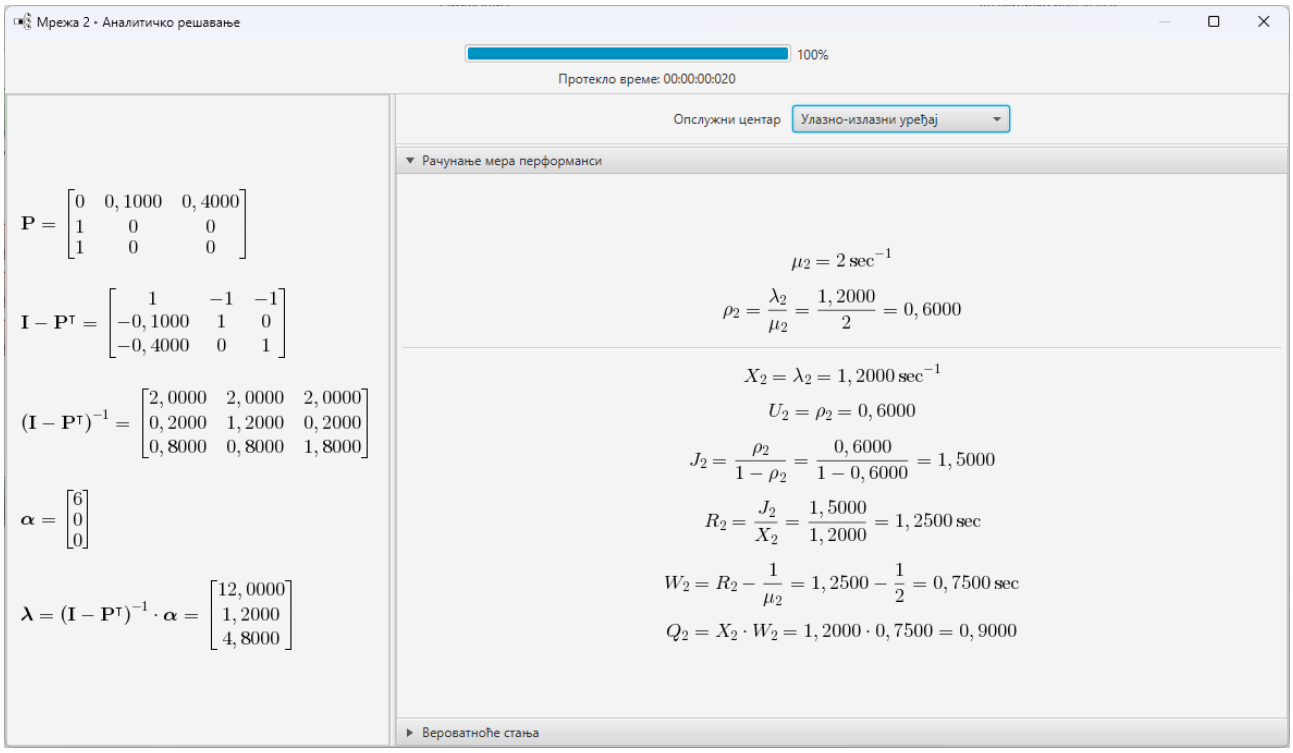
Слика 7.7: *Покретање аналитичког решавања модела.*



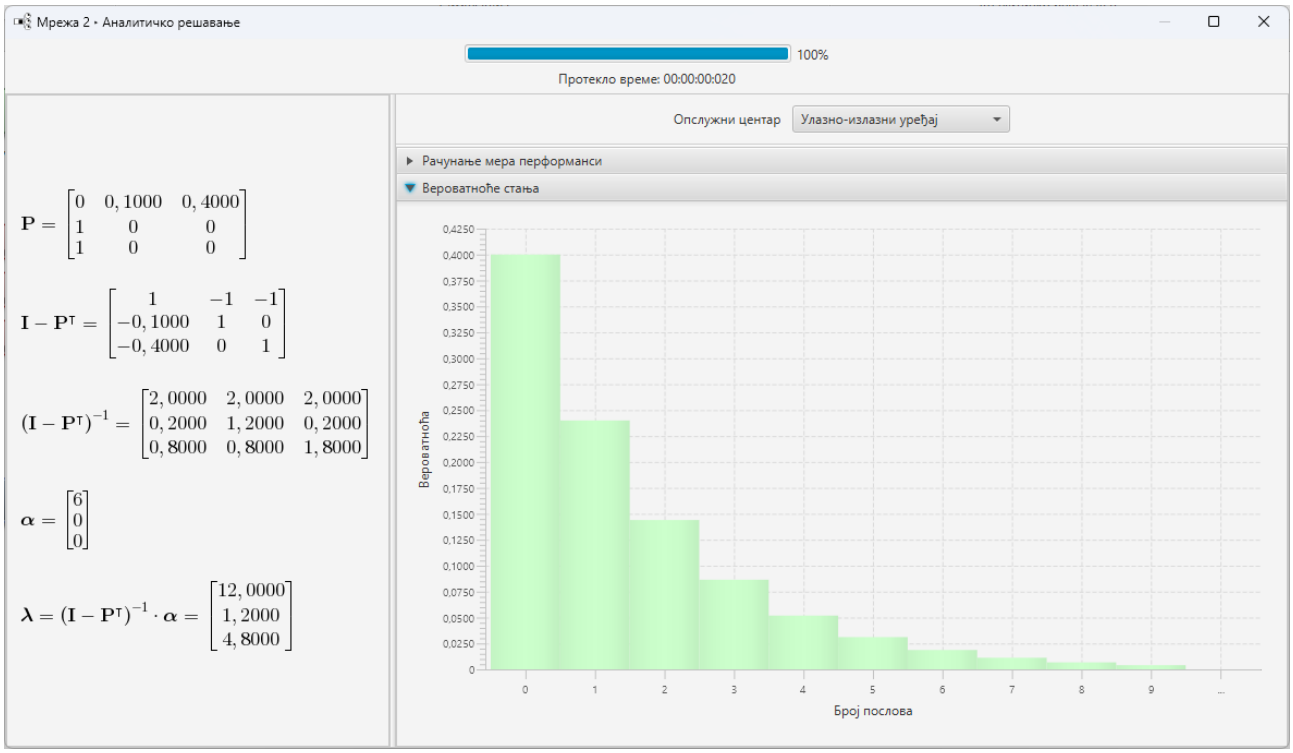
Слика 7.8: *Модални прозор након завршетка аналитичког решавања модела, са изабраним опслужним центром* Опслужни центар 1.



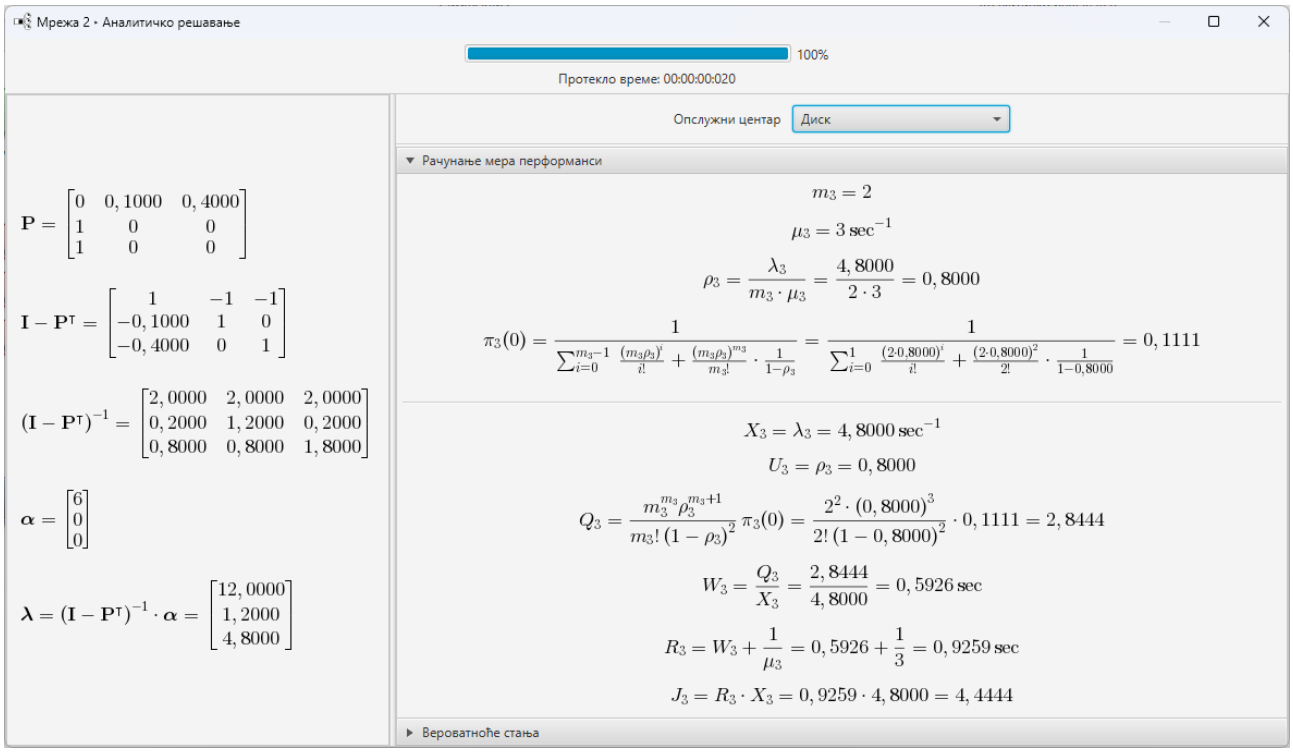
Слика 7.9: *Модални прозор након завршетка аналитичког решавања модела, са изабраним опслужним центром* Опслужни центар 1 *и картицом* Вероватноће стања.



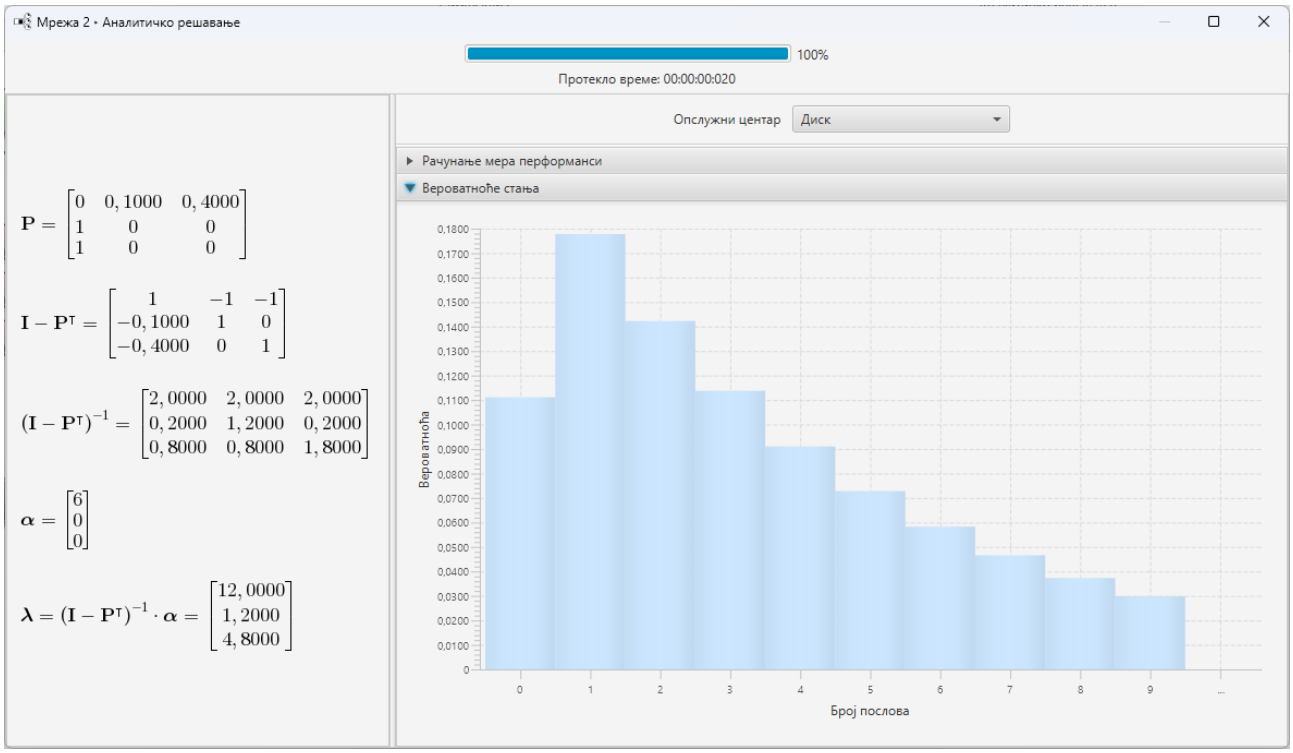
Слика 7.10: *Модални прозор након завршетка аналитичког решавања модела, са изабраним опслужним центром* Улазно-излазни уређај.



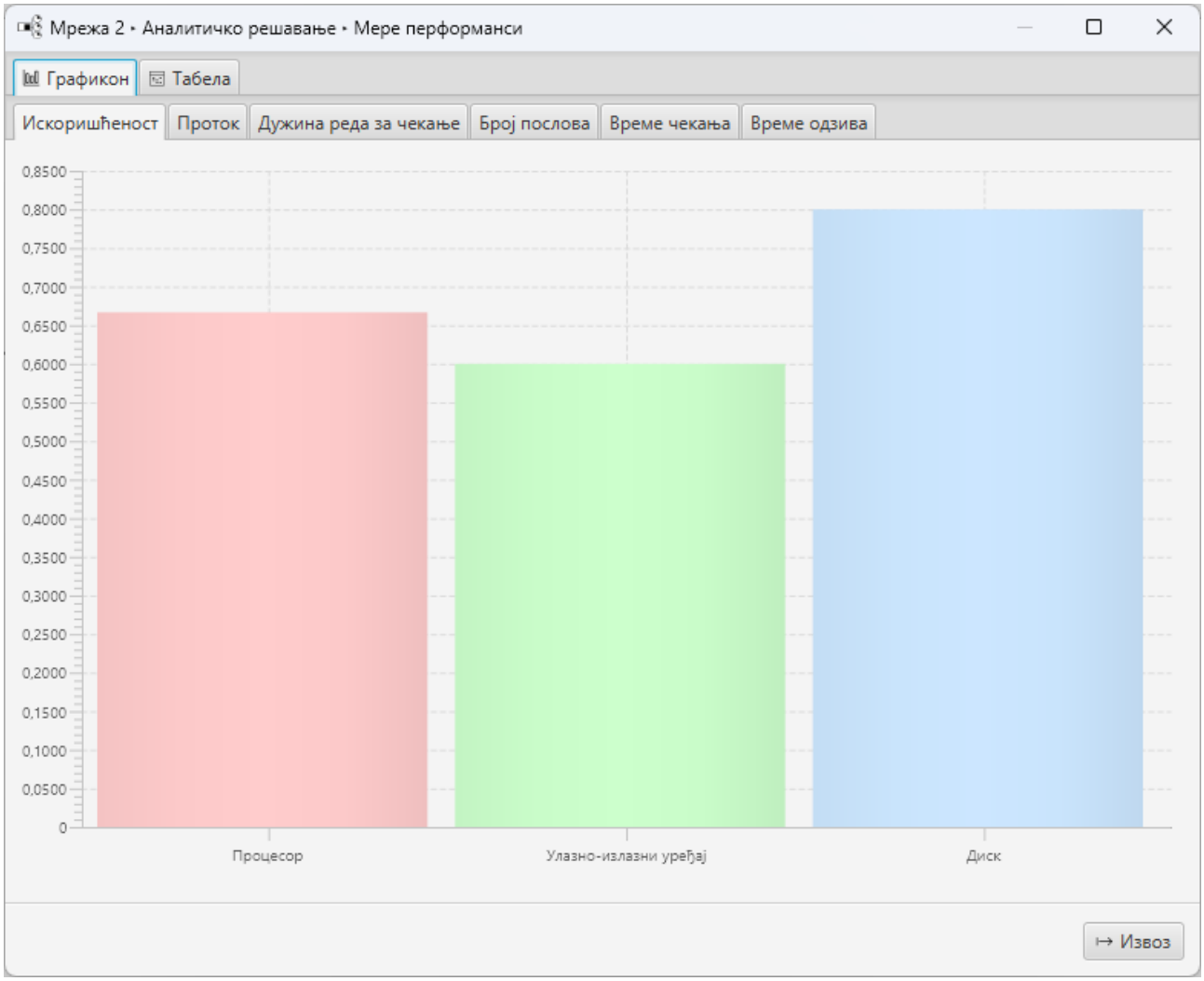
Слика 7.11: *Модални прозор након завршетка аналитичког решавања модела, са изабраним опслужним центром* Улазно-излазни уређај *и картицом* Вероватноће стања.



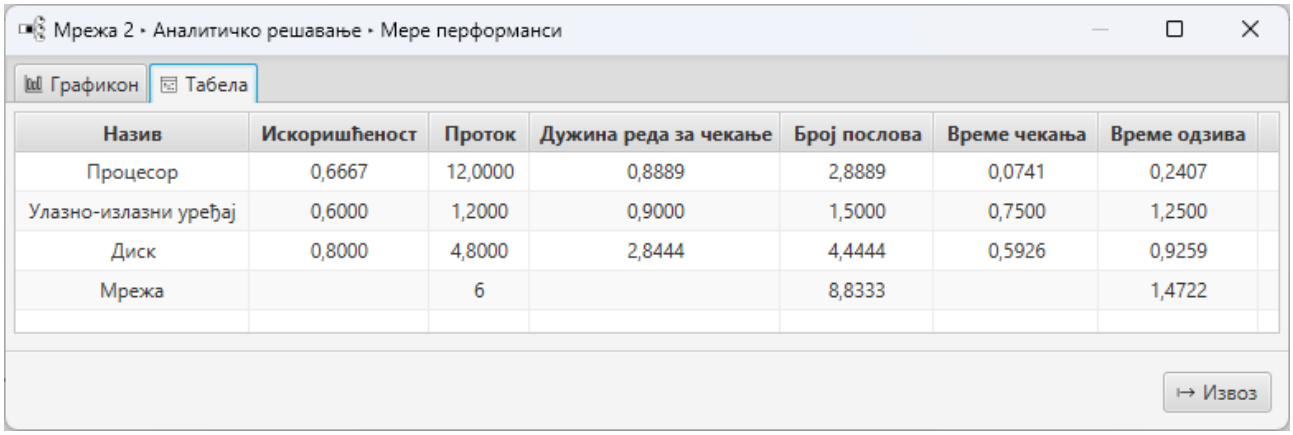
Слика 7.12: *Модални прозор након завршетка аналитичког решавања модела, са изабраним опслужним центром* Диск.



Слика 7.13: *Модални прозор након завршетка аналитичког решавања модела, са изабраним опслужним центром* Диск *и картицом* Вероватноће стања.

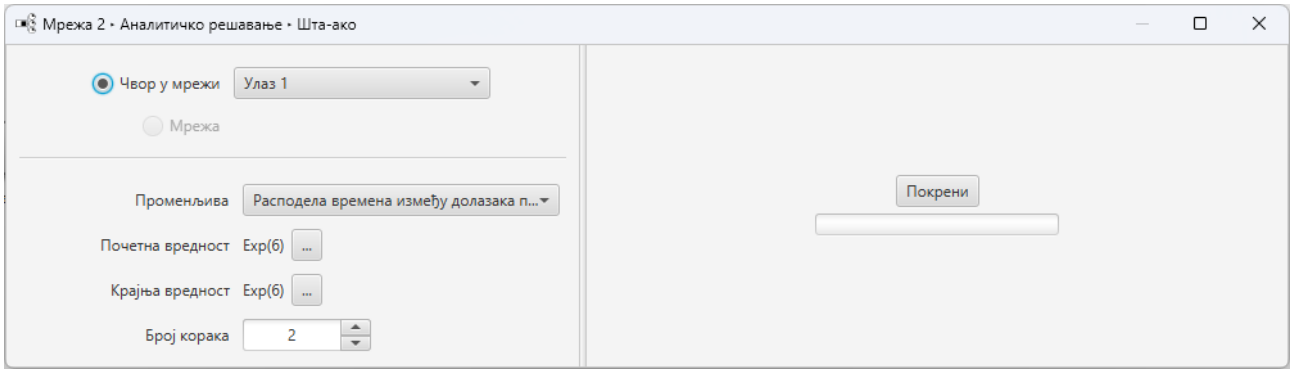


Слика 7.14: *Модални прозор за преглед мера перформанси, при чему је изабрана картица* Графикон.

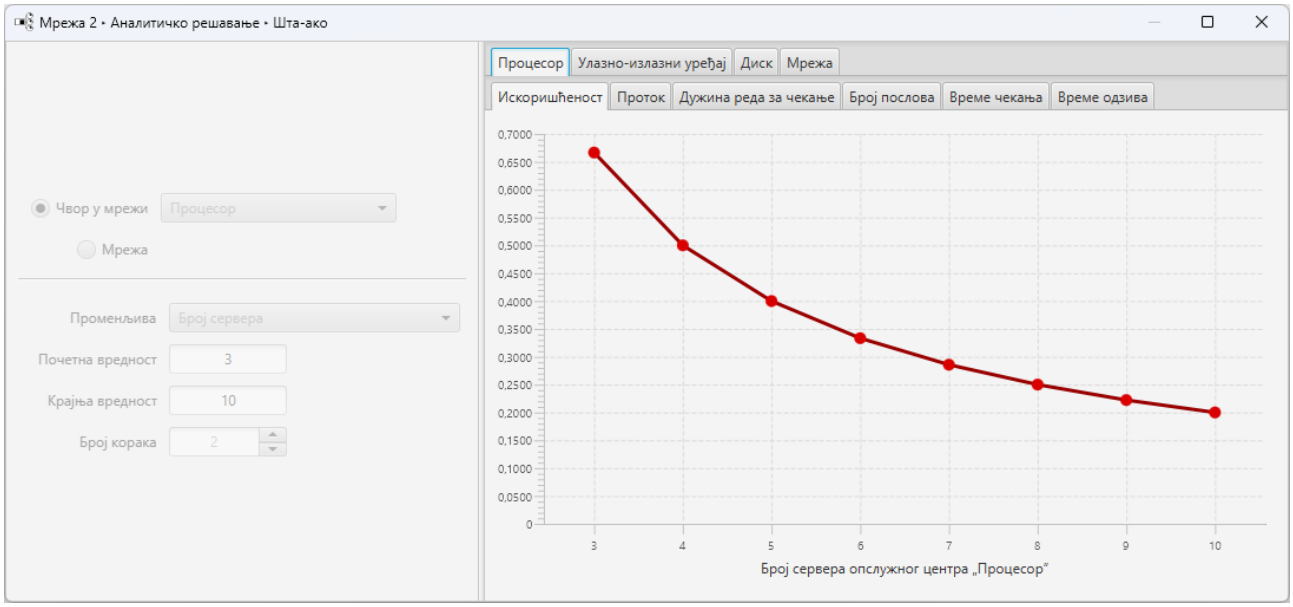


Назив	Искоришћеност	Проток	Дужина реда за чекање	Број послова	Време чекања	Време одзива
Процесор	0,6667	12,0000	0,8889	2,8889	0,0741	0,2407
Улазно-излазни уређај	0,6000	1,2000	0,9000	1,5000	0,7500	1,2500
Диск	0,8000	4,8000	2,8444	4,4444	0,5926	0,9259
Мрежа		6		8,8333		1,4722

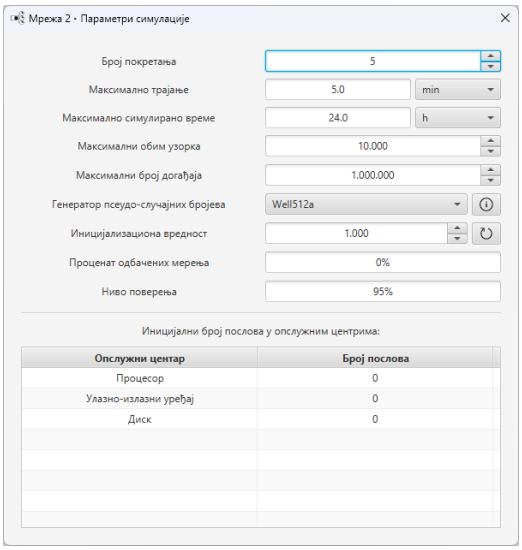
Слика 7.15: *Модални прозор за преглед мера перформанси, при чему је изабрана картица* Табела.



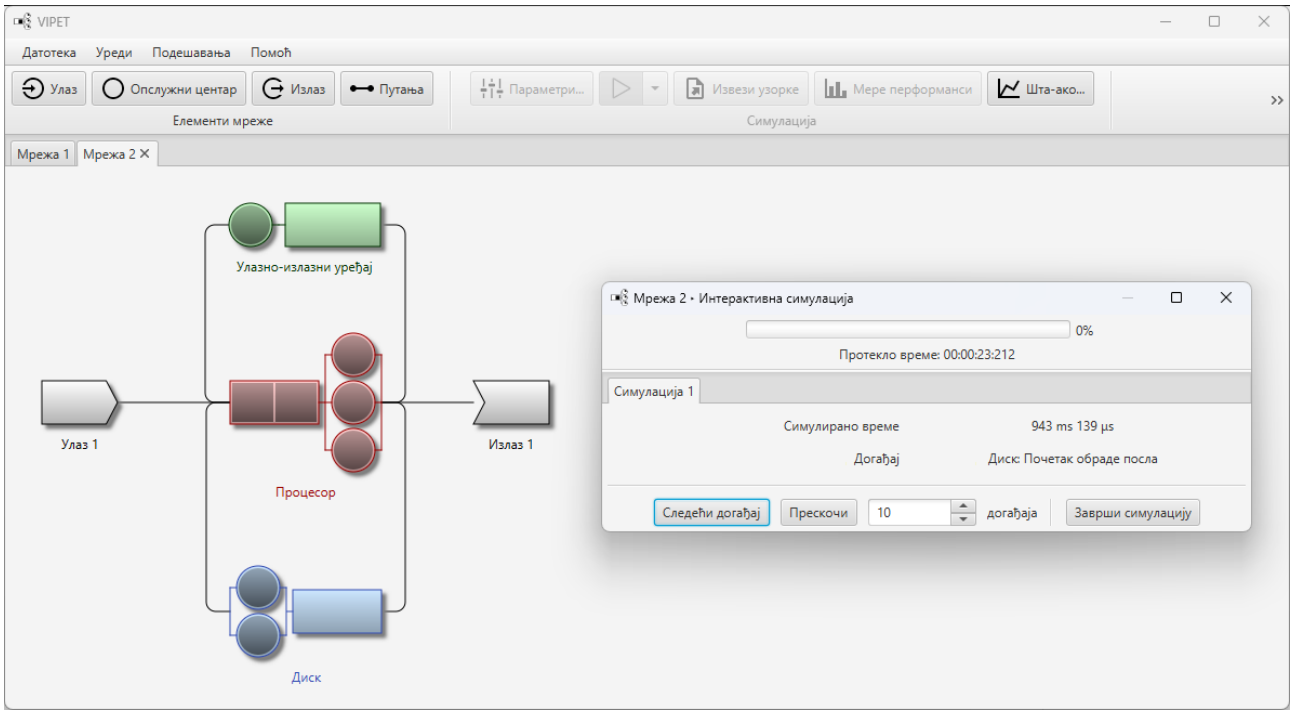
Слика 7.16: *Модални прозор за „шта-ако" анализу.*



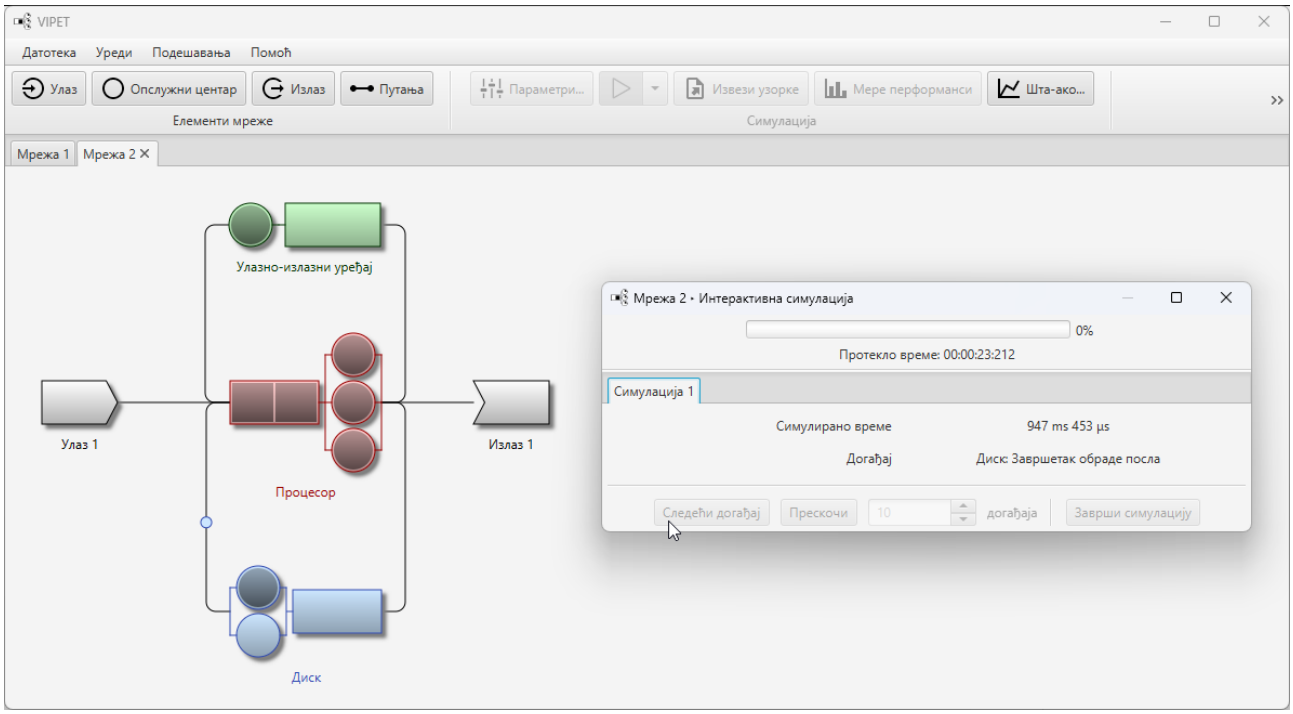
Слика 7.17: *Модални прозор за „шта-ако" анализу након покретања аналитичког решавања мреже.*



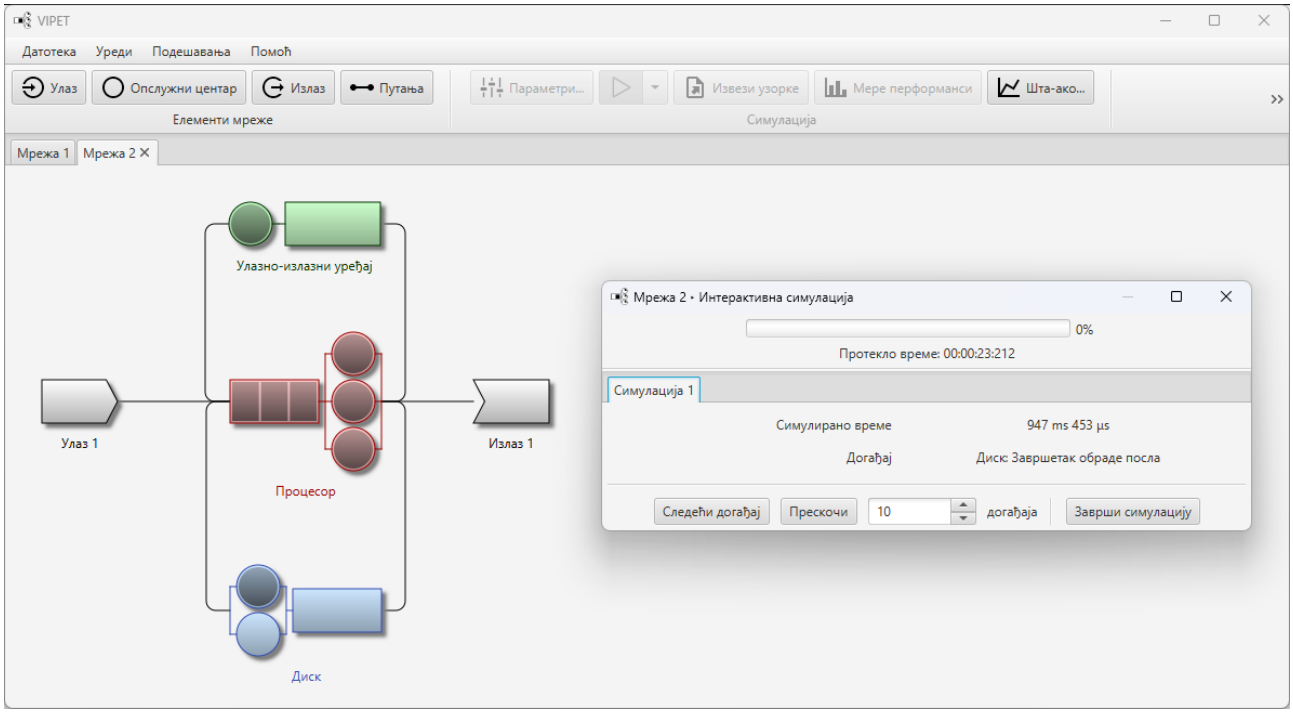
Слика 7.18: *Модални прозор* **Параметри симулације.**



Слика 7.19: *Главни прозор апликације током извршавања интерактивне симулације модела.*



Слика 7.20: *Главни прозор апликације након притиска на дугме* Следећи догађај.



Слика 7.21: *Главни прозор апликације након завршетка анимације преласка посла.*

Поглавље 8

ЗАКЉУЧАК

У овом раду реализован је интерактиван педагошки алат са графичким корисничким интерфејсом за лако креирање модела система на бази мрежа са опслужним центрима, као и одређивање релевантних мера перформанси моделираних система применом симулационих и аналитичких метода. Алат омогућава студентима да кроз практичан рад стекну дубље разумевање теоријских концепата представљених на предмету *Перформансе рачунарских система*, као и да експериментишу са различитим моделима и параметрима модела на једноставан и интуитиван начин.

Имајући у виду да је апликација пројектована на начин који омогућава да се њен скуп функционалности лако може проширити, неке идеје које могу послужити као инспирација за даљи развој апликације и будуће студентске радове наведене су у следећој листи (поређане су од најједноставније до најкомплексније за реализацију):

1. обавестити корисника о доступности нових верзија апликације и омогућити једноставно ажурирање апликације;

2. омогућити кориснику да бира које мере перформанси жели да добије симулацијом или аналитичким решавањем како би се смањило време израчунавања и количина оперативне меморије коју апликација користи;

3. омогућити кориснику да групише чворове у подмреже, што би допринело бољој прегледности и организацији модела; такође, могли би бити дефинисани још неки елементи мреже са специфичним понашањем и улогом у моделу;

4. реализација *Undo/Redo* функционалности која би подразумевала да се кориснику омогући да поништи или понови одређени број претходних радњи, а допринела би бољем корисничком искуству;

5. имплементација концепта *класе послова*, при чему припадност некој класи дефинише расподелу времена између долазака послова те класе у мрежу (ако је класа *отворена*), времена обраде послова те класе у оквиру опслужних центара, рутирање послова те класе кроз мрежу и број послова те класе у мрежи (ако је класа *затворена*); увођењем класа омогућило би се креирање *мешовитих мрежа*, односно мрежа које истовремено садрже послове који улазе у мрежу и, после обраде у опслужним центрима мреже, излазе из ње, као и послове који циркулишу искључиво унутар мреже;

6. имплементација још неких аналитичких метода (MVA алгоритам за затворене мреже, уз одговарајуће визуелно приказивање итерација алгоритма; подршка за BCMP[1] мреже итд.);

7. имплементација (1) напреднијих метода за статистичку обраду узорака добијених симулацијом и (2) аутоматског прекидања симулације у тренутку постизања захтеване тачности резултата.

[1]BCMP мреже (по ауторима *Baskett*, *Chandy*, *Muntz* и *Palacios*) представљају генерализацију Џексонових и Гордон–Њуелових мрежа. Омогућавају анализу система у којима опслужни центри могу имати различите дисциплине распоређивања и типове расподела времена опслуживања, уз очување стационарног решења у облику производа.

ЛИТЕРАТУРА

[1] ACM и IEEE Computer Society Joint Task Force. *Computer Engineering Curricula 2016: Curriculum Guidelines for Undergraduate Degree Programs in Computer Engineering*. Техн. изв. ACM и IEEE Computer Society, 2016. url: https://www.acm.org/binaries/content/assets/education/ce2016-final-report.pdf.

[2] Милан Меркле. *Вероватноћа и статистика*. Академска мисао, 2021.

[3] Gunter Bolch и др. *Queueing Networks and Markov Chains: Modeling and Performance Evaluation with Computer Science Applications*. Jonh Wiley & Sons, 2006.

[4] Павле Младеновић. *Вероватноћа и статистика*. Завод за уџбенике, 2019.

[5] Yue Jiang. *Lecture 9: The CLT and Confidence Intervals*. Duke University. 2022. url: https://www2.stat.duke.edu/courses/Spring22/sta102.001/slides/lec-9.html#24.

[6] Hisashi Kobayashi, Brian L. Mark и William Turin. *Probability, Random Processes, and Statistical Analysis*. Cambridge University Press, 2012. isbn: 9780521895446.

[7] J. R. Jackson. „Networks of Waiting Lines". У: *Operations Research* 5.4 (1957.), стр. 518–521. doi: 10.1287/opre.5.4.518.

[8] J. R. Jackson. „Jobshop-Like Queueing Systems". У: *Management Science* 10.1 (1963.), стр. 131–142. doi: 10.1287/mnsc.10.1.131.

[9] William J. Gordon и Gordon F. Newell. „Closed Queuing Systems with Exponential Servers". У: *Operations Research* 15.2 (1967.), стр. 254–265. doi: 10.1287/opre.15.2.254. url: https://doi.org/10.1287/opre.15.2.254.

[10] G. Casale и G. Serazzi. *JMT: Java Modelling Tools – User Manual*. Politecnico di Milano (DEIB) и Imperial College London. Јун 2024. url: https://jmt.sourceforge.net/Papers/JMT_users_Manual.pdf.

[11] Robert C. Martin. *Design Principles and Design Patterns*. Техн. изв. Object Mentor Inc., 2000. url: https://web.archive.org/web/20150906155800/http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf.

[12] Steve McConnell. *Code Complete*. 2. издање. Redmond, WA: Microsoft Press, јун 2004.